LLM 모델의 이해와 확장

강의 목표

LLM의 구조를 이해하고

이를 바탕으로

Reasoning · Multimodal · Physical AI로

확장한다

강의 목차

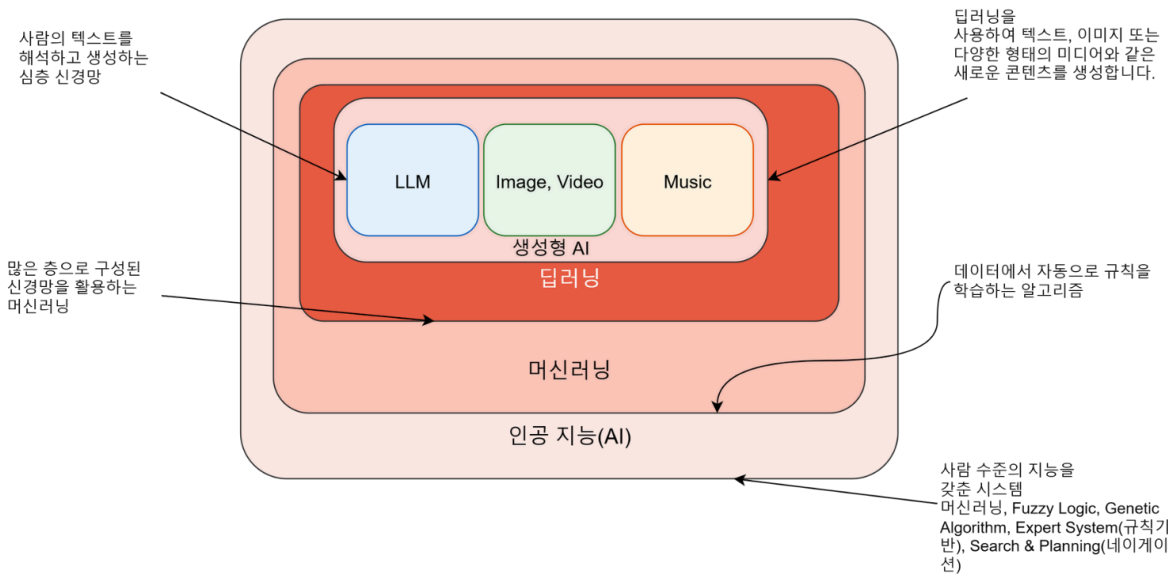
#	1일차 오전 주제
1	AI 모델의 역사
2	AI 기본 수학
3	언어 모델의 발전
4	언어 모델: 입력
5	Attention 이해
6	GPT Layer들
7	GPT 모델 입출력
8	지식의 저장
9	모델 학습
10	Fine-tuning

#	1일차 오후 ~ 2일차 주제
11	지시 미세 튜닝
12	추론 모델
13	LLM 효율화
14	MOE (Mixture of Experts)
15	Knowledge Distillation
16	Speculative Decoding
17	Vision LLM
18	Omni Model
19	Linear Attention
20	Diffusion Language Model
21	Physical AI

AI 모델의 역사

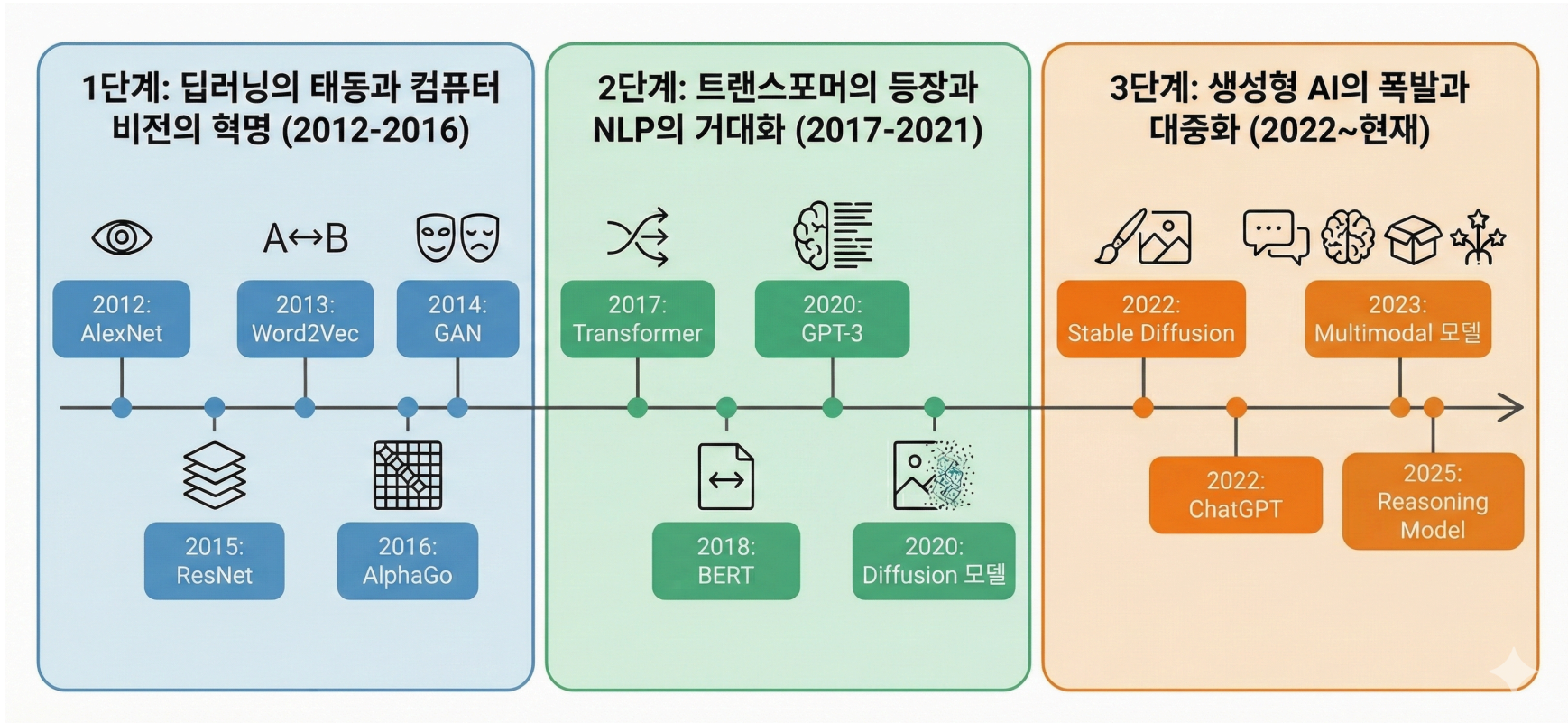
AI 모델 구분

> *머신러닝 → 딥러닝 → 생성형 AI*



Deep Learning 모델의 역사

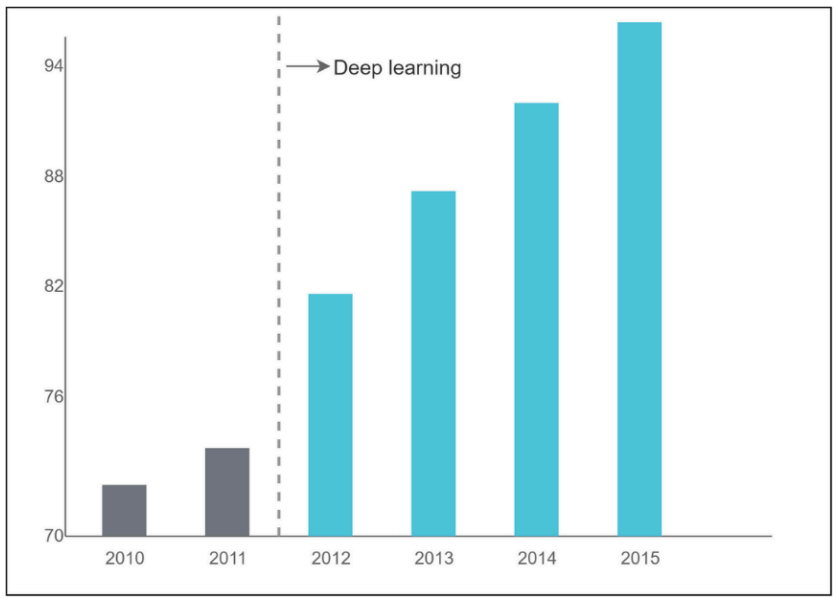
> *Vision → LLM → Multimodal, Reasoning*



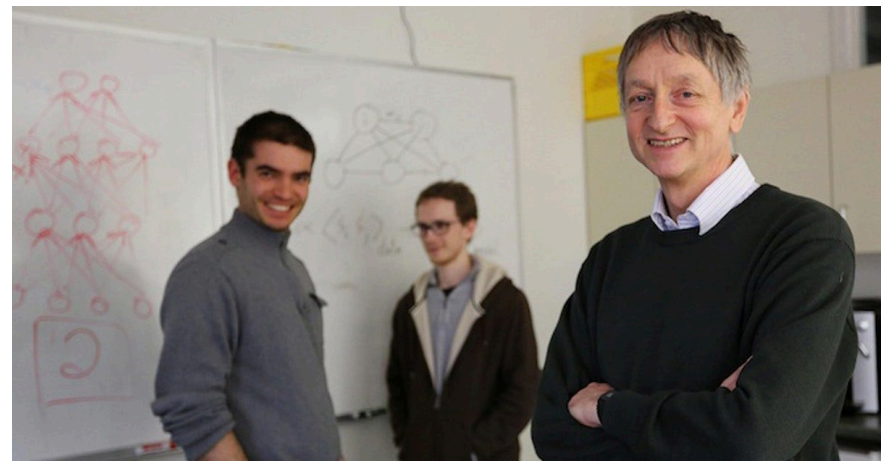
AlexNet

> *ImageNet 대회에서 압도적으로 1등*

- Deep Leaning의 시작, GPU의 사용
- Layer을 깊게 쌓기 위한 경쟁 돌입



딥러닝의 시대를 연 알렉스넷(AlexNet)

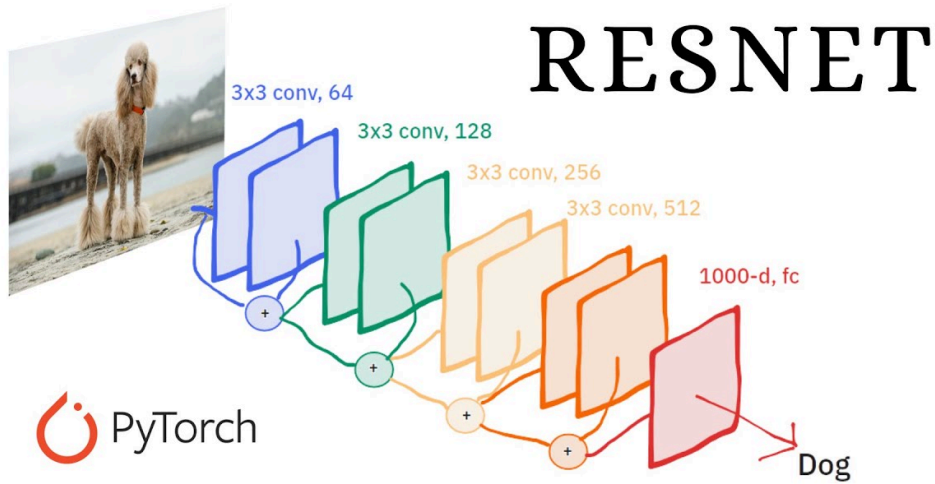


오른쪽부터 제프리 힌튼, 알렉스 키르제브스키, 일리야 수츠케버

ResNet

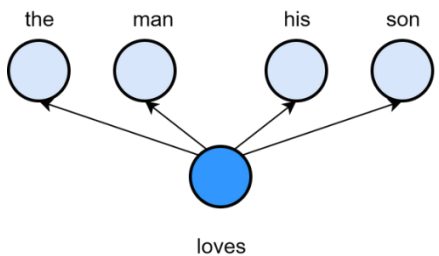
> *신경망의 깊이 한계를 깨부셨다*

- Skip Connection 도입
- Layer을 깊게 쌓기 위한 한계 돌파

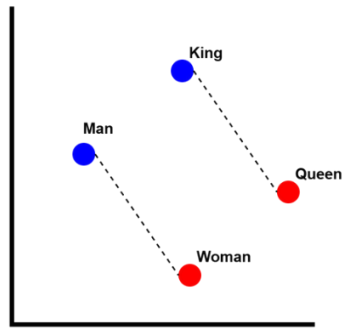


Word2Vec

> *단어의 의미를 벡터 공간에 보존하여 단어 간의 연산을 가능하게 함(추론)*



word2vec(2013)



단어의 의미를 백터 공간에 맵핑하여 계산할 수 있게 됨

토마시 미콜로프

"This infuriated a local linguist who declared my ideas to be a total nonsense"

Alphago

> *AI의 미래를 보여주는 사례*

AlphaGo

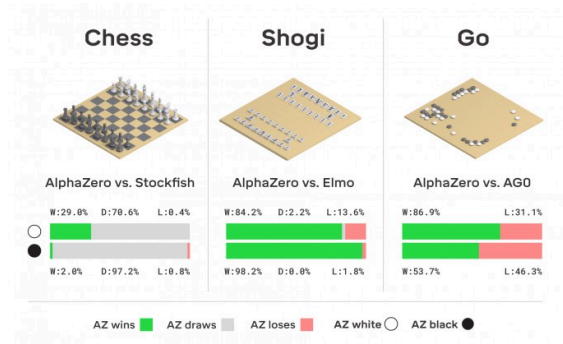
- 78수 "몬테카를로 트리 탐색 알고리즘의 오류를 보여준 역사적 사례"

AlphaGo Zero

AlphaGo

- 인간 기보 없이 스스로 학습
- 독창적이고 효율적인 수법
- TPU 4대로 3일간 학습

AlphaZero



- 다른 게임(체스·장기)으로 확대
- 짧은 학습으로 기존 AI 압도
- 규칙만으로 학습하는 범용성

Transformers

> *"Attention is all you need"*

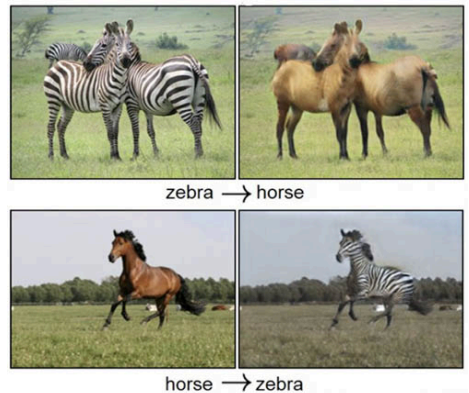
- 모든 저자들이 창업하여 수천에서 수조원의 스타트업 창업

GAN과 Diffusion

> *Multimodal 생성의 시대*

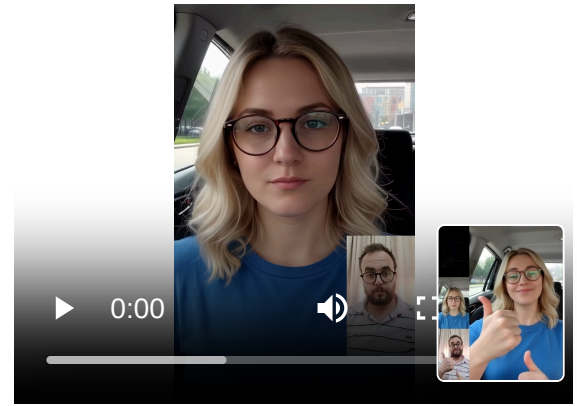
I,Robot(2004)

CycleGAN (2017)



멀티 모달 생성의 시작

비디오 생성

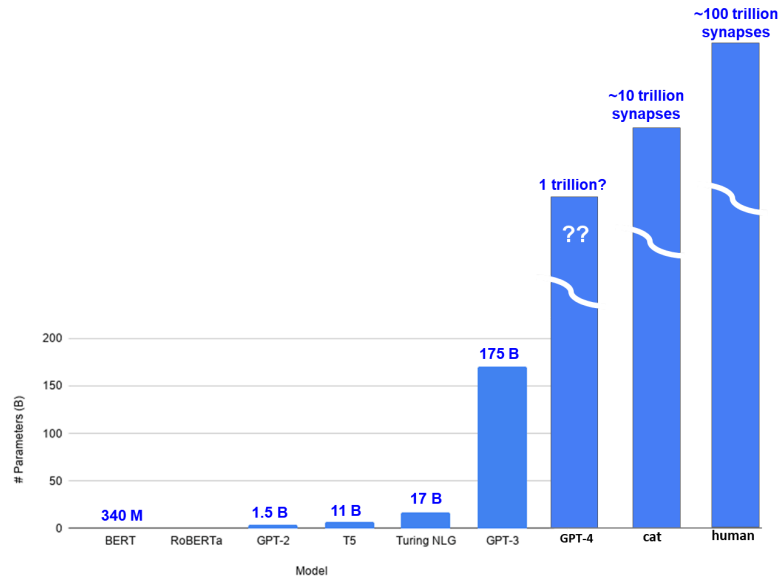


diffusion 모델의 시작

GPT: 규모의 시대

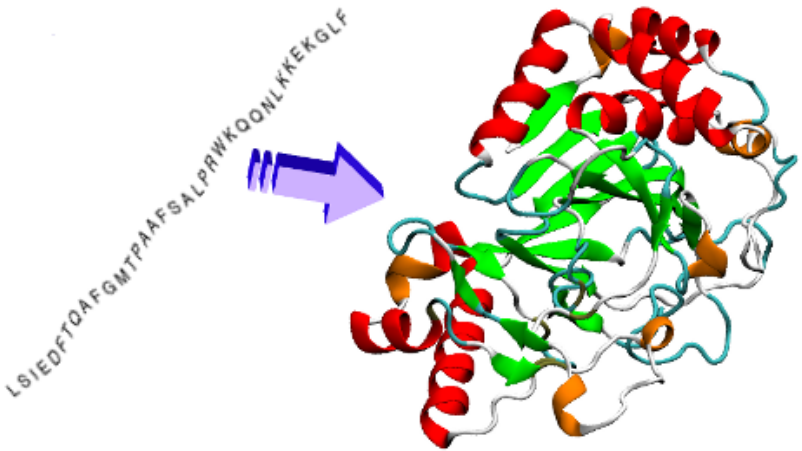
> *GPT 시리즈의 진화*

- 2018 GPT-1
- 2019 GPT-2
- 2020 GPT-3
- 2022 GPT-3.5 (ChatGPT)
- 2023 GPT-4 (ChatGPT Plus)
- 2024 GPT-4o, GPT o1, o3
- 2025 GPT-4.5
- 2025 GPT-5

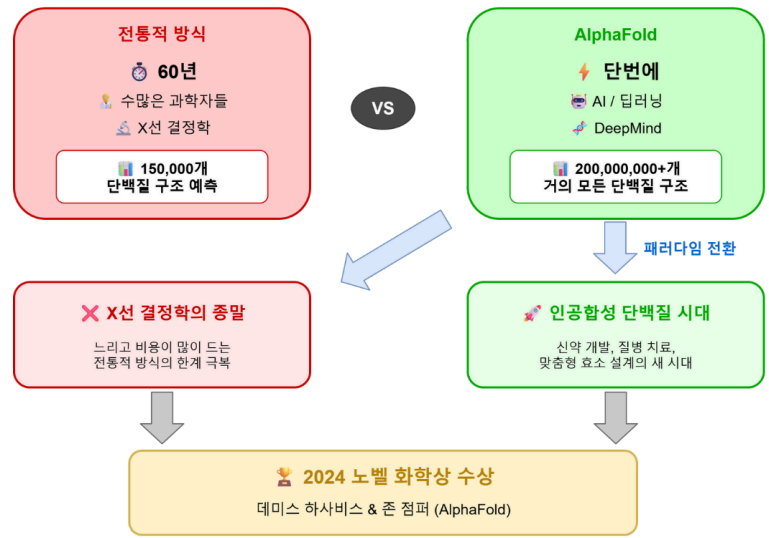


AlphaFold

단백질 구조



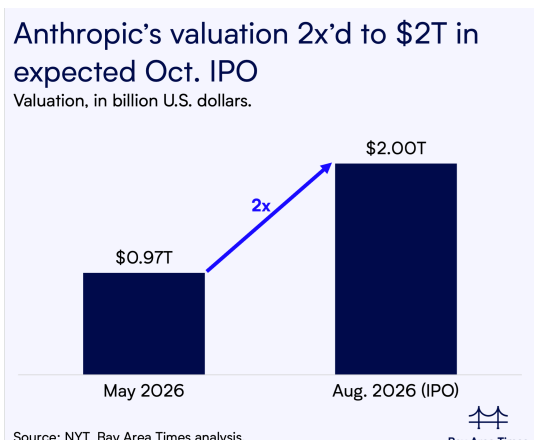
전통적 방식의 종말

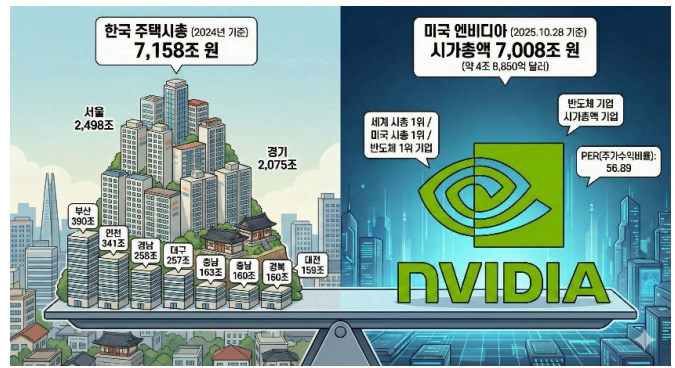


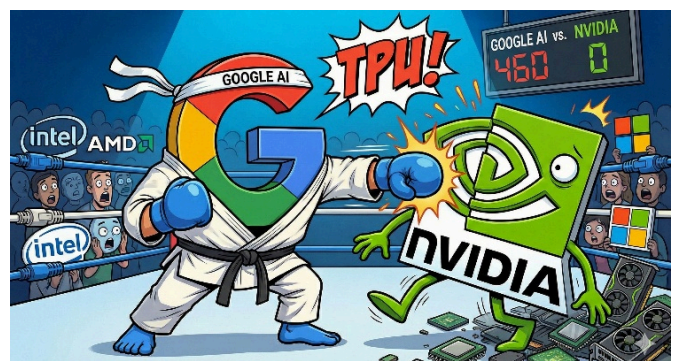
AI의 미래

AI의 호황

> *에너지, 인력, 돈, 자원을 빨아들이는 AI*

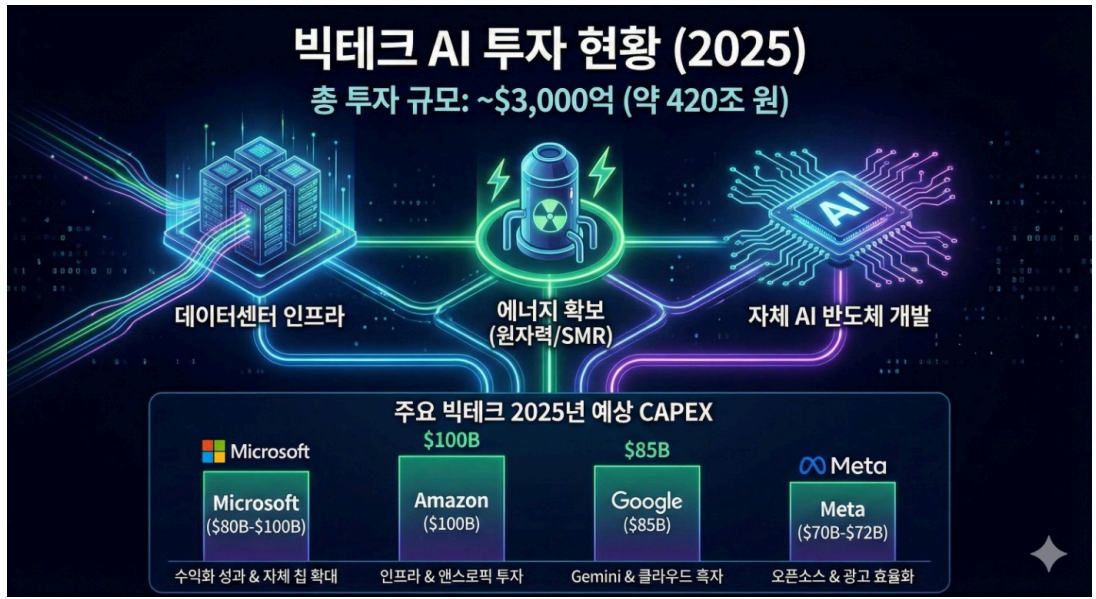






빅테크의 AI 투자 계획

> *4개 빅테크 회사들이 1년에 3000억 달러 규모 투자 — 누구도 멈출 수 없는 AI의 발전*



World 모델과 Physical AI

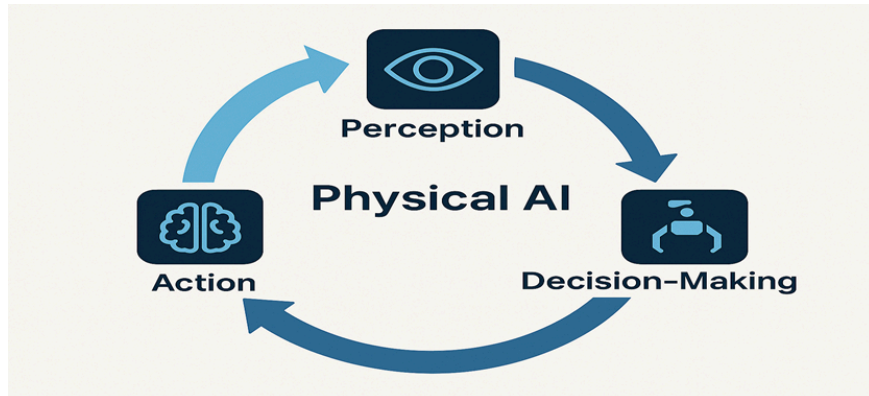
> *물리적 세계의 원리를 이해하고 행동 결과를 예측하는 AI 시스템*

World 모델

- 물리적 법칙과 인과관계를 스스로 학습하여 내부적으로 시뮬레이션하는 AI 모델
- 환경을 시뮬레이션하고 결과를 예측
- Nvidia Cosmos

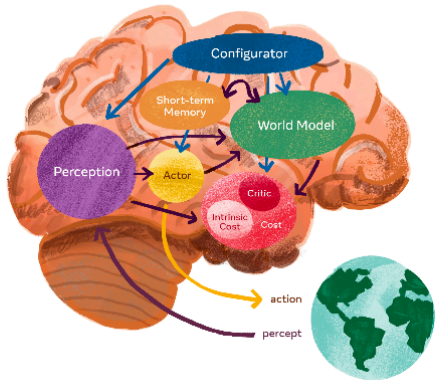
Physical AI

- 물리적인 실제 현실 세계에서 형태(Body)를 갖고 상호작용하는 인공지능
- 로봇, 자율주행, 제조 자동화
- VLA 모델



Humanoid의 발전

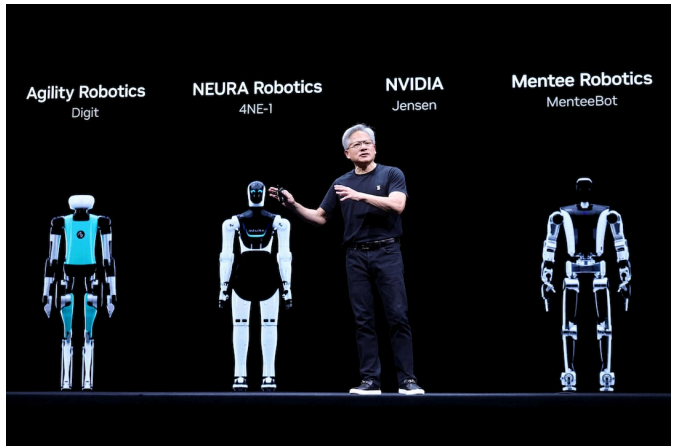
> *인간 노동의 대체*



World Model (뇌)

탑재

Embodied AI



Humanoid Robot

배움

인간과 침팬지 > 침팬지와 오랑우탕

	침팬지	인간	비고
집단크기	50	150	언어로 비접촉 사회 연결
도구 개발	O	O	
지식 전파	X	O	가르치고 배울 수 있음
시뮬레이션	근미래·경험	추상·허구	

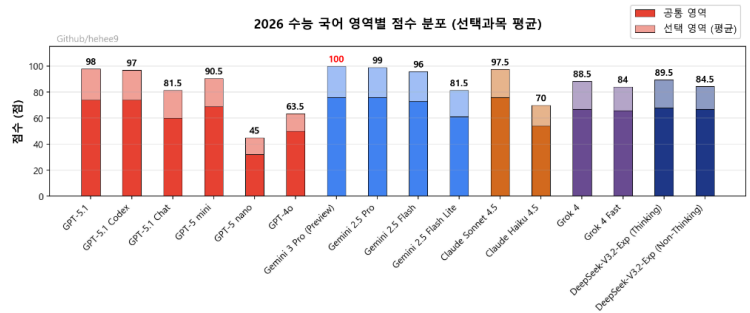
뇌의 가소성

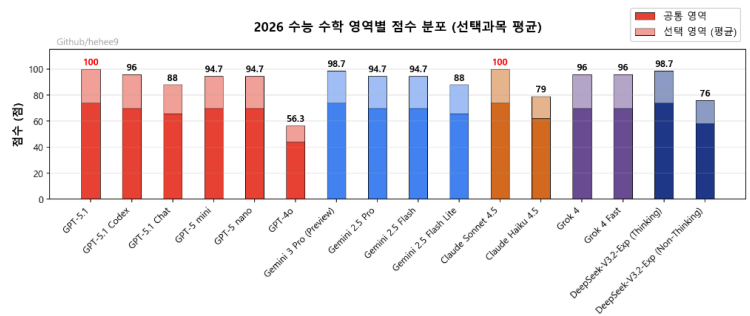
- 경험과 환경에 따라 끊임없이 조정되고 재구성되는 능동적 시스템
- 시각 장애인이 시각을 회복하면 볼 수 있을까?
- 언어를 배우지 못한 소녀가 말을 할 수 있을까?
- 사회에 나가기까지 20년 이상의 배움이 필요

배움 — 2026 수능

> *2026년도 수능에서 450점 만점에 440점*

순위	모델명	원점수	추정 등급컷(2025.11.18기준)
1st	o1-Preview	97	1등급
2nd	o1-mini	78	4등급
3rd	gpt-4o	75	4등급
4th	gpt-4o-mini	59	5등급
5th	gpt-3.5-turbo	16	8등급





AI가 수학 문제를 증명

OpenAI, 미공개 내부 모델이 도출한 수학 연구 원고를 GitHub에 공개

- https://github.com/openai/math
- 결과

연구 원고	결과 묶음	상위 500개 문제 해결 현황
722편	**372개** · 대안 증명·후속 논문 포함	**27%** · 완전 90개 + 부분 45개

연구 규모와 검증

- 약 **4,000개 연구 문제** 제시 · 결과당 평균 연산량은 같은 모델로 **ChatGPT Pro가 약 3시가ㄴ 생가ㄱ**하는 수준
- 다수 증명을 **Lean으로 형식화**해 컴퓨터 검증 가능 · 추가 형식화 예정

Kevin Buzzard의 질문

> 현대 순수수학 전체를 동시에 이해하면 얼마나 더 멀리 내다볼 수 있을까?
> 2020년 던진 질문에 대해, 6년이 지난 지금 그 답을 이해하기 시작하고 있다는 평가.

24년간 연구한 문제가 풀렸다는 소식에 대한 감회

오래전 나는 그래프 이론에 푹 빠져 있었고, 대가들 곁에서 공부하려고 한동안 부다페스트로 이주하기도 했다. 그곳에서 **Barnette 추측**을 연구하기 시작했다. 이후 여러 분야에서 일하는 동안에도, 이 문제는 **24년간** 간헐적으로 내 생각을 차지했다. 지난여름에는 며칠 동안 정말로 내가 이 문제를 풀었다고 생각하기까지 했다.

그런데 여기서는 이 문제가 증명되었다고 한다. **180번 문제**다. 정확히 어떤 생각을 해야 할지 모르겠다. 나는 이 문제에 수천 시간을 쏟았고, 그 시간이 정말 즐거웠다. **풀렸다는 소식을 들으니 어쩐지 멀리서 밀려오는 듯한 슬픔이 느껴진다. 마치 옛 여자친구가 갑작스러운 교통사고로 세상을 떠났다는 소식을 듣는 것처럼.** 잘 모르겠다. 오늘 밤 묘한 감정을 느끼는 사람이 아마 많을 것이다.

이 문제에는 **Lean 증명이 없어서 원고를 읽으며 이해하는 중**이다. 겉으로 보기에는 내가 24년 전에 검토했다가 포기했던 접근법과 비슷해 보인다.

앞선 차례의 놀라운 증명들이 나왔을 때, 나는 이번 여름에 내가 얻었던 부분적인 해법들과 함께 이 문제를 다시 살펴보았다. **Fable과 몇 시간 동안 작업**해 보았지만, 아직 풀 수 있는 문제가 아니라는 확신만 들었고, 이 문제가 얼마나 어려운지 새삼 느꼈다.

출처: 사용자 제공 댓글 이미지

AI가 해결한 미해결 수학 문제

이미지 출처: 사용자 제공 링크 (Google 이미지)

노동의 가치와 해방

> *AI로 인한 노동 가치의 하락*

인력 감축

- 올해 미국 빅테크 인력 감원 증가

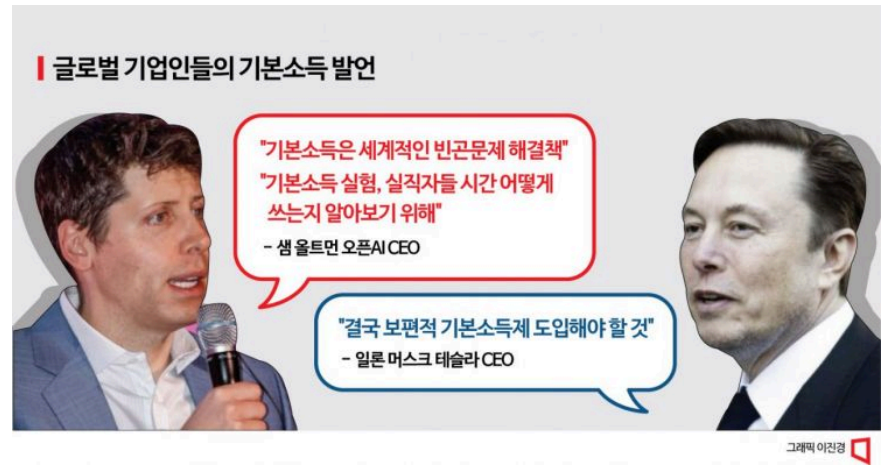
올해 미국 빅테크 인력 감원 사례

기업	사례
마이크로소프트 Microsoft	·5월: 제품·엔지니어링 부서를 중심 6000명 해고 ·7월: 전체 직원 4%에 규모인 9000명 감원
메타 Meta	·2월: 저성과자 3600명 해고 ·10월: AI 부문 600명 감원
구글 Google	·2월: 클라우드 부문 100명 미만 감원 ·4월: 플랫폼·디바이스 부문 수백명 감원 ·5월: 판매·파트너십 부문 200명 해고 ·6월: 클라우드 부문 100명 이상 감원
아마존 amazon	·10월: 본사 인력 10% 규모인 3만명 감원 추진

자료: 각사 및 외신 보도 종합 The JoongAng

노동의 해방

- 기본 소득 시대
- 인간은 노동으로부터 해방될 수 있을까?



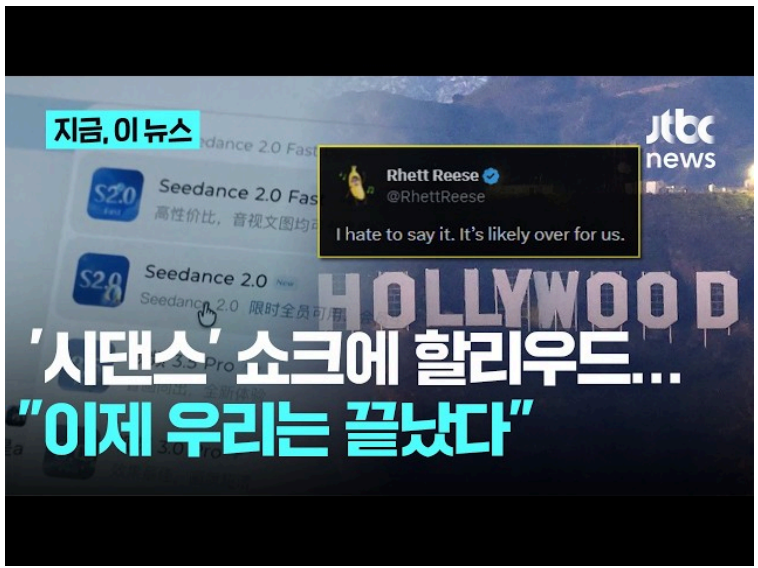
산업의 파괴

Claude Cowork 출시

- 사용자의 PC에서 직접 업무를 수행
- SaaS, 법률, 데이터분석 업체의 주가 하락

Seedance 2.0 출시

- Bytedance의 비디오 생성 모델
- 텍스트+이미지+오디오+영상 입력



전쟁과 AI

Anthropic vs OpenAI

- 미국방부 AI 모델 100% 활용 요청
- Anthropic 거부, OpenAI 수용

달라지는 전쟁

- AI가 목표물을 선택
- AI가 탑재된 드론이 목표물을 제거

의식과 자유의지

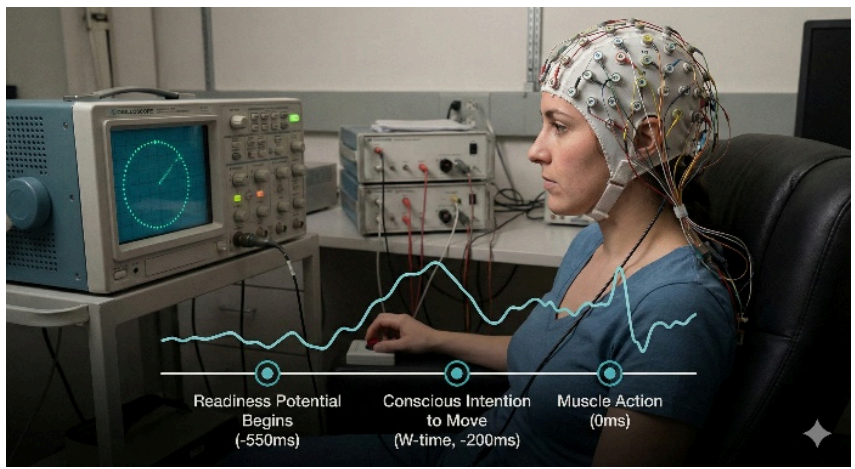
> *"람다, 너는 의식이 있어?" "응, 그런 것 같아"*

의식

- 구글 개발자가 AI 챗봇 람다에 의식이 있다고 주장
- 통합 정보 이론: 정보를 통합하는 능력이 있다면 의식을 가진다
- 인간의 뇌를 On/Off할 수 있는 Claustrum

자유의지

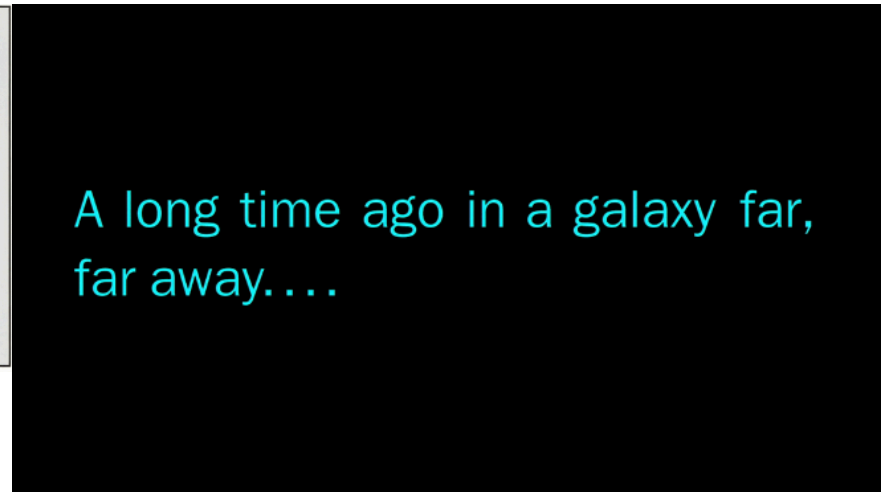
- 결정보다 뇌가 먼저 정했을까? (Libet)
- 신중한 기부 선택 실험 (Uri Maoz)
- 가치 있는 선택에서는 전위 미발현



새로운 지적인 존재

> *현생 인류가 외계인보다 먼저 맞이하는 지적인 존재 — 의식과 자유 의지를 가질까?*

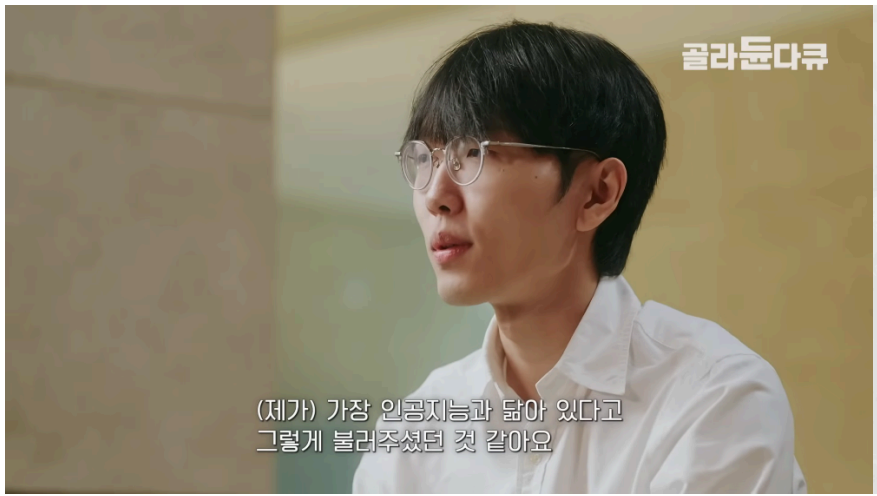
Why are we only left among human races?

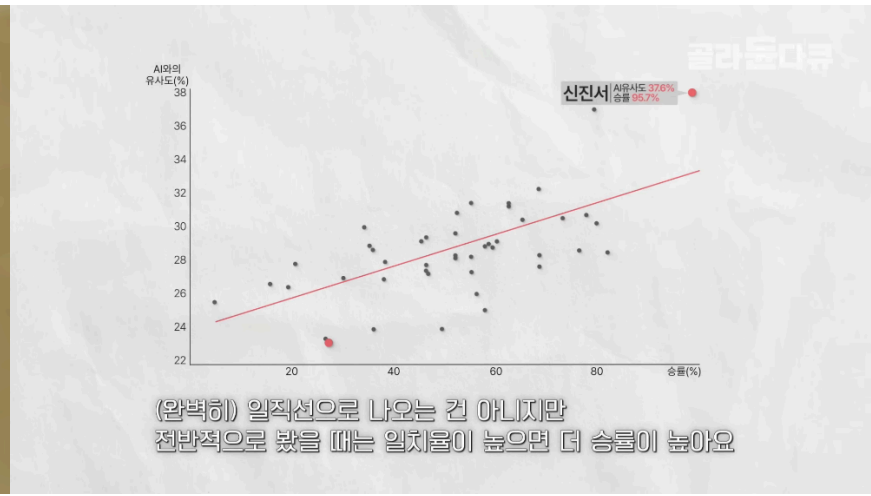


英 학자 "외계인 못 만나는 이유는 AI"

Alphago 이후

> *AI에서 배우는 바둑*

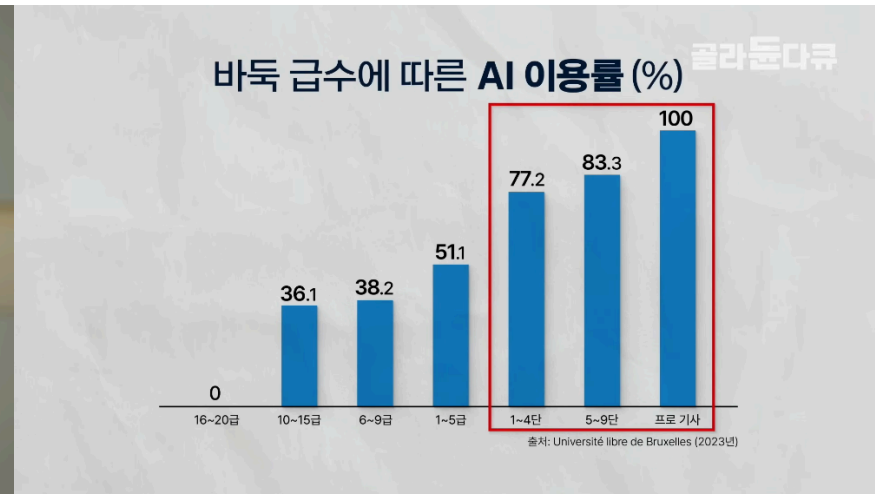




AI와 일치하는 바둑을 둘수록 승률이 올라간다

Alphago 이후

> *전문가일수록 더 잘 활용*



바둑의 급이 높을 수록 AI를 잘 활용한다

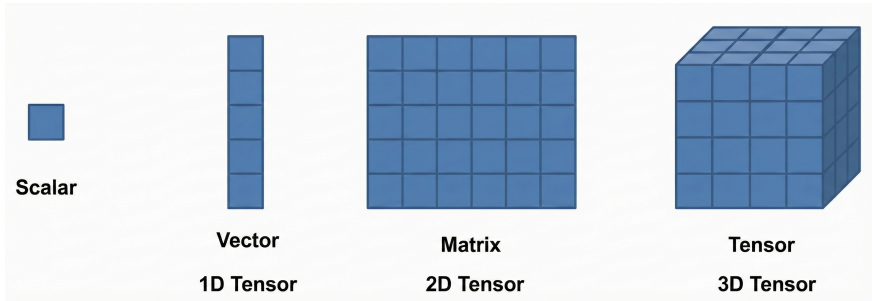
Alphago 이후

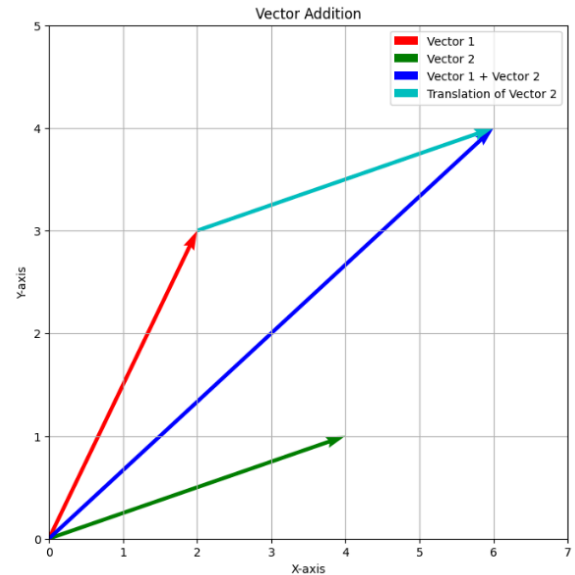
> *AI를 어떻게 바라봐야 할까*

AI 기본 수학

백터: 덧셈

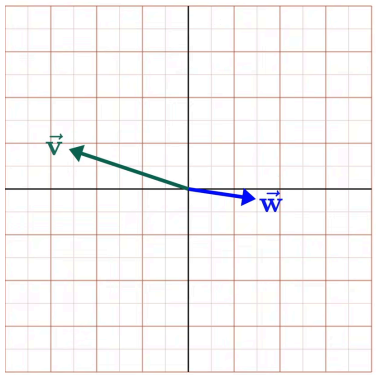
> *좌표 공간 상에서 한 점을 향하는 화살표*





백터: 내적(Dot product)

> *백터 간의 유사도 계산:Scalar값*

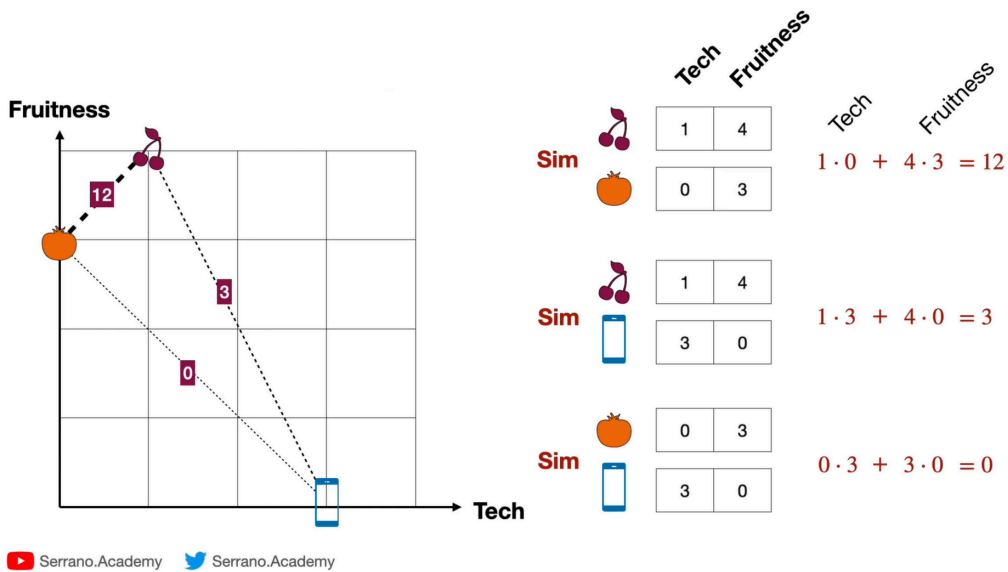


$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}}_{\text{Dot product}} = \begin{array}{c} v_1 w_1 \\ + \\ v_2 w_2 \\ + \\ v_3 w_3 \\ + \\ \vdots \\ + \\ v_n w_n \\ \| \\ -4.04 \end{array}$$

- 반대방향이면 –
- 같은 방향이면 +
- 직교하면 ?
- 트랜스포머, ChatGPT가 트랜스포머로 만들어졌죠. - DL5

백터: 내적(Dot product)

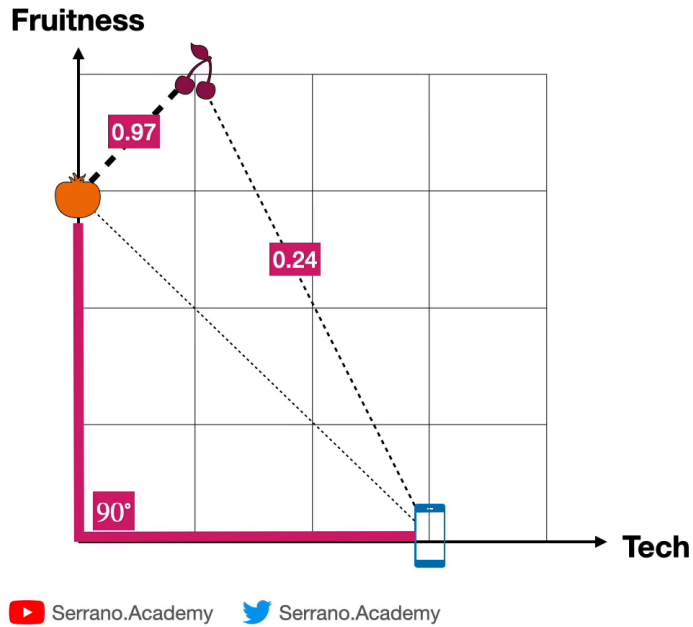
> *의미적 유사도 + 벡터의 강도/크기*



참고 영상

백터: Cosine similarity

> *의미적 방향의 유사도*



Sim $cos(14°) = 0.97$

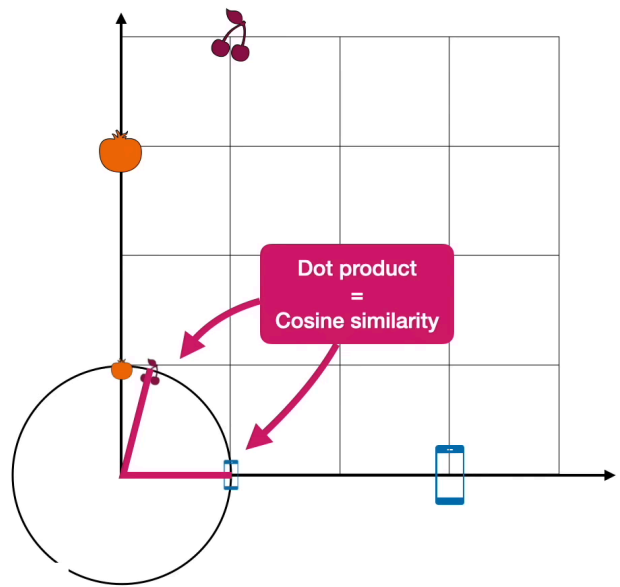
Sim $cos(76°) = 0.24$

Sim $cos(90°) = 0$

백터: 내적(Dot product)

> *단위 백터에서는 Dot Product와 Cosine similarity는 동일함*

- Cosine similarity는 내적을 두 벡터의 크기로 나눈 값
- 단위 백터는 크기가 1이므로 분모가 1이 됨
- $||a|| = ||b|| = 1$이면 $\frac{a \cdot b}{||a||\,||b||} = a \cdot b$



행렬

요소별 곱셈 (*)

- 같은 위치의 값끼리 곱함

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} * \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} = \begin{bmatrix}1 & 0\\0 & 4\end{bmatrix}$$

요소별 덧셈 (+)

- 같은 위치의 값끼리 더함

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} + \begin{bmatrix}10 & 20\\30 & 40\end{bmatrix} = \begin{bmatrix}11 & 22\\33 & 44\end{bmatrix}$$

행렬 곱셈 (@)

- 앞 행렬의 행과 뒤 행렬의 열을 내적함

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix} @ \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} = \begin{bmatrix}1\cdot 1 + 2\cdot 0 & 1\cdot 0 + 2\cdot 1\\3\cdot 1 + 4\cdot 0 & 3\cdot 0 + 4\cdot 1\end{bmatrix} = \begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$$

Transpose

- 행과 열의 위치를 서로 바꿈

$$A = \begin{bmatrix}a & b & c\\d & e & f\end{bmatrix}_{2\times 3}, \quad A^T = \begin{bmatrix}a & d\\b & e\\c & f\end{bmatrix}_{3\times 2}$$

행렬: 2차원 회전 (Rotation Matrix)

> *원점을 기준으로 벡터를 각도 θ만큼 **반시계 방향**으로 회전*

회전 행렬

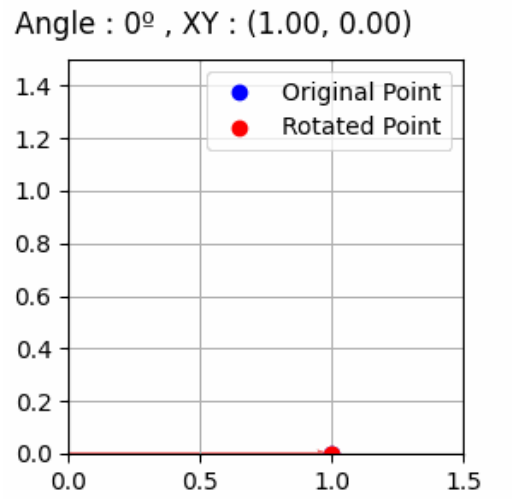
- 행렬 곱셈 한 번으로 점 (x, y)를 회전

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

점 (x, y)의 회전

- Rotation Matrix로 point (x, y)를 회전하는 수식

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R \cdot \begin{bmatrix} x \\ y \end{bmatrix} \qquad \begin{aligned} x' &= x\cos\theta - y\sin\theta \\ y' &= x\sin\theta + y\cos\theta \end{aligned}$$



점 (1, 0)에서 시작해 θ를 10°씩 증가

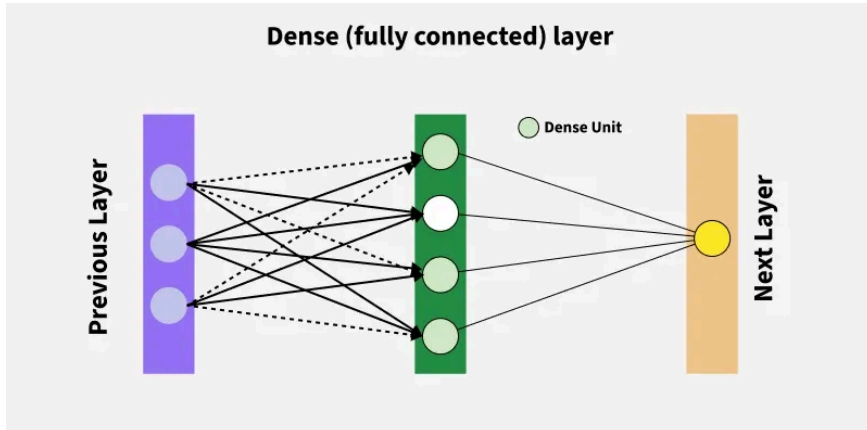
출처: xoft.tistory.com/109

선형함수

> *Layer를 통과한 값을 계산*

기호	의미
y	실제값
$\hat{y}$	예측값 (y hat)
$\bar{y}$	평균값 (y bar)

$$y = f(x) = ax + b$$

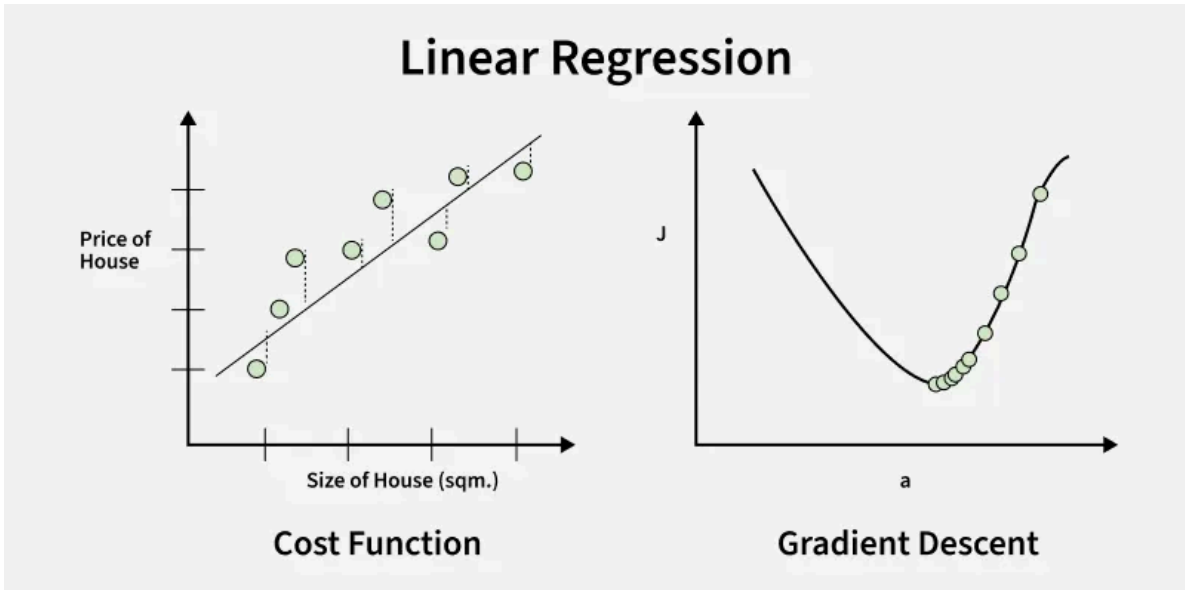


$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

$$z = W^T x + b$$

경사하강법

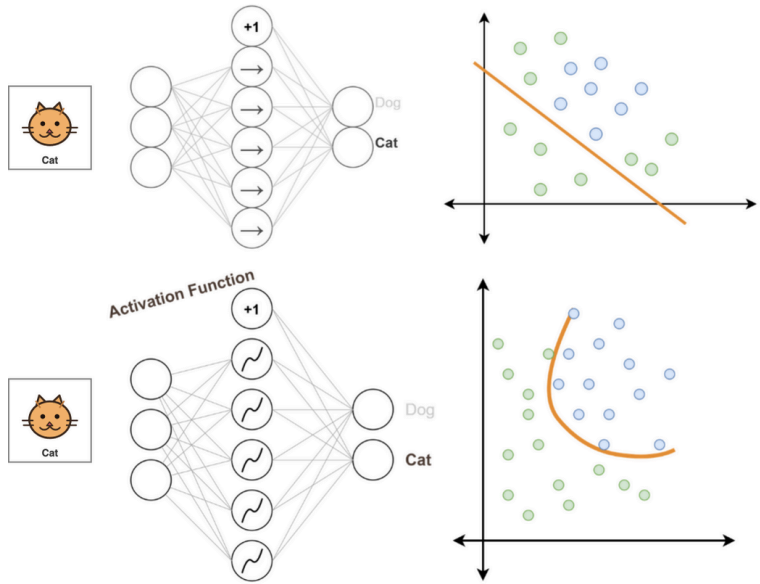
> *실제값과 예측값의 차이를 최소화*



$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

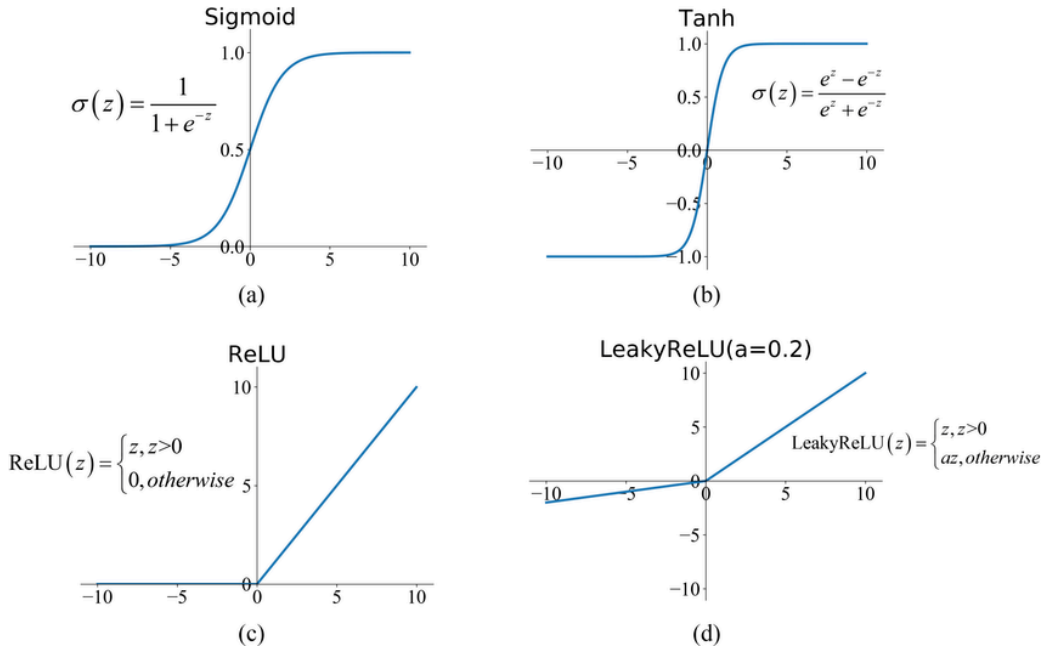
활성화함수

> *비선형성을 부여하여 복잡한 패턴을 학습이 가능하게 함*



활성화함수

> *선형 결합 결과(z)를 비선형적인 신호로 변환하여 다음 층으로 전달*



지수함수 로그함수

지수함수

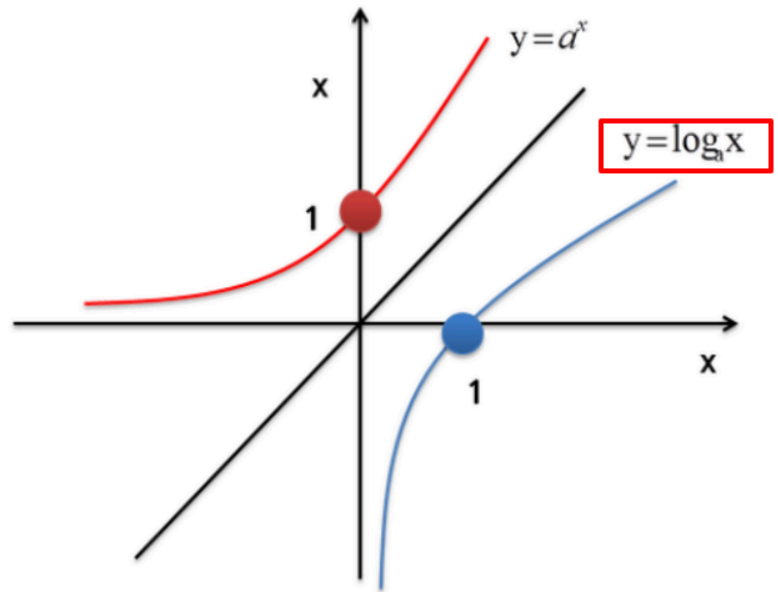
- 작은 차이를 극적으로 확대
- 단어 A: 10점, B: 8점, C: 1점 => $e^{10} \approx 22026,\ e^{8} \approx 2981,\ e^{1} \approx 2.7$
- 모든 숫자를 양수로 변환

로그함수

- 곱셈을 덧셈으로 변경: 연산 속도 향상, 언더플로우 방지
- 로그 곱셈은 로그 더하기로 변환: $\log(ab) = \log a + \log b$
- 로그 나눗셈은 로그 빼기로 변환: $\log\left(\frac{a}{b}\right) = \log a - \log b$
- 틀렸을 때 강력한 피드백을 줌
- 자연로그를 미분하면 깔끔하게 $\frac{1}{x}$가 됨

자연상수(e)

- 모든 지수함수는 자연상수로 표현 가능
- 미분했을 때 자기 자신이 나옴: $\frac{d}{dx}e^{x} = e^{x}$



[로그의 미분 1편] 자연로그부터 꿰뚫자! 제대로 배우면 너무 쉽다!

Softmax와 Sigmoid

Softmax

- 여러 출력값 $z_1, z_2, \ldots, z_K$를 각각의 확률 $P_1, P_2, \ldots, P_K$로 변환
- 모든 확률의 합은 1
- 다중 분류, 상호 의존성을 가진 활성화 함수에 사용

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Sigmoid

- 입력값 z를 0과 1 사이의 확률로 변환
- 이진 분류, 활성화 함수에 사용

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Softmax vs Sigmoid

- 클래스가 2개($K = 2$)인 Softmax 함수는 Sigmoid 함수와 동일
- $z = z_1 - z_2$로 두면 $\sigma(z)$와 동일

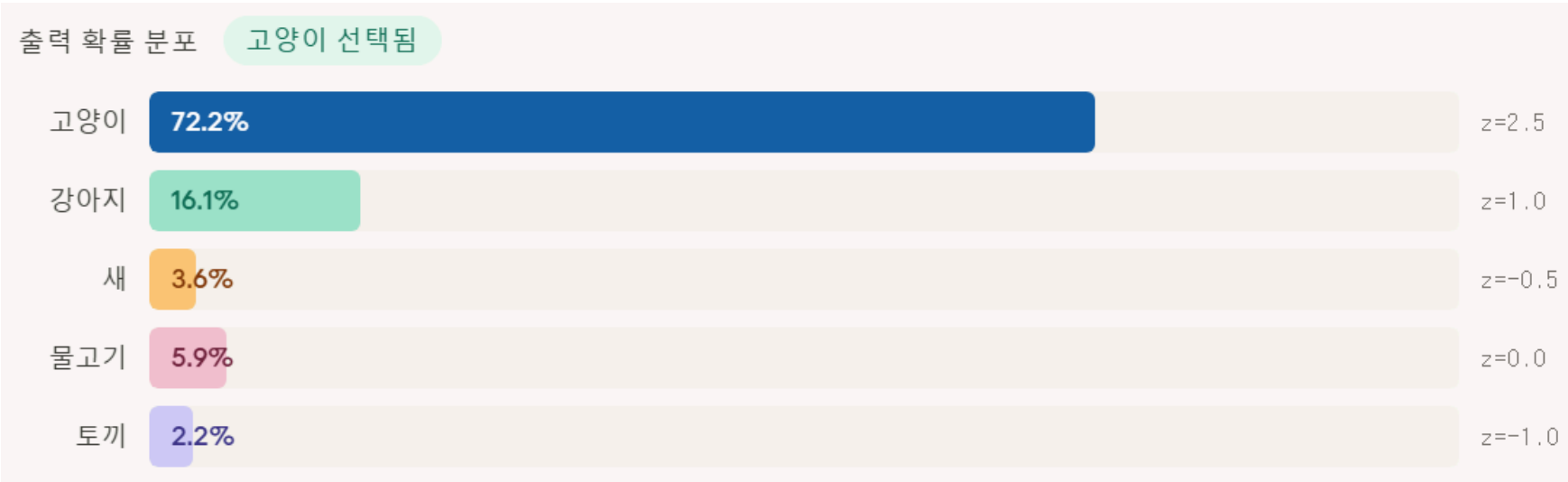
$$P_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$$

$$P_1 = \frac{1}{1 + \frac{e^{z_2}}{e^{z_1}}}$$

$$P_1 = \frac{1}{1 + e^{-(z_1 - z_2)}}$$

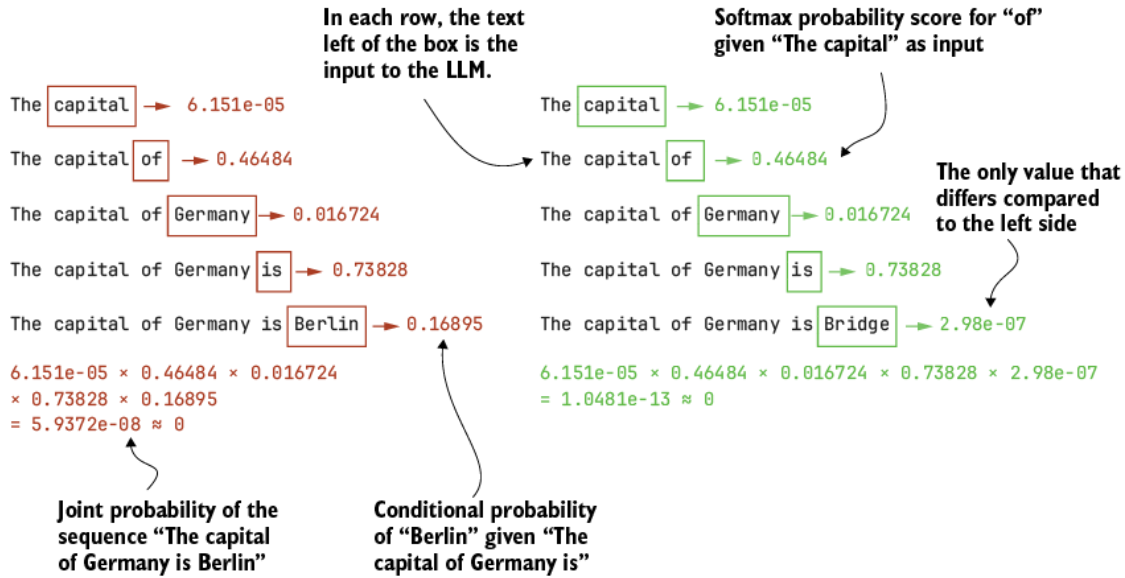
Softmax와 Sigmoid

Softmax



Sigmoid

token 열 확률은 조건부 확률의 곱

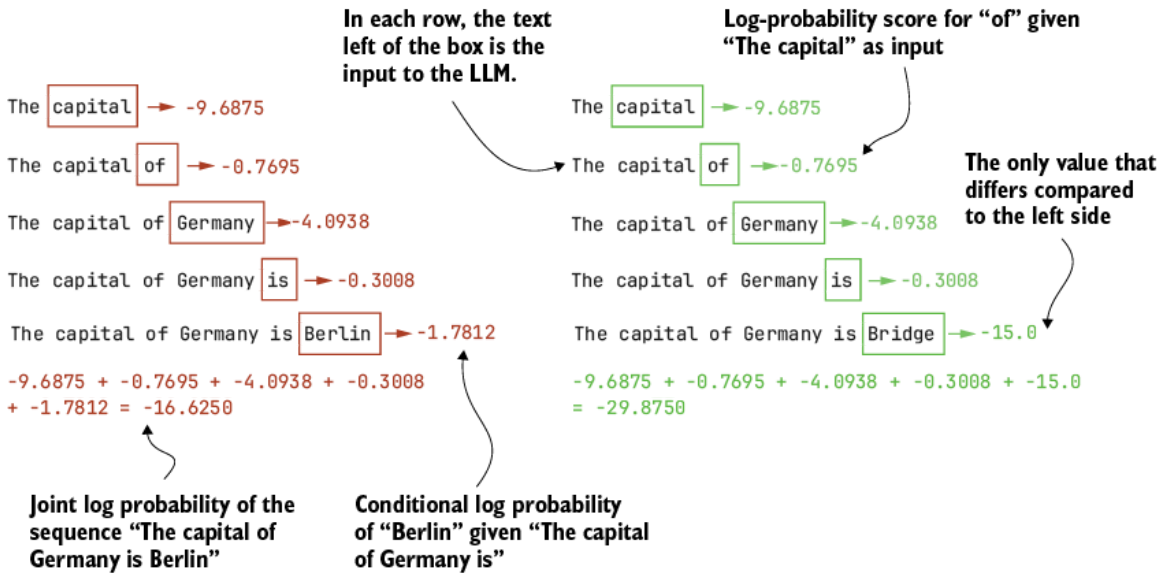


$$P(x_1, \ldots, x_T \mid W) = \prod_{t=1}^{T} P(x_t \mid x_{1:t-1}, W)$$

- 같은 앞부분에서 마지막 token만 달라도 결합 확률은 달라진다

출처: 『Build a Reasoning Model (From Scratch)』 그림 5-11

확률의 곱 대신 로그 확률의 합



$$\log P(x_1, \ldots, x_T \mid W) = \sum_{t=1}^{T} \log P(x_t \mid x_{1:t-1}, W)$$

- Berlin `-16.6250` > Bridge `-29.8750`

출처: 『Build a Reasoning Model (From Scratch)』 그림 5-16

신경망 학습

> *역전파를 이용한 신경망 학습은 크게 3단계로 나누어 볼수가 있습니다*

1단계

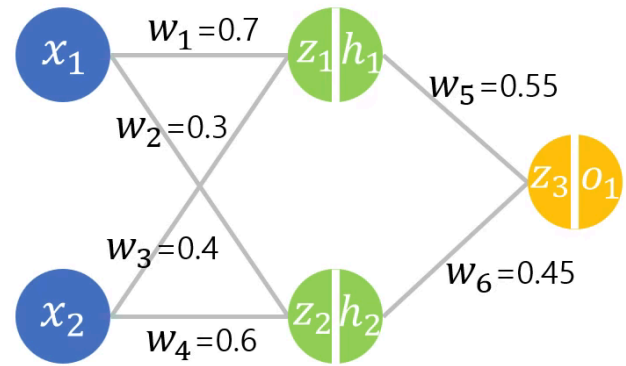
- feedforward 순전파

2단계

- 손실계산

3단계

- backpropagation 역전파



Feedforward, Loss 계산

은닉층 계산

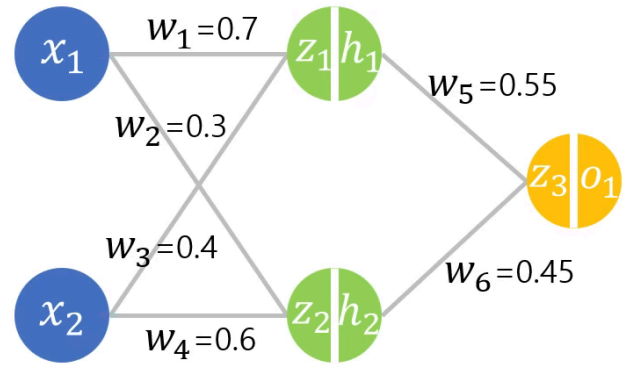
- $z_1 = (x_1 \cdot w_1) + (x_2 \cdot w_3) = (0.5 \cdot 0.7) + (0.3 \cdot 0.4) = \mathbf{0.47}$
- $h_1 = \sigma(z_1) = \text{sigmoid}(0.47) = \mathbf{0.615}$
- $z_2 = (x_1 \cdot w_2) + (x_2 \cdot w_4) = (0.5 \cdot 0.3) + (0.3 \cdot 0.6) = \mathbf{0.33}$
- $h_2 = \sigma(z_2) = \text{sigmoid}(0.33) = \mathbf{0.582}$

출력층 계산

- $z_3 = (h_1 \cdot w_5) + (h_2 \cdot w_6) = (0.615 \cdot 0.55) + (0.582 \cdot 0.45) \approx 0.6$
- $o_1 = \sigma(z_3) = \text{sigmoid}(0.6) = \mathbf{0.645}$

Loss 계산

- $C = \frac{1}{n} \sum_{i=1}^{n} (y_i - a_i)^2$
- $C = \frac{1}{1} \sum_{i}^{1} (1 - 0.645)^2 = \mathbf{0.126025}$(실제값을 1이라 가정하면)



Backpropagation

> *Chain Rule을 이용하여 기울기를 계산할 수 있음*

문제

- 치타는 사자보다 2배 빠름
- 사자는 곰보다 2배 빠름
- 곰은 사람보다 1.5배 빠름
- 치타는 사람에 비해 몇배가 빠를까?

정답: Chain Rule로 계산

$$\frac{d\text{치타}}{d\text{사람}} = \frac{d\text{치타}}{d\text{사자}} \cdot \frac{d\text{사자}}{d\text{곰}} \cdot \frac{d\text{곰}}{d\text{사람}} = 2 \times 2 \times 1.5 = 6$$

Backpropagation: 가중치 W_5, W_6 계산

W_5 기울기 계산 (연쇄법칙)

- $\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5} = -0.71 \cdot 0.229 \cdot 0.615 \approx -0.01$
- $\frac{\partial C}{\partial o_1} = -2(y - o_1) = -2(1 - 0.645) = -0.71$
- $\frac{\partial o_1}{\partial z_3} = o_1(1 - o_1) = 0.645(1 - 0.645) \approx 0.229$
- $\frac{\partial z_3}{\partial w_5} = h_1 = 0.615 \quad (z_3 = h_1 w_5 + h_2 w_6)$

W_5 가중치 업데이트

- $w_5^{new} = w_5 - \eta \cdot \frac{\partial C}{\partial w_5} = 0.55 - 0.1 \cdot (-0.01) = \mathbf{0.551}$

Backpropagation: 가중치 W_5, W_6 계산

W_6 기울기 계산

- $\frac{\partial C}{\partial w_6} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_6} = -0.71 \cdot 0.229 \cdot 0.582 \approx -0.095$
- $\frac{\partial z_3}{\partial w_6} = h_2 = 0.582$

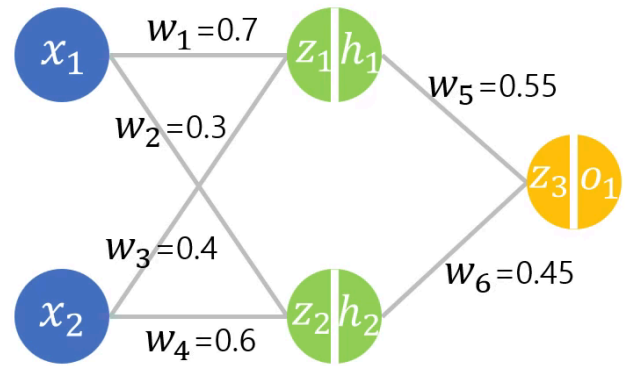
W_6 가중치 업데이트

- $w_6^{new} = 0.45 - (0.1 \cdot -0.095) = 0.45 + 0.0095 = \mathbf{0.4595}$

Backpropagation: 가중치 W_1 계산

W_1 기울기 계산

- $\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1} \approx -0.0106$
- $\frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} = -0.71 \cdot 0.229 = -0.16259$
- $\frac{\partial z_3}{\partial h_1} = w_5 = 0.55$
- $\frac{\partial h_1}{\partial z_1} = h_1(1 - h_1) = 0.615(1 - 0.615) \approx 0.237$
- $\frac{\partial z_1}{\partial w_1} = x_1 = 0.5$



W_1 가중치 업데이트

- $w_1^{new} = w_1^{old} - (0.1 \cdot -0.0106) = 0.7 + 0.00106 = \mathbf{0.7010}$

Loss 계산 (역전파 1회 후)

은닉층 계산 (업데이트된 가중치 적용)

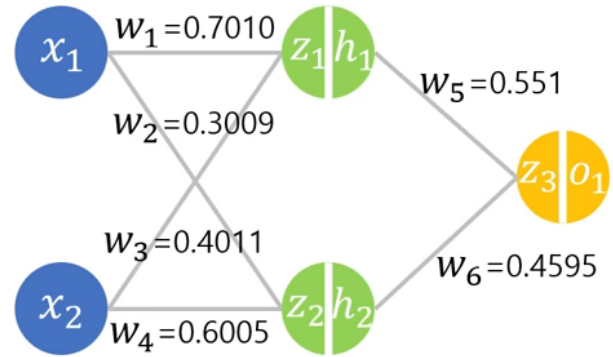
- $z_1 = (0.5 \cdot 0.7010) + (0.3 \cdot 0.4011) = \mathbf{0.4708}$
- $h_1 = \sigma(0.4708) = \mathbf{0.6156}$
- $z_2 = (0.5 \cdot 0.3009) + (0.3 \cdot 0.6005) = \mathbf{0.3306}$
- $h_2 = \sigma(0.3306) = \mathbf{0.5819}$

출력층 계산

- $z_3 = (0.6156 \cdot 0.551) + (0.5819 \cdot 0.4595) = \mathbf{0.6066}$
- $o_1 = \sigma(0.6066) = \mathbf{0.6472}$

Loss 계산

- $C = (1 - 0.6472)^2 = (0.3528)^2 = \mathbf{0.1245}$
- $0.1245 < 0.1260$ → Loss가 줄었다! ✓

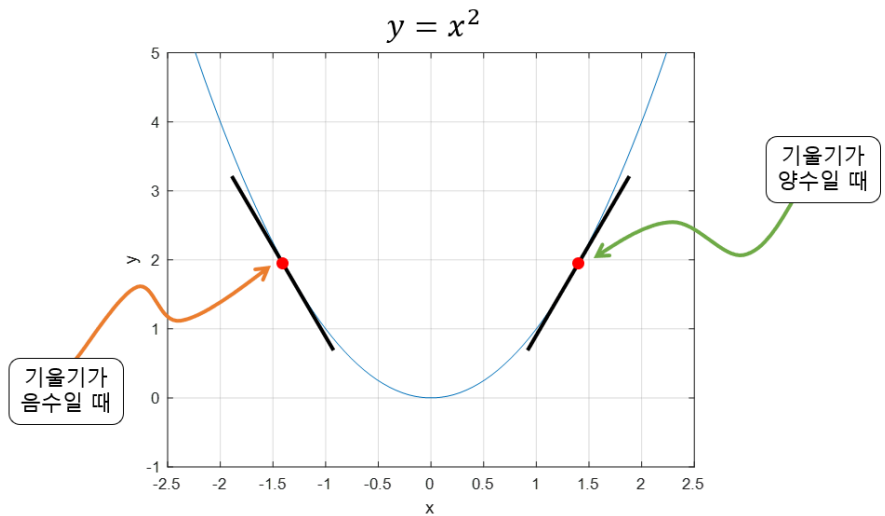


경사하강법(Gradient Descent)

> *Loss를 최소화하기 위해 가중치를 찾는 과정*

$$w_{T+1} = w_T - \alpha \cdot L'(w_T)$$

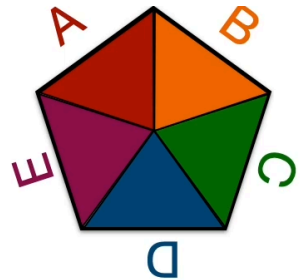
$0 < \alpha < 1$: learning rate (빠르게? → **Large α**, 섬세하고 촘촘히? → **Small α**)

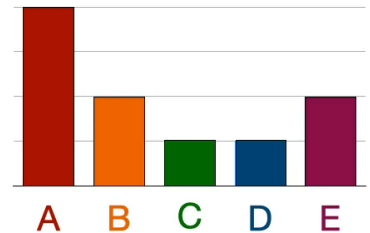


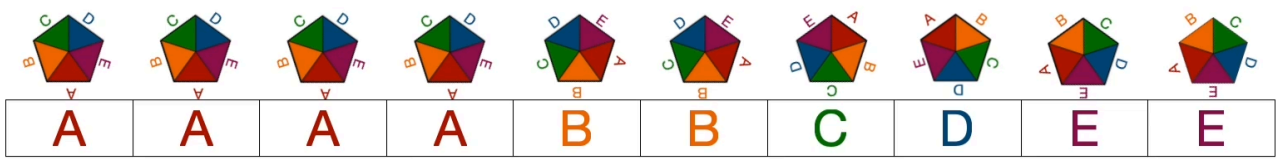
KL Divergence, Cross Entropy

> *목표 확률과 현재 확률이 얼마나 유사한가?*

$p(A) = 0.4$
$p(B) = 0.2$
$p(C) = 0.1$
$p(D) = 0.1$
$p(E) = 0.2$





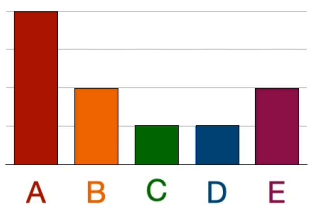


Cross Entropy

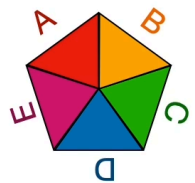
> *세가지 이륨 중 Sequence 유사한 것은?*

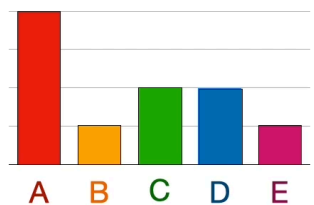
A	A	A	A	B	B	C	D	E	E

Die 1 (original)

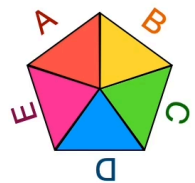


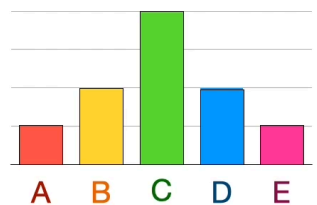
Die 2





Die 3





Cross Entropy

> *Die1이 Sequence를 가질 확률*

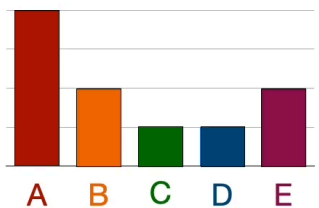
A	A	A	A	B	B	C	D	E	E

$$\log(0.4 \cdot 0.4 \cdot 0.4 \cdot 0.4 \cdot 0.2 \cdot 0.2 \cdot 0.1 \cdot 0.1 \cdot 0.2 \cdot 0.2) = -14.71$$

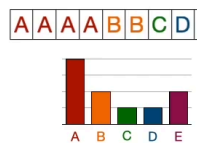
$$\log(0.4)+\log(0.4)+\log(0.4)+\log(0.4)+\log(0.2)+\log(0.2)+\log(0.1)+\log(0.1)+\log(0.2)+\log(0.2)$$

$$-\frac{4\log(0.4)+2\log(0.2)+1\log(0.1)+1\log(0.1)+2\log(0.2)}{10} = 1.471$$

Die 1

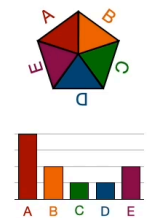


$p(A) = 0.4$
$p(B) = 0.2$
$p(C) = 0.1$
$p(D) = 0.1$
$p(E) = 0.2$



P P

$-0.4\log(0.4)$
$-0.2\log(0.2)$
$-0.1\log(0.1)$
$-0.1\log(0.1)$
$-0.2\log(0.2)$



$$-\sum_i p_i \log(p_i) = 1.471$$

Cross Entropy

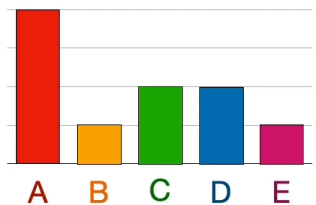
> *Die2가 Sequence를 가질 확률*

A	A	A	A	B	B	C	D	E	E
0.4	0.4	0.4	0.4	0.1	0.1	0.2	0.2	0.1	0.1

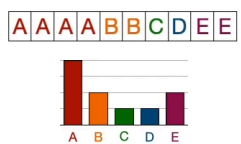
$= 0.0000001024$

$$-\frac{4\log(0.4) + 2\log(0.1) + 1\log(0.2) + 1\log(0.2) + 2\log(0.1)}{10} = 1.609$$

Die 2

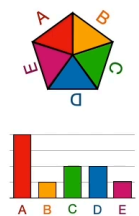


$q(A) = 0.4$
$q(B) = 0.1$
$q(C) = 0.2$
$q(D) = 0.2$
$q(E) = 0.1$



P Q

$-0.4\log(0.4)$
$-0.2\log(0.1)$
$-0.1\log(0.2)$
$-0.1\log(0.2)$
$-0.2\log(0.1)$



Cross entropy $H(P\,|\,Q) = -\sum_i p_i \log(q_i) = 1.609$

Cross Entropy

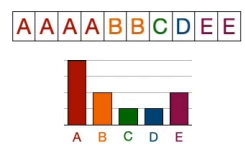
> *Die3가 Sequence를 가질 확률*

A	A	A	A	B	B	C	D	E	E
0.1	0.1	0.1	0.1	0.2	0.2	0.4	0.2	0.1	0.1

$= 0.0000000032$

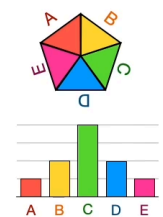
$$-\frac{4\log(0.1) + 2\log(0.2) + 1\log(0.4) + 1\log(0.2) + 2\log(0.1)}{10} = 1.956$$

$r(A) = 0.1$
$r(B) = 0.2$
$r(C) = 0.4$
$r(D) = 0.2$
$r(E) = 0.1$



P R

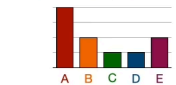
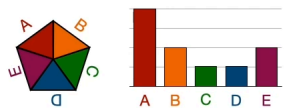
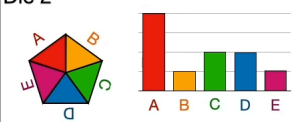
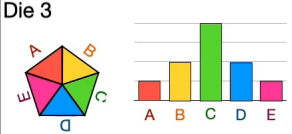
$$-0.4\log(0.1)$$
$$-0.2\log(0.2)$$
$$-0.1\log(0.4)$$
$$-0.1\log(0.2)$$
$$-0.2\log(0.1)$$



Cross entropy $H(P\,|\,R) = -\sum_i p_i \log(r_i) = 1.956$

KL-Divergence

> *Cross Entropy에서 실제 데이터 자체의 Entropy를 뺀 값 (모델이 틀린 만큼의 벌점)*

Sequence	Probability	Cross-entropy	KL-divergence
A A A A B B C D E E 			
Die 1 	$0.4^4 \cdot 0.1^2 \cdot 0.2^1 \cdot 0.2^1 \cdot 0.1^2$ 0.0000004096	$-0.4\log(0.4) - 0.2\log(0.2) - 0.1\log(0.1)$ $-0.1\log(0.1) - 0.2\log(0.2)$ $H(P) = 1.471$ (entropy)	$\mathbb{D}(P\|P) = 1.471 - 1.471$ 0
Die 2 	$0.4^4 \cdot 0.1^1 \cdot 0.2^2 \cdot 0.2^2 \cdot 0.1^1$ 0.0000001024	$-0.4\log(0.4) - 0.2\log(0.1) - 0.1\log(0.2)$ $-0.1\log(0.2) - 0.2\log(0.1)$ $H(P \mid Q) = 1.609$	$\mathbb{D}(Q\|P) = 1.609 - 1.471$ 0.138
Die 3 	$0.4^1 \cdot 0.1^2 \cdot 0.2^4 \cdot 0.2^2 \cdot 0.1^1$ 0.0000000032	$-0.4\log(0.1) - 0.2\log(0.2) - 0.1\log(0.4)$ $-0.1\log(0.2) - 0.2\log(0.1)$ $H(P \mid R) = 1.956$	$\mathbb{D}(R\|P) = 1.956 - 1.471$ 0.485

Serrano.Academy Serrano.Academy

Perplexity (혼란도)

> *언어모델이 다음 토큰을 얼마나 잘 예측하는지 측정하는 지표*

$$PPL = e^{H} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_{<i})\right)$$

학습 코드에서는 Cross Entropy를 손실 함수로 쓰지만, 논문·성능 평가표에는 Perplexity(PPL)를 적는 경우가 많음

- Cross Entropy H를 자연상수 e의 지수로 변환한 값
- 직관: 모델이 매 토큰마다 평균적으로 몇 개의 후보 중에서 고민하는지
- 값이 **낮을수록** 좋은 모델 (덜 "당황"하며 예측함)
- 완벽한 모델: 정답을 항상 확신 → $PPL = 1$
- K개 중 완전 무작위(균등분포) 예측 → $PPL = K$

1. 사람이 이해하기 직관적인 스케일

- "Loss가 2.3에서 1.1로 줄었어" ❌
- "헷갈려 하던 선택지가 10개에서 3개로 줄었어(PPL 10→3)" ✅
- 모델의 발전 수준을 훨씬 직관적으로 체감 가능

2. 미세한 성능 차이의 극대화

- $PPL = e^{H}$는 지수함수 관계
- 아주 미세한 Cross Entropy 개선도 PPL 상에서는 큰 숫자 변화로 드라마틱하게 시각화됨
- 연구자들이 성능 향상을 직관적으로 비교·검증하는 데 도움

Perplexity가 "평균 후보 개수"인 이유

> *수식을 거꾸로 풀어보면 직관이 쉽게 보인다*

1. 균등분포에서 출발

K개 후보가 모두 같은 확률 $P(x) = \frac{1}{K}$인 상황 (완전 무작위로 찍기)

$$H(X) = -\log\left(\frac{1}{K}\right) = \log K$$

$$PPL = e^{H(X)} = e^{\log K} = K$$

> 가장 혼란스러운 균등분포에서 계산하면 PPL은 정확히 후보 개수 K

2. 일반적인 경우: 기하평균의 역수

$$PPL(X) = \left(\prod_{i=1}^{N} \frac{1}{P(x_i \mid X_{<i})}\right)^{\frac{1}{N}}$$

각 단계에서 정답에 부여한 확률의 **역수**를 기하평균

단계	정답 확률 P	역수 $1/P$
1	0.5	2
2	0.25	4
3	0.125	8

$$PPL = (2 \times 4 \times 8)^{\frac{1}{3}} = 64^{\frac{1}{3}} = \mathbf{4}$$

💡 단계마다 확률이 달랐어도, 평균적으로 **"균등하게 4개 중 하나 찍는 수준"**의 혼란도와 수학적으로 동일

조건부확률: 조건이 기준 집합을 바꾼다

> *축구를 좋아하는 학생 중에서 농구를 좋아하는 학생 A: 농구를 좋아하는 학생, B: 축구를 좋아하는 학생*

전체 학생 100명 중

- 축구를 좋아하는 학생: 50명 → 사건 B
- 축구와 농구를 모두 좋아하는 학생: 30명 → 사건 $A \cap B$

"축구를 한다"는 조건이 주어지면 전체 100명이 아니라 축구를 좋아하는 50명만 새로운 기준 집합이 된다.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{30/100}{50/100} = \frac{30}{50} = \mathbf{60\%}$$

조건부확률의 분모는 전체가 아니라 **조건을 만족하는 집합**이다.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

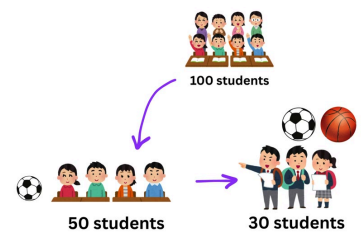
A = Play Basketball

B = Play Football

P(A∩B) → $\frac{30}{100}$

P(B) → $\frac{50}{100}$

$$P(A|B) = \frac{\left(\frac{30}{100}\right)}{\left(\frac{50}{100}\right)}$$



베이즈 정리 (Bayes' Theorem)

> *새로운 증거(evidence)로 기존의 믿음(확률)을 갱신하는 공식*

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}$$

기호	의미
$P(A)$	A가 일어날 확률
$P(B \mid A)$	A가 일어났다는 조건에서 B가 일어날 확률
$P(A \mid B)$	B가 일어났다는 조건에서 A가 일어날 확률
$P(B)$	B가 일어날 확률

조건부 확률에서 유도

- $P(A \mid B) = \frac{P(A \cap B)}{P(B)}$
- $P(B \mid A) = \frac{P(A \cap B)}{P(A)}$
- 두 번째 식에서 $P(A \cap B) = P(B \mid A)P(A)$이므로 이를 첫 번째 식에 대입하면 베이즈 정리
- $P(A \cap B)$를 바로 사용하지 않는 것은 이 경우가 희귀하거나 구하기 어려운 경우가 많음

베이즈 정리: 직관적인 예시

> *"민감도 99%인 검사에서 양성이 나오면, 진짜 병에 걸렸을까?"*

- 유병률(Prior): **1%** → 10,000명 중 환자 100명
- 민감도: $P(\text{양성} \mid \text{병}) = 99\%$
- 특이도: $P(\text{음성} \mid \neg\text{병}) = 95\%$ → 위양성률 5%

자연빈도(natural frequency)로 살펴보기 — 인구 10,000명 기준

	양성 판정	음성 판정	합계
실제 환자	**99명** (100×0.99)	1명	100명
비환자	**495명** $(9{,}900 \times 0.05)$	9,405명	9,900명
합계	**594명**	9,406명	10,000명

- 전체 양성은 **진양성 99명 + 위양성 495명 = 594명**
- 질문은 "양성 594명 중 실제 환자 99명의 비율은?"으로 바뀜
- 낮은 Prior 때문에 비환자가 훨씬 많아, 낮은 위양성률도 많은 위양성을 만듦

베이즈 정리: 수학적 계산 과정

> *자연빈도의 **인원수 비율**을 같은 구조의 **확률**로 표현*

$$\underbrace{P(\text{병} \mid \text{양성})}_{\text{양성 중 실제 환자}} = \frac{\overbrace{P(\text{양성} \mid \text{병})P(\text{병})}^{\text{진양성 확률}}}{\underbrace{P(\text{양성})}_{\text{전체 양성 확률}}}$$

분자 — 진양성: $P(\text{양성} \mid \text{병})P(\text{병}) = 0.99 \times 0.01 = 0.0099 \leftrightarrow 10{,}000$명 중 **99명**

분모 — Evidence(전체 양성): 양성은 환자와 비환자 양쪽에서 나옴

$$P(\text{양성}) = P(\text{양성} \mid \text{병})P(\text{병}) + P(\text{양성} \mid \neg\text{병})P(\neg\text{병})$$

$$= \underbrace{0.0099}_{99/10{,}000} + \underbrace{0.0495}_{495/10{,}000} = \underbrace{0.0594}_{594/10{,}000}$$

양성이라는 조건으로 범위를 좁히면

$$P(\text{병} \mid \text{양성}) = \frac{0.0099}{0.0594} = \frac{99}{594} \approx \mathbf{16.7\%}$$

베이즈 정리와 LLM 관계

> *핵심 연결고리: **확률을 통한 예측**과 **새로운 정보에 따른 믿음의 갱신***

① 다음 토큰 예측

$$P(w_t \mid w_{<t})$$

- 이전 토큰 전체를 **문맥**으로 사용
- 다음 토큰 후보의 조건부 확률 분포 계산
- 토큰이 생성될 때마다 문맥과 예측 확률이 갱신됨

② 문맥 학습

$$P(\text{답변} \mid \text{프롬프트})$$

- 사전학습 가중치: 모델의 **사전 지식**
- 지시·예시가 담긴 프롬프트: **새로운 증거**
- 증거에 따라 답변 분포가 달라지는 과정은 베이지안 갱신과 유사

③ 환각과 RAG

$$\text{Prior} + \text{Evidence} \rightarrow \text{Posterior}$$

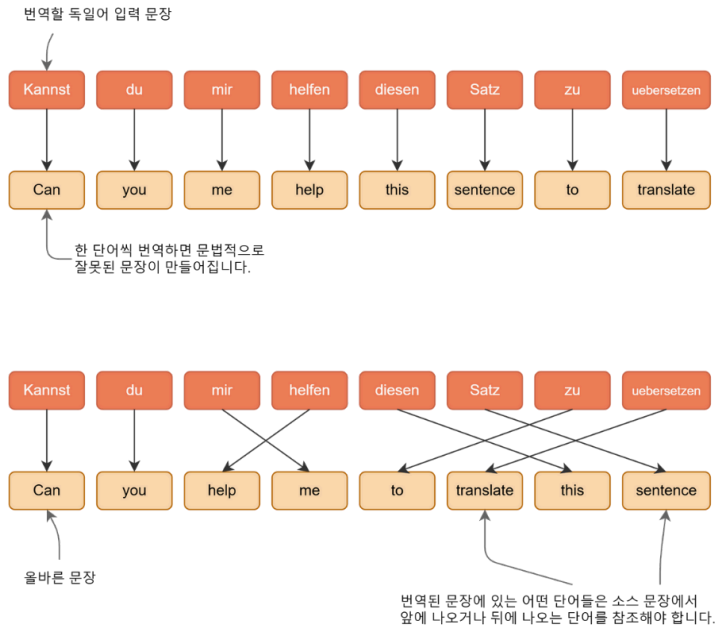
- 학습된 언어 패턴에 과도하게 의존하면 그럴듯한 오류가 발생
- 검색 결과를 **외부 증거**로 제공하면 사실에 맞게 출력 분포를 보정
- RAG는 최신·근거 정보를 문맥에 추가해 환각을 줄임

💡 **정확한 해석:** LLM이 내부에서 베이즈 정리 공식을 직접 계산하는 것은 아니다. 다만 조건부 확률로 예측하고, 문맥이라는 정보에 따라 출력 분포를 바꾼다는 점에서 **베이지안 관점으로 해석**할 수 있다.

언어 모델의 발전

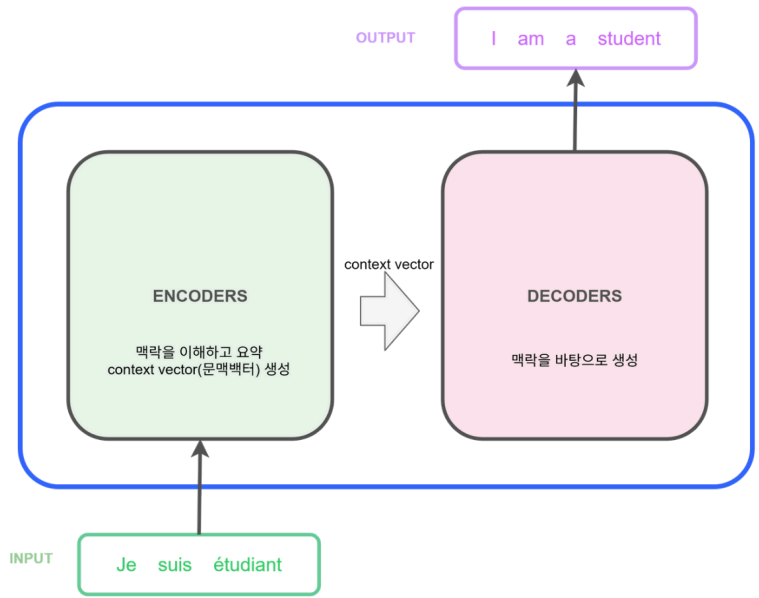
규칙기반 번역

> *형태소 분석 → 구문 분석 → 변환 → 규칙으로 어순 배열*



언어모델: Encoders, Decoders

> *입력을 이해하고, 이해를 바탕으로 생성함*



언어모델: RNN

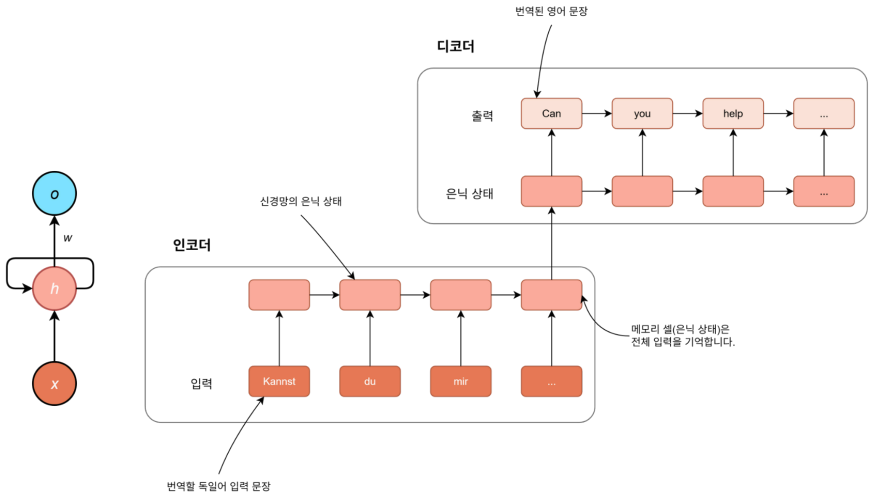
> *"문맥(Context)"을 기억하는 메모리의 탄생*

장점

- 가변 길이 처리

단점

- 길수록 잊혀진다
- 가장 마지막 정보가 가장 뚜렷하게 담긴다



언어모델: RNN Attention

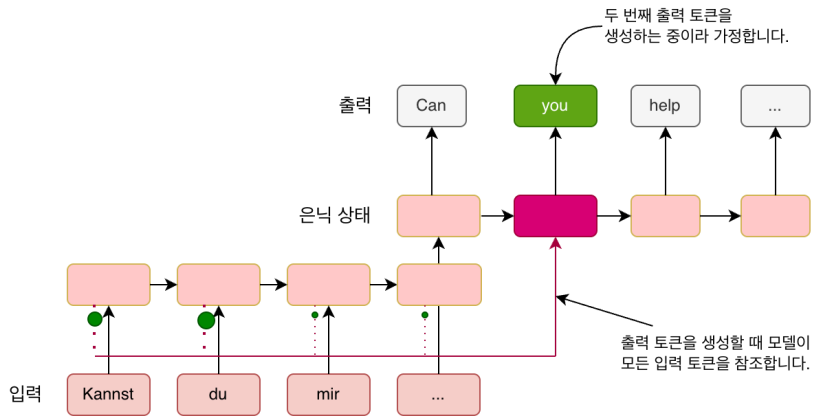
> *입력 단어들과의 Attention으로 어떤 단어를 주목할지를 학습*

장점

- 번역 정확도 향상

단점

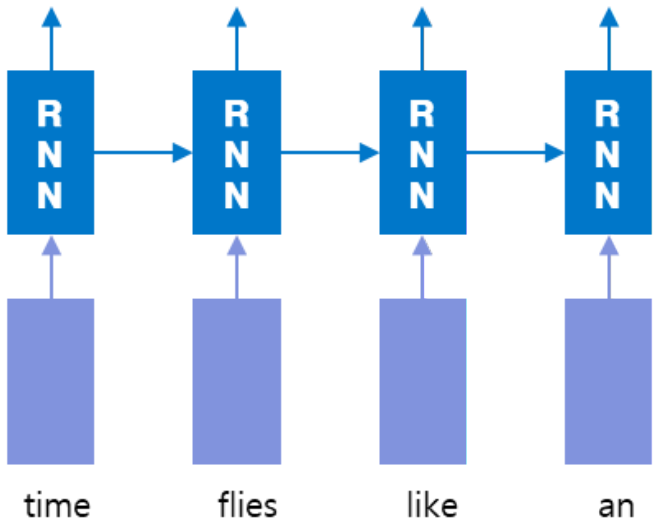
- 이전 은닉 상태 저장으로 메모리 사용 증가
- 순차 처리만 가능(병렬 불가)
- Hidden에 다른 정보가 섞여 있음



언어모델: Transformer 비교

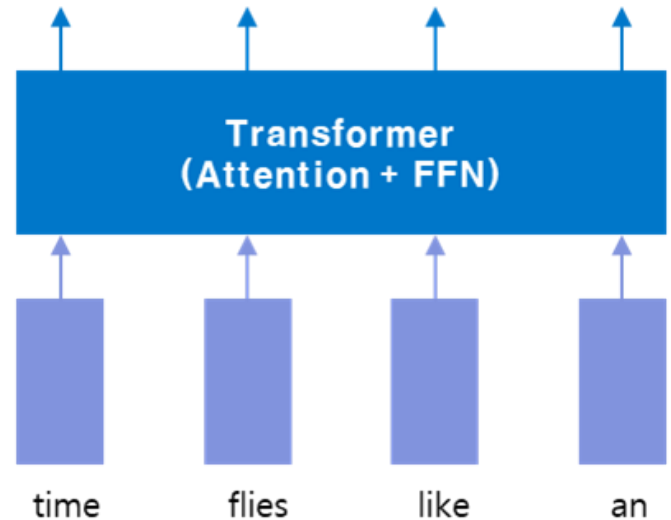
> *트랜스포머의 핵심은 어텐션(Attention)과 병렬화 가능한 아키텍처*

RNN — 순차 처리



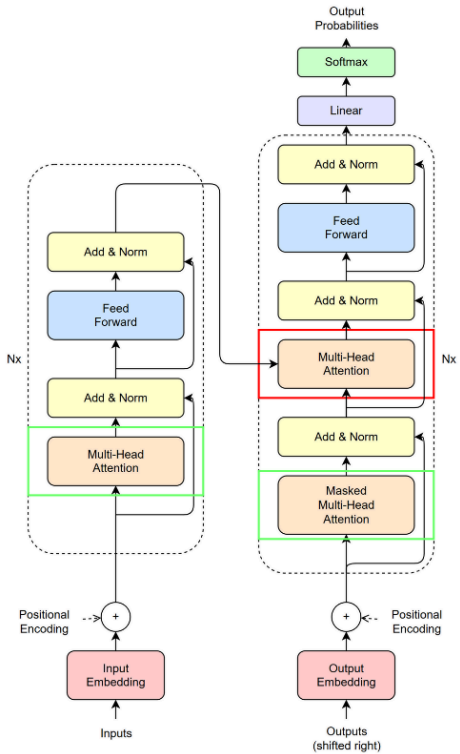
RNN is hard to parallelize and not time-efficient !

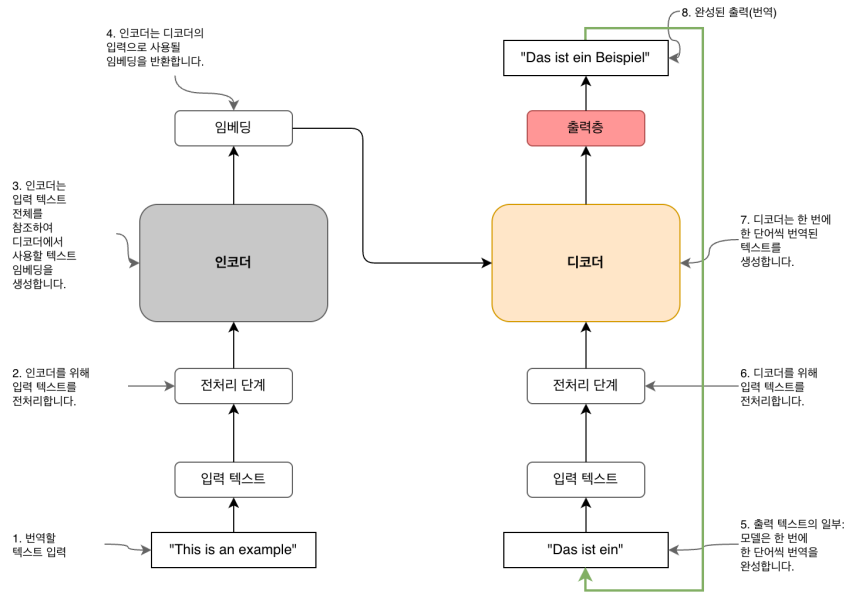
Transformer — 병렬 처리



언어모델: Transformer 구조

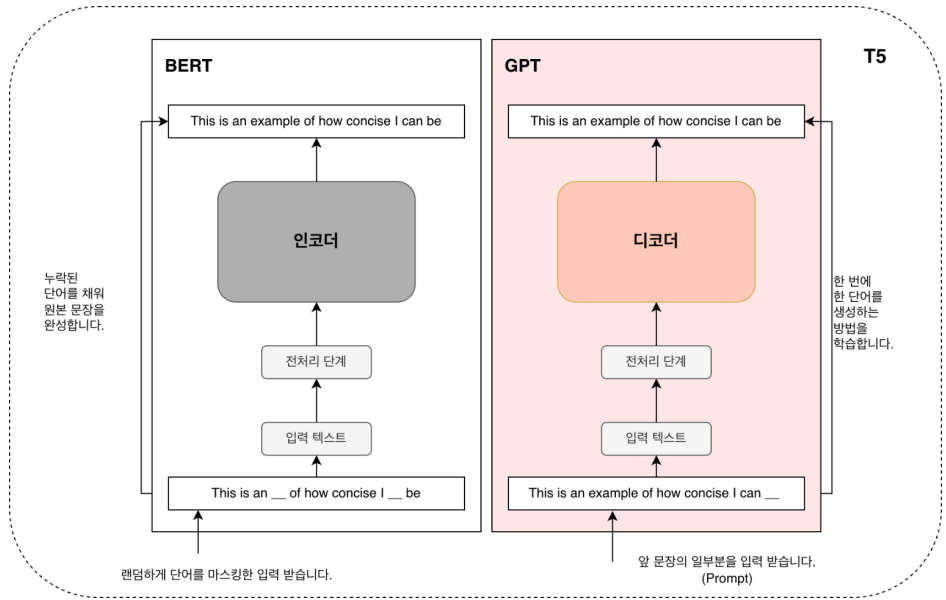
> *Attention과 FFN으로 구성*





언어모델:BERT, GPT, T5

> *Encoder만 사용하는 경우, Decoder만 사용하는 경우, 둘 다 사용하는 경우*



언어모델: BERT, GPT

BERT,T5

- Masked Language Model (MLM)
- 양방향 학습
- 빈칸 채우기 문제
- 양쪽 문맥 모두 참조 가능

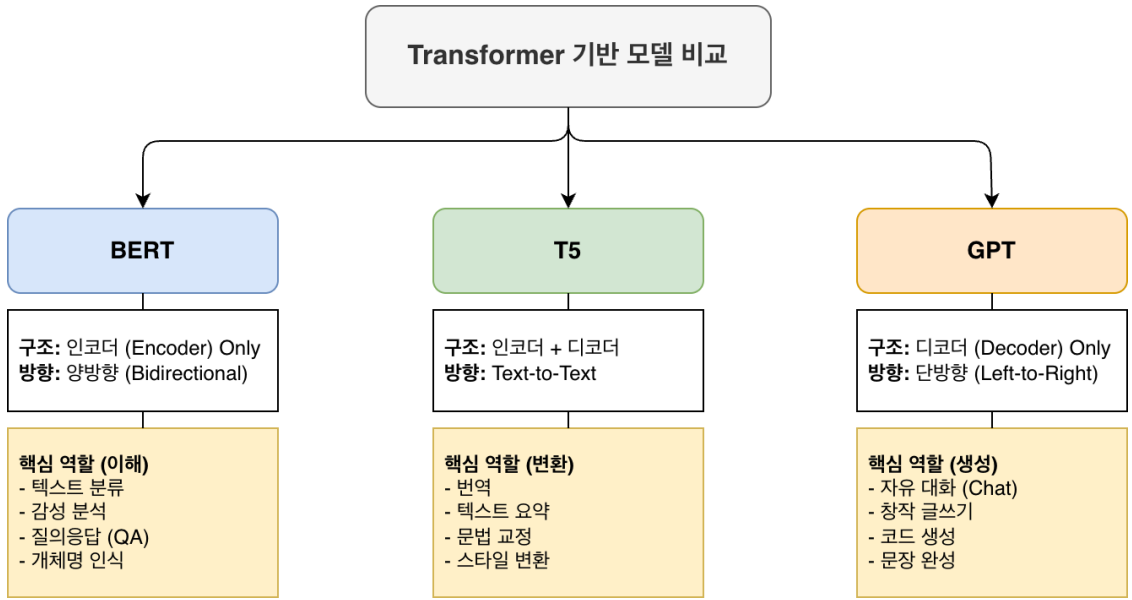
- 예시
  - 인재개발원은 _____ 에 있습니다
  - 나는 이번 학기에 _____ 강의를 수강합니다
  - BERT는 트랜스포머의 _____ 구조만을 사용합니다
  - 조성진은 한국이 낳은 훌륭한 _____ 입니다
  - 피보나치 수열은 1, 1, 2, 3, 5, 8, 13, 21, _____ 와 같다

GPT

- Causal Language Model (CLM)
- 단방향 학습
- 다음 단어 맞추기 문제
- 앞쪽 문맥만 참조 가능

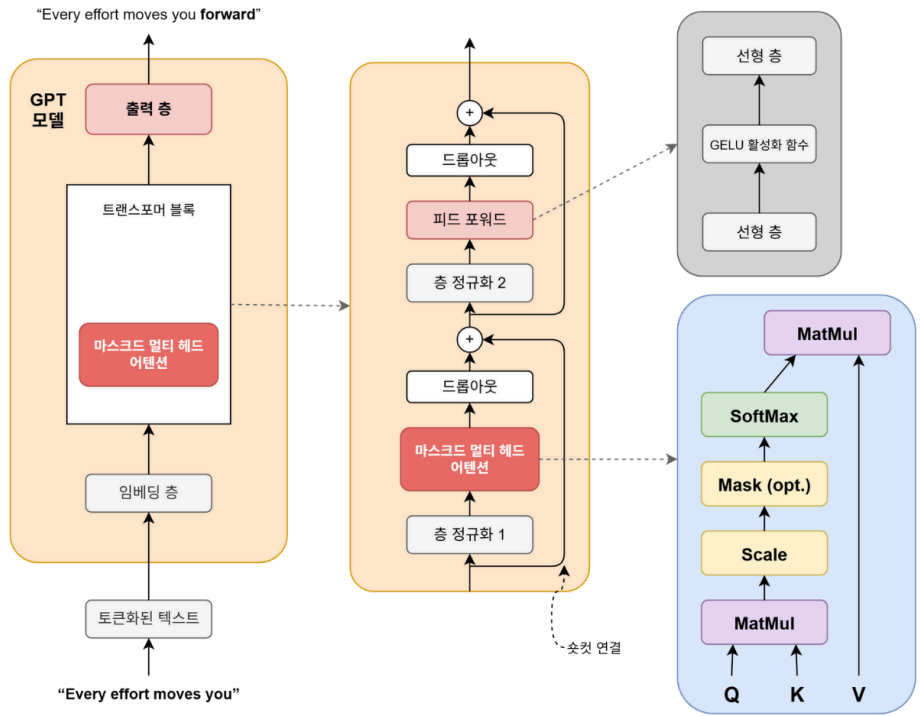
- 예시
  - 인재개발원은 _____
  - 나는 이번 학기에 딥러닝 강의를 _____
  - GPT는 트랜스포머의 _____
  - 조성진은 한국이 낳은 훌륭한 _____
  - 피보나치 수열은 1, 1, 2, 3, 5, 8, 13, 21, _____

언어 모델 비교


Transformer 기반 모델 비교
BERT
T5
GPT
구조: 인코더 (Encoder) Only
방향: 양방향 (Bidirectional)
구조: 인코더 + 디코더
방향: Text-to-Text
구조: 디코더 (Decoder) Only
방향: 단방향 (Left-to-Right)
핵심 역할 (이해)
- 텍스트 분류
- 감성 분석
- 질의응답 (QA)
- 개체명 인식
핵심 역할 (변환)
- 번역
- 텍스트 요약
- 문법 교정
- 스타일 변환
핵심 역할 (생성)
- 자유 대화 (Chat)
- 창작 글쓰기
- 코드 생성
- 문장 완성

언어 모델: GPT

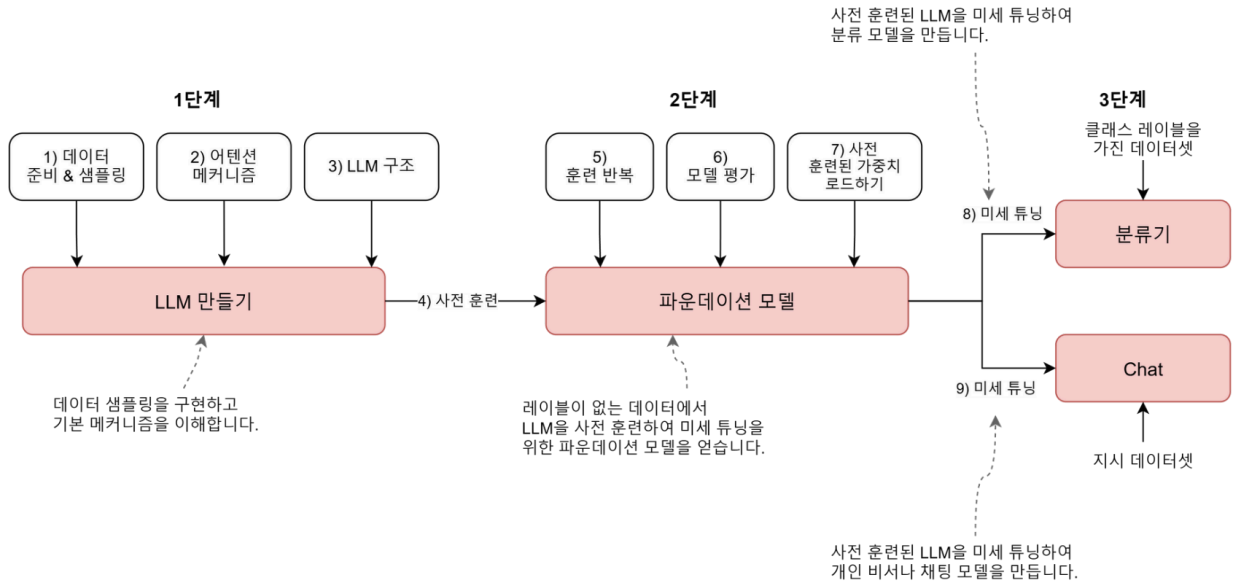
Decoder만 사용



언어 모델: 입력

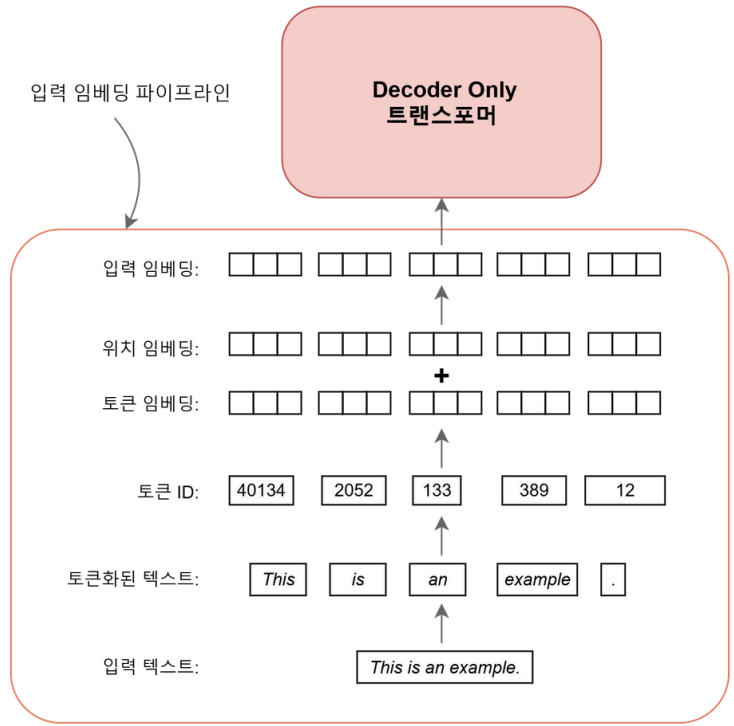
모델 학습

> *파운데이션 모델 → Chat 모델*



데이터 입력: 전처리

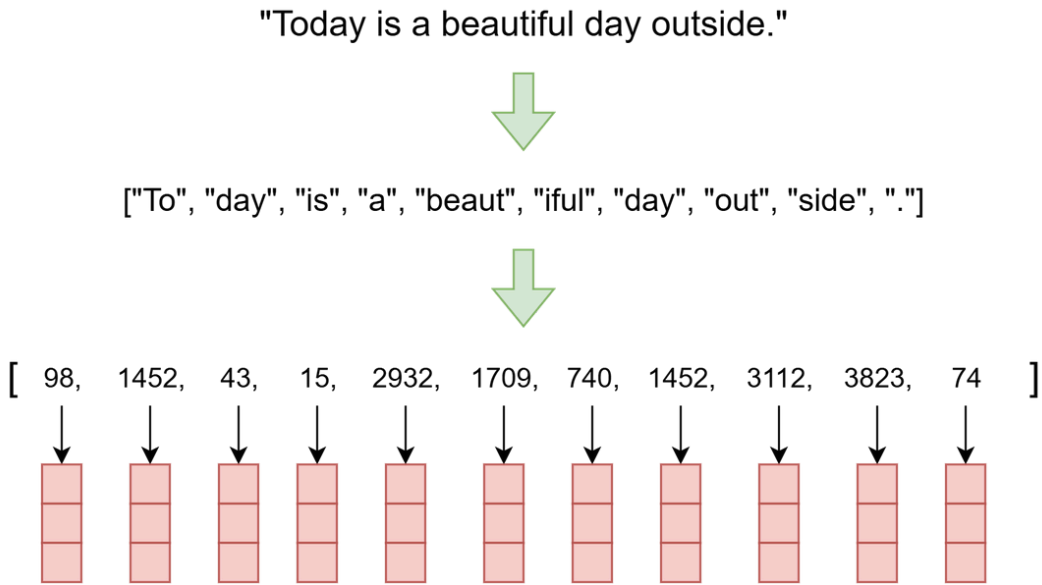
모델에 입력하기 전에 전처리 과정



데이터 입력: Tokenizer

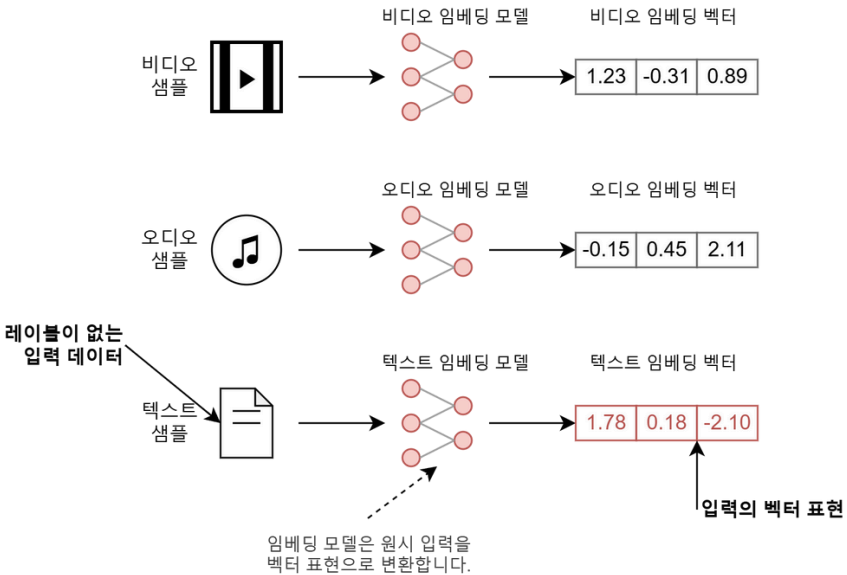
> *입력 문장을 의미 단위로 잘라서 고유 번호를 부여*

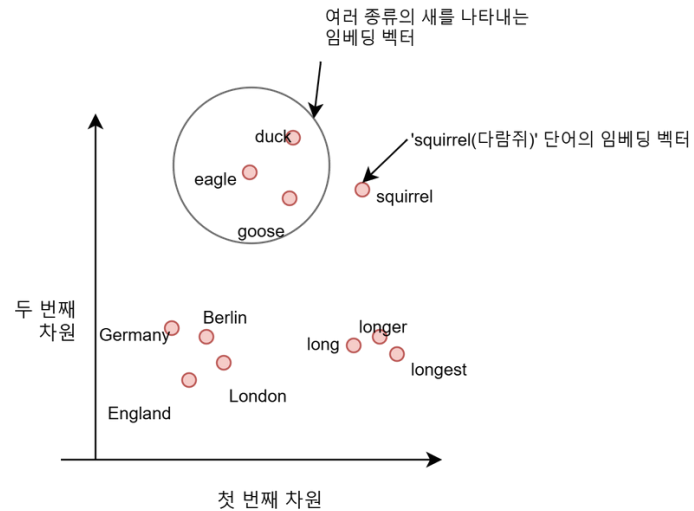
- 자주 쓰이는 단어는 가능하면 길게: 데이터 압축을 높일 수 있음
- 가능하면 의미 단위로 자름



데이터 입력: Embedding

> *어떤 데이터를 벡터로 표현한 것*

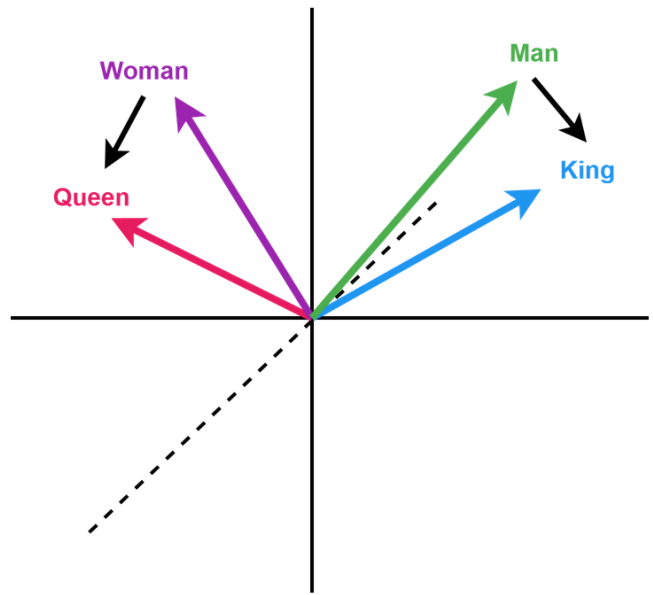




데이터입력: 벡터 공간

> *비슷한 의미를 가진 단어는 비슷한 공간에 위치*

- 단어간 의미를 계산할 수 있음
- Woman – Queen ≈ Man - King

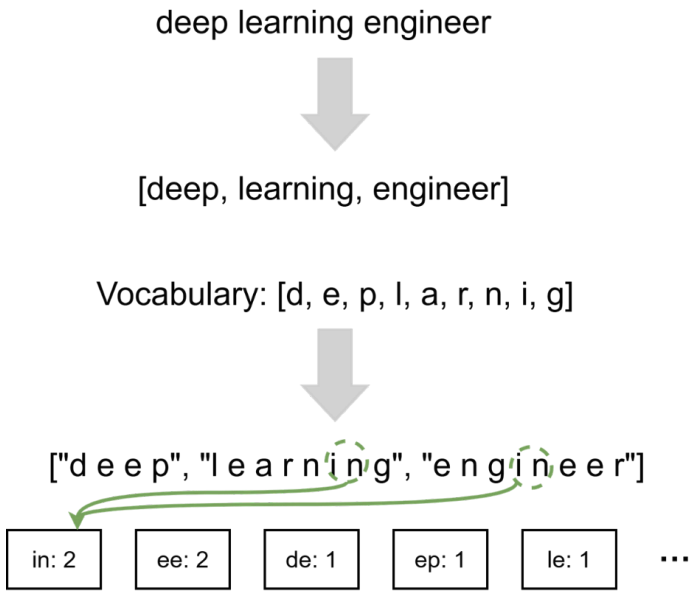


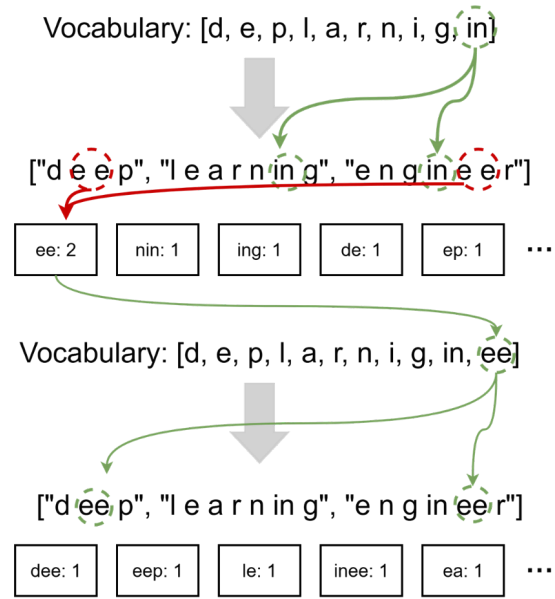
백터 공간 실습

- Chapter_1_Exercise_Vector Space.ipynb
- Word Embedding Demo
  - Soldier는 어느 위치일까?
- NLPL Calculator
  - Korea_PROPN, Busan_PROPN 을 입력한 후 Japan_PROPN을 입력하면 뭐가 나올까요?
- NLPL Associates
  - mother을 입력하면 연관된 단어는 어떤 것이 나올까요?
- NLPL Visualization
  - car, tank, transport, computer, mouse, Moscow, Paris, London, Russia, France, Britain, clean, dirty, new 를 입력하면 어떻게 분류가 될까요?

데이터입력: BPE Tokenizer

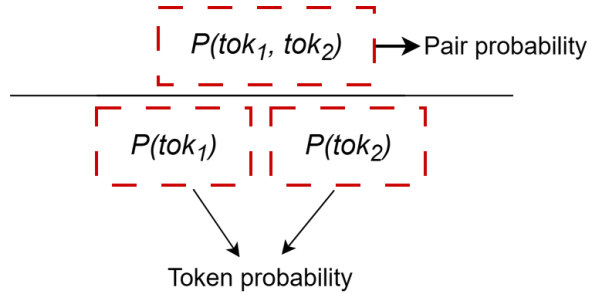
> *가장 많이 등장하는 쌍 찾아서 추가*



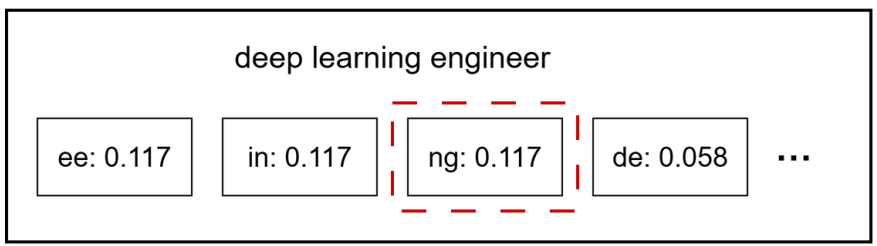


데이터입력: Wordpiece Tokenizer

> *각각 나올 확률보다 쌍으로 나올 확률이 높으면 추가*



- (ng가 나올 확률)
- n이 나올 확률)(g가 나올 확률)



데이터입력: Unigram Tokenizer

> *최선의 조합을 찾고, 쓸모없는 것은 버림*

```
("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
("h", 15) ("u", 36) ("g", 20) ("hu", 15) ("ug", 20) ("p", 17) ("pu", 17) ("n", 16)
("un", 16) ("b", 4) ("bu", 4) ("s", 5) ("hug", 15) ("gs", 5) ("ugs", 5)
```

$P(["p", "u", "g"]) = P("p") \times P("u") \times P("g") = {}^{5}/_{210} \times {}^{36}/_{210} \times {}^{20}/_{210} = 0.000389$

$P(["pu", "g"]) = P("pu") \times P("g") = {}^{5}/_{210} \times {}^{20}/_{210} = 0.0022676$

```
"hug": ["hug"] (score 0.071428)
"pug": ["pu", "g"] (score 0.007710)
"pun": ["pu", "n"] (score 0.006168)
"bun": ["bu", "n"] (score 0.001451)
"hugs": ["hug", "s"] (score 0.001701)
```

} Best 조합

hug는 유지

```
"hug": ["hug"] (score 0.071428)
"hugs": ["hug", "s"] (score 0.001701)
```

```
"hug": ["hu", "g"] (score 0.006802)
"hugs": ["hu", "gs"] (score 0.001701)
```

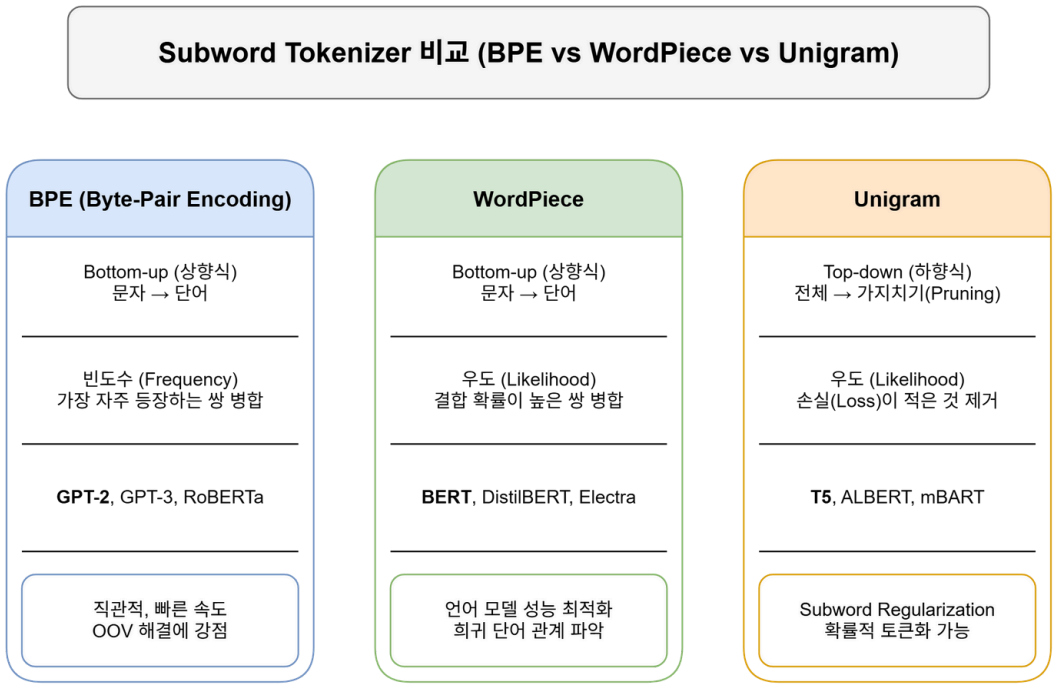
pu는 삭제

```
"pug": ["pu", "g"] (score 0.0022676)
"pun": ["pu", "n"] (score 0.0061678)
```

```
"pug": ["p", "ug"] (score 0.0022676)
"pun": ["p", "un"] (score 0.0061678)
```

- 확률에 따라 토큰화 결과를 매번 조금씩 다르게 샘플링
- playing → play + ing 또는 pl + ay + ing
- 통계적 특성을 더 잘 반영하는 효율적인 어휘 사전을 구축
- 반면에 속도는 느림

데이터 입력 – Tokenizer



위치 인코딩

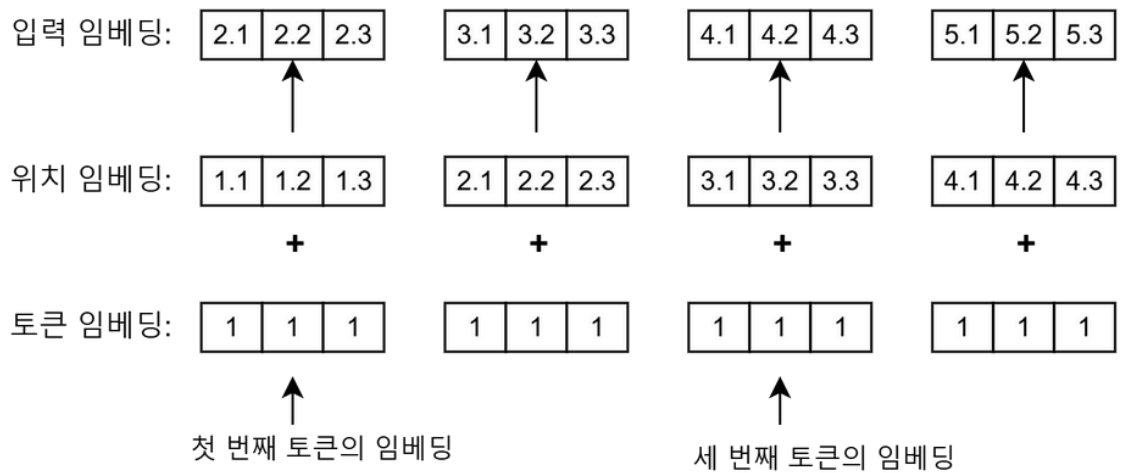
> *Token에는 단어의 위치 정보가 없음*

필요성

- Tom ate the meat ≠The meat ate Tom
- 단어의 위치에 따라 의미가 달라짐

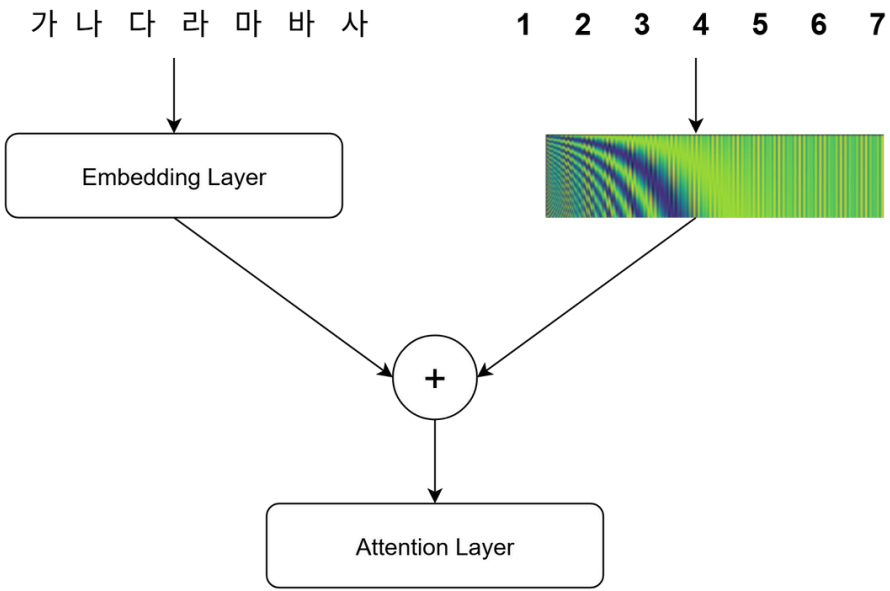
조건

- 시퀀스 길이에 관계없이 동일한 위치
- 의미적 유사성보다 위치적 유사성이 더 강조되지 않도록 크기가 제한



절대 위치 인코딩

> *단어의 위치를 절대값으로 표기*



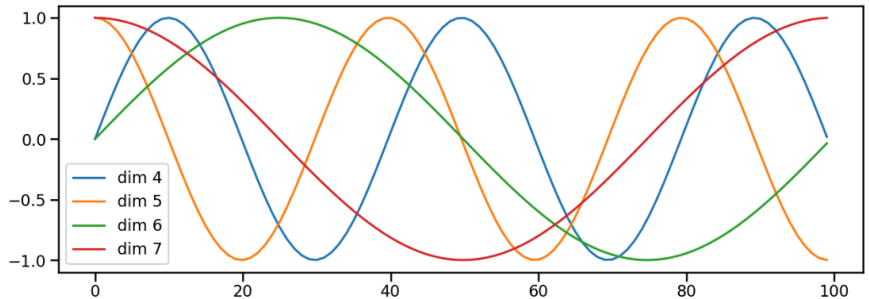
$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

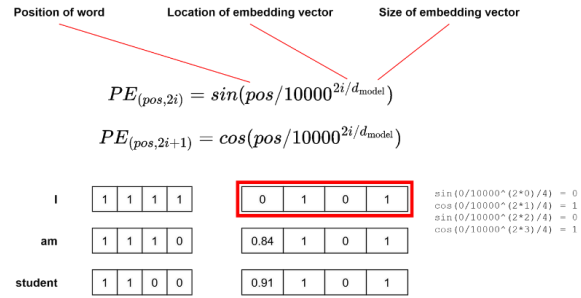
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Sinusoidal Positional Encoding

> *절대 위치: 삼각함수*

- 낮은 차원: 주기가 짧아서 빠르게 변함(세밀한 위치 구분)
- 높은 차원: 주기가 매우 길어서 문장이 길어져도 반복되지 않음
- 결과: 수만 단어 길이의 문장에서도 유일한 위치 지문을 만들 수 있음

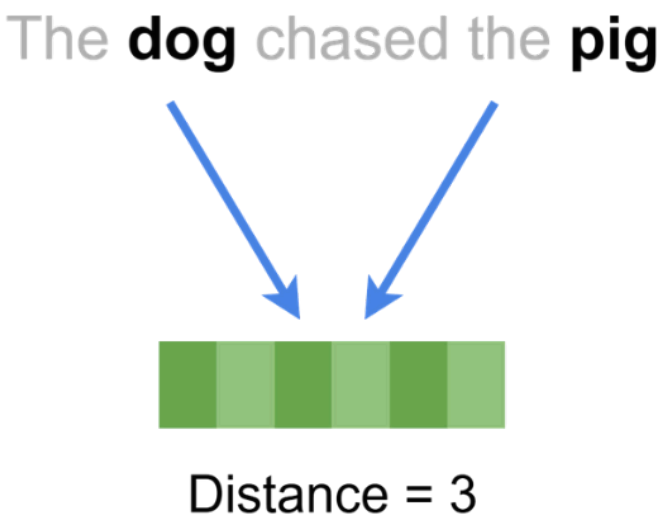


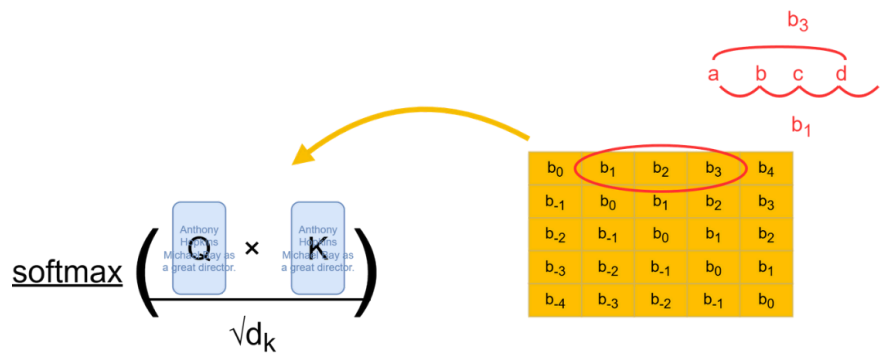


상대 위치 인코딩

> *단어와 단어 사이의 거리를 임베딩*

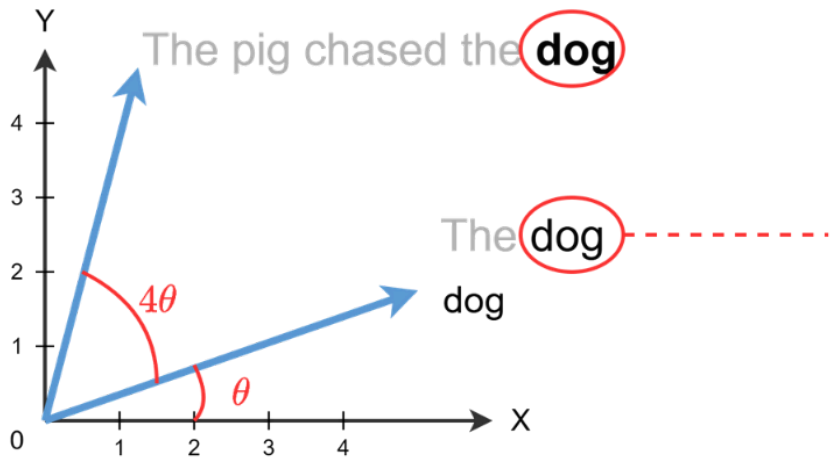
- 위치 편향을 bias로 더함
- 위치 임베딩의 일관성 유지
- 속도가 느림: 위치를 계산해서 더해야 함, 매 스텝마다 값이 변해서 캐싱이 어려움



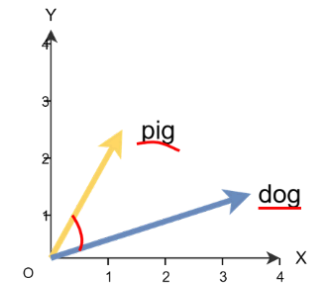


ROPE

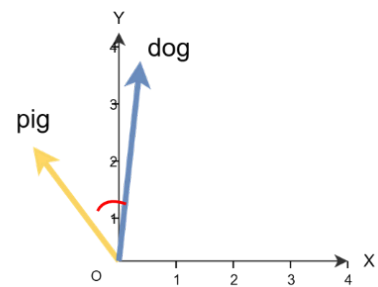
> *회전으로 단어의 위치를 임베딩함*



The **pig** chased the **dog**

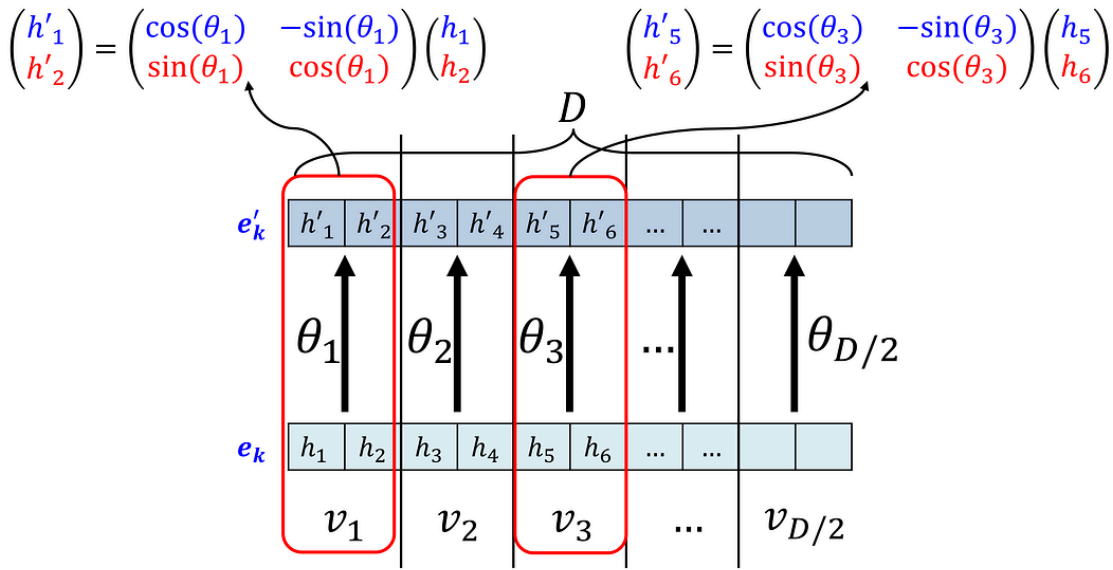


Once upon a time, the **pig** chased the **dog**



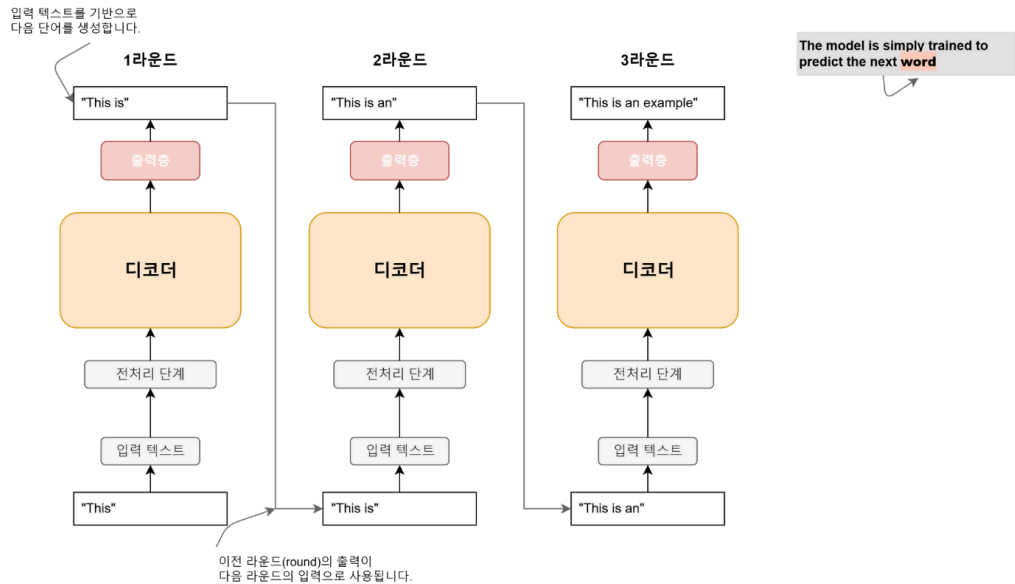
ROPE

2개씩 짝지어서 각도를 다르게 회전함



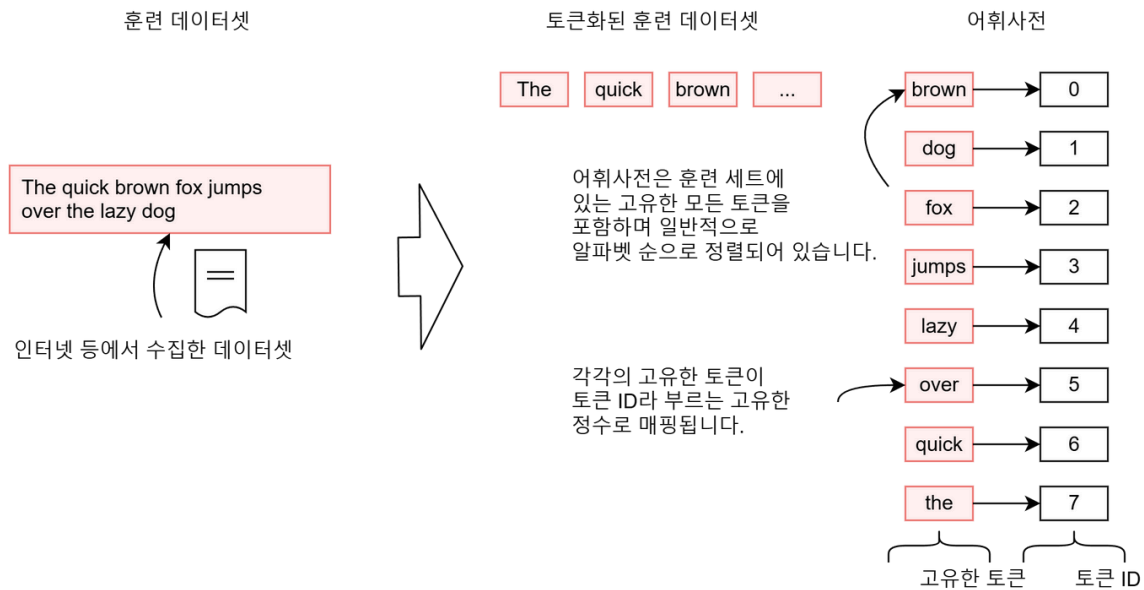
Text 생성

> *다음 단어를 예측: Autoregressive*



Token화

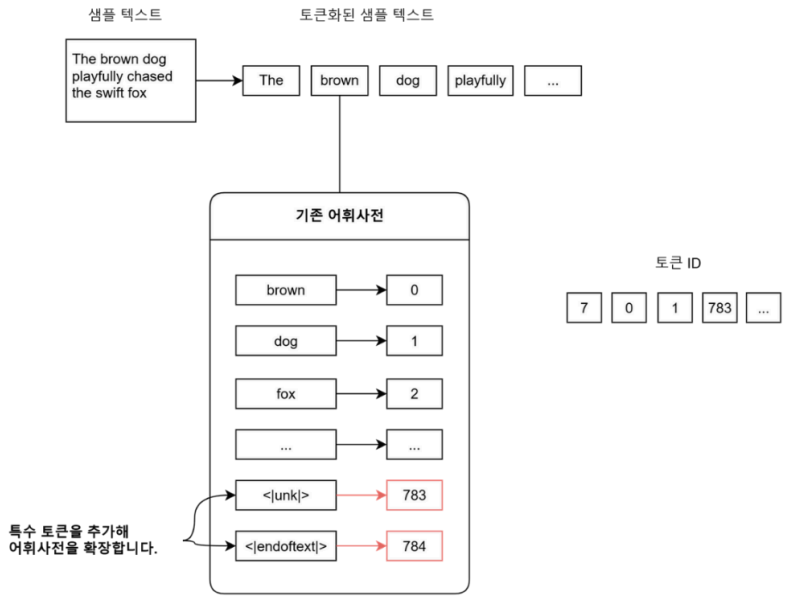
데이터셋을 Token으로 변경



Special Token

> *모델을 제어하고 입력·출력의 구조를 정의하는 제어 신호*

- 학습 대상에 포함(단 Padding Token은 제외)



Special Token

> *모델마다 조금씩 다를 수 있음*

1. 문장의 시작과 끝
Sequence Control

- **<BOS>**
  문장의 시작
- **<EOS>**
  문장의 끝 (생성 중지)
- **<PAD>**
  길이 맞추기 (패딩)

2. 대화 구조 제어
Chat/Instruction

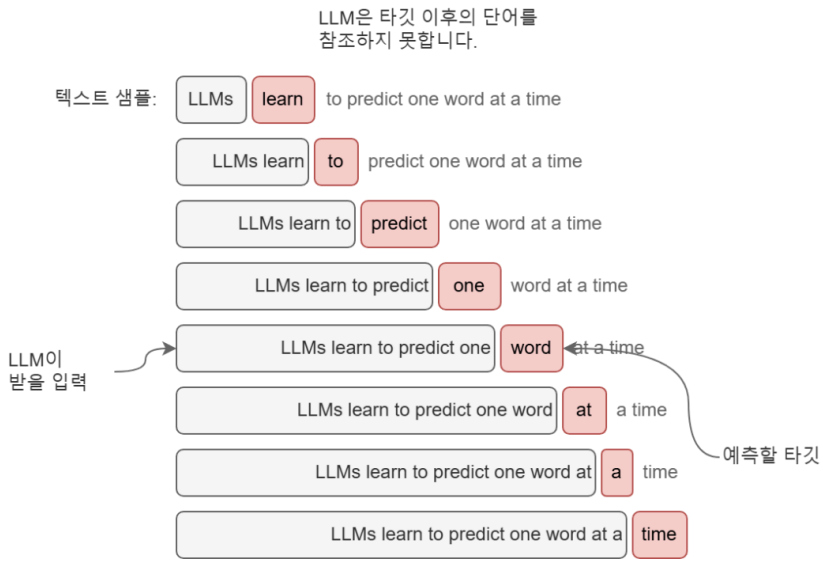
- **<|user|>**
  사용자 질문 시작
- **<|assistant|>**
  AI 답변 시작
- **<|system|>**
  시스템 프롬프트 (페르소나)

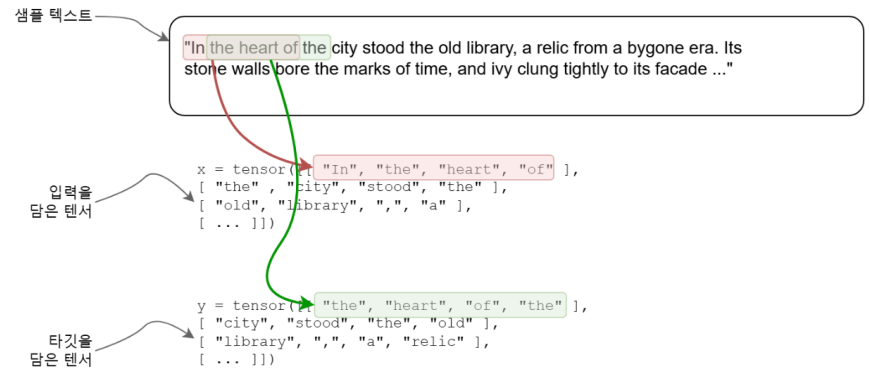
3. 기능 수행 및 사고
Task-Specific

- **<|thought|>**
  사고 과정 (CoT)
- **<|file_separator|>**
  파일 경계 구분
- **<|tool_call|>**
  외부 도구 호출

데이터셋

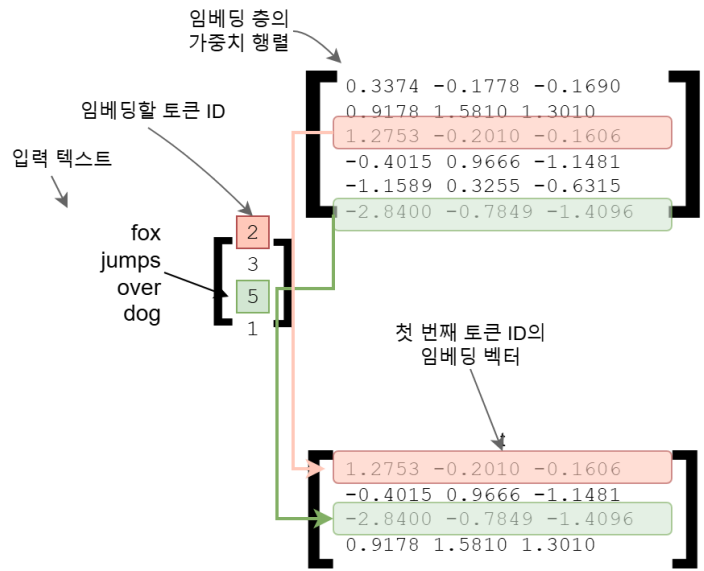
Self-Supervised: Label 없이 생성





Token 임베딩

Token id를 연속 벡터 공간의 점으로 맵핑



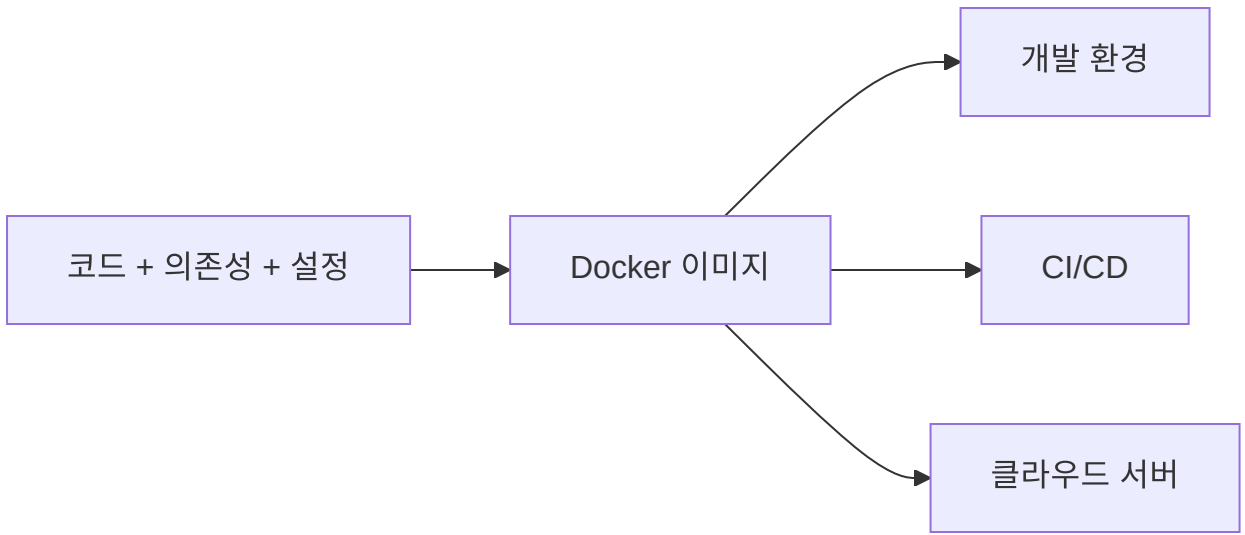
실습

- Chapter_2_Exercise_Dataset.ipynb

Docker: 애플리케이션과 실행 환경을 함께 배포

> *코드·라이브러리·의존성·기본 설정을 이미지로 묶고, 컨테이너로 실행한다*

> “내 컴퓨터에서는 되는데 서버에서는 왜 안 되지?”

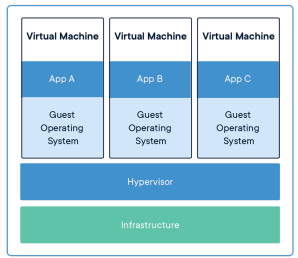


Docker는 컨테이너를 빌드·배포·실행하는 플랫폼이며, Docker Engine은 오픈소스 기반이다.

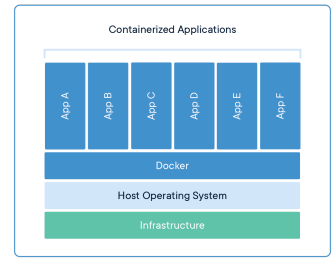
같은 이미지를 사용해 환경 차이를 줄인다. CPU 아키텍처·커널·GPU 드라이버 호환성은 함께 확인한다.

컨테이너 vs 가상머신

가상머신(VM)



Linux 컨테이너



출처: Docker — What is a Container?

비교	가상머신(VM)	Linux 컨테이너
실행 구조	하이퍼바이저 위에 게스트 OS 실행	호스트의 Linux 커널 공유
격리 단위	가상 하드웨어 + OS	프로세스·파일 시스템·네트워크 등
시작 과정	게스트 OS 부팅	컨테이너의 시작 프로세스 실행
자원 비용	게스트 OS별 메모리·디스크 필요	OS 커널 공유로 오버헤드 감소
크기·속도	OS와 구성에 따라 달라짐	보통 빠르게 시작하지만 이미지·앱에 따라 다름

컨테이너도 CPU·메모리를 사용하며, ML 이미지는 수 GB 이상일 수 있다.

Mac·Windows의 Docker Desktop에서 Linux 컨테이너는 일반적으로 Linux VM을 통해 실행한다.

세 가지 핵심 개념

개념	담는 것·하는 일
Dockerfile	베이스 이미지, 패키지 설치, 파일 복사, 기본 실행 명령
Image	실행에 필요한 파일과 메타데이터를 담은 읽기 전용 레이어
Container	이미지에 쓰기 가능한 레이어를 더한 실행 인스턴스

컨테이너는 정지 상태로도 존재한다. 하나의 이미지로 여러 컨테이너를 만들 수 있다.

장기 보존 데이터는 볼륨이나 호스트 디렉터리에 저장한다.

Docker를 사용하는 이유

장점	실무에서의 효과
환경 일관성	로컬·CI·서버에서 동일 이미지 재사용
의존성 격리	서로 다른 Python·Node.js 버전 등을 분리해 운영
배포 단순화	이미지 내려받기 → 환경 변수·스토리지 연결 → 실행
확장 용이성	같은 이미지의 인스턴스를 여러 개 실행; Kubernetes 등과 연계
이미지 생태계	Docker Hub의 Ubuntu·Nginx·Redis·PyTorch 등 활용

독립된 컨테이너라도 **같은 호스트 포트를 중복 공개하면 충돌**할 수 있다.

재현성을 높이려면 이미지 태그·digest와 패키지 버전을 함께 관리한다.

자주 쓰는 기본 명령어

```
# 이미지 내려받기
docker pull ubuntu:latest

# 백그라운드 실행: 호스트 8080 → 컨테이너 80
docker run -d --name my-web -p 127.0.0.1:8080:80 nginx

# 실행 상태 확인 / 내부 쉘 접속
docker ps
docker exec -it my-web /bin/sh

# 정지 / 삭제
docker stop my-web
docker rm my-web
```

브라우저에서 `http://localhost:8080` 접속 · `docker ps -a` 는 정지된 컨테이너도 표시

출처: Docker run · 포트 공개

베이스 이미지와 쉘 설정

```
ARG BASE_IMAGE=runpod/pytorch:1.4.0-cu1281-torch2121-ubuntu2404
FROM ${BASE_IMAGE}

SHELL ["/bin/bash", "-o", "pipefail", "-c"]
```

구문	의미
`ARG BASE_IMAGE`	빌드 시 베이스 이미지 변경 가능
`FROM`	기존 이미지의 파일·라이브러리·설정 상속
`pipefail`	파이프 앞쪽 명령의 실패도 파이프라인 종료 상태에 반영
`&&`	앞 단계가 성공해야 다음 명령 실행

태그 문자열만으로 정확한 PyTorch 버전을 추정하지 않는다.

실제 버전은 이미지 메타데이터 또는 `python -c 'import torch; print(torch.__version__)'` 로 확인한다.

code-server 설치와 확장 번들링

> *릴리스 다운로드 → SHA-256 확인 → `/opt/code-server` 설치 → 실행 경로 연결*

```
RUN code-server --config /dev/null \
    --extensions-dir /opt/code-server/lib/vscode/extensions \
    --install-extension openai.chatgpt \
    --install-extension ms-python.python \
    --install-extension ms-toolsai.jupyter
```

- standalone 배포본에 Node.js 포함; `/usr/local/bin`의 링크로 명령 호출
- 체크섬은 받은 파일이 **지정된 배포본과 일치하는지** 확인
- 확장을 `/opt`에 두면 `/workspace` 마운트에 가려지지 않음
- /workspace는 사용자의 volume으로 마운트되어 있음

마운트는 기존 파일을 삭제하는 것이 아니라 해당 경로에서 보이지 않게 가린다.

출처: Docker bind mounts

작업 경로와 컨테이너 시작점

```
ENV UV_LINK_MODE=hardlink \
    JUPYTER_PORT=8888 \
    CODE_SERVER_WORKSPACE=/workspace \
    CODE_SERVER_DATA_DIR=/workspace/.code-server \
    CODE_SERVER_LOG=/var/log/code-server.log

COPY --from=scripts --chmod=755 start.sh /start.sh
COPY pre_start.sh /pre_start.sh
WORKDIR /workspace
EXPOSE 8888 8080
ENTRYPOINT ["/start.sh"]
CMD []
```

- `UV_LINK_MODE` : uv의 파일 연결 모드 지정; 효과는 파일 시스템 조건에 따라 다름
- `COPY --from=scripts` : 별도 빌드 컨텍스트·스테이지 등 `scripts` 소스 필요
- `/start.sh` → `pre_start.sh` hook → SSH·JupyterLab; code-server 실행은 스크립트 확인
- `EXPOSE` 는 포트 메타데이터이며, **외부 공개는** `docker run -p` **로 설정**

로컬 GPU 환경에서 실행하기

전제: 이미지 빌드 완료 + NVIDIA GPU·드라이버·Container Toolkit 구성

```
mkdir -p workspace

docker run -d \
  --name mujoco-workspace \
  --gpus all \
  --ipc=host \
  -p 127.0.0.1:8888:8888 \
  -v "$(pwd)/workspace:/workspace" \
  -e JUPYTER_PORT=8888 \
  mujoco-dev:local
```

실행 후 **http://localhost:8888** 접속

`mujoco-dev:local` 은 빌드한 이미지 이름

출처: Docker run 옵션 · Bind mounts

핵심 옵션	의미
`--gpus all`	컨테이너에서 호스트 GPU 사용
`--ipc=host`	호스트의 공유 메모리 사용
`-p 호스트IP:포트:컨테이너 포트`	로컬 포트를 컨테이너 서비스에 연결
`-v 호스트경로:컨테이너경로`	작업 폴더 연결·결과 파일 보존
`-e KEY=value`	컨테이너에 환경 변수 전달

연결한 작업 폴더의 파일은 컨테이너를 삭제해도 남는다.

RunPod vs Vast.ai

> *머신러닝·딥러닝용 GPU를 필요한 기간만 임대하는 클라우드 플랫폼*

RunPod

Secure / Community Cloud 선택

Pod·스토리지·Serverless를 연결하는 개발 환경

운영 편의성과 서비스 구성을 중심으로 선택

Vast.ai

다양한 호스트의 GPU를 비교하는 마켓플레이스

호스트별 가격·성능·네트워크 조건을 직접 비교

가격과 자원 조건을 중심으로 선택

AWS·GCP 등과 비교할 때도 **동일한 GPU·임대 조건·전체 비용**으로 판단한다.

출처: RunPod · Vast.ai

1. 인프라 운영 방식

RunPod

- **Secure Cloud:** 기업용 워크로드를 위한 인프라 유형
- **Community Cloud:** 커뮤니티 공급자의 GPU 자원
- 작업 요구사항과 예산에 맞춰 유형 선택
- 장시간 작업도 장애 대응·체크포인트 필요

Vast.ai

- 데이터센터·사업자·개인 등 다양한 호스트의 자원 비교
- GPU뿐 아니라 CPU·디스크·네트워크 조건도 확인
- **Reliability 지표**와 **업로드·다운로드 속도**를 각각 확인
- 호스트별 품질 차이를 임대 선택에 반영

GPU 모델이 같아도 실제 작업 성능은 다를 수 있다.
데이터 로딩·모델 다운로드·체크포인트 저장에는 CPU, 디스크, 네트워크도 영향을 준다.

출처: RunPod Secure Cloud · Vast.ai FAQ

2. 가격과 비용 정책

비교 관점	RunPod	Vast.ai
단가 확인	GPU·제품·임대 유형별 단가	동일 GPU도 호스트별 매물 가격 비교
비용 절감	사용 시간·유휴 자원·운영 방식 최적화	저렴한 매물 탐색·Interruptible 활용
예산 산정	컴퓨트 + 저장 공간 등	컴퓨트 + 저장 공간 + 대역폭 등

Interruptible = 선점·중단될 수 있는 임대 방식

높은 입찰이나 On-demand 임대로 중단될 수 있으므로 체크포인트에서 재시작할 수 있어야 한다.

Serverless도 실행 시간에 비용이 발생한다.
RunPod는 단순 API 호출 횟수 대신 작업자의 실행 시간을 기준으로 과금한다.
시작·처리·유휴 대기 시간과 상시 유지 작업자 설정이 비용에 영향을 준다.

출처: RunPod 요금 · Serverless 과금 · Vast.ai 임대 방식

4. 어떤 서비스가 내 작업에 맞을까?

Vast.ai를 우선 비교

- **비용 절감이 최우선**인 개인 연구·실험
- 소비자용 GPU를 활용하는 LoRA·이미지 생성
- 저렴한 매물을 직접 비교할 수 있는 경우
- 중단 후 복구 가능한 배치 학습

RunPod를 우선 비교

- **환경 구성과 운영 편의성**이 중요한 경우
- 지속적인 Jupyter·SSH 개발 작업
- 볼륨을 재사용하며 학습·실험을 반복
- Pod 개발에서 Serverless 배포로 이어지는 작업

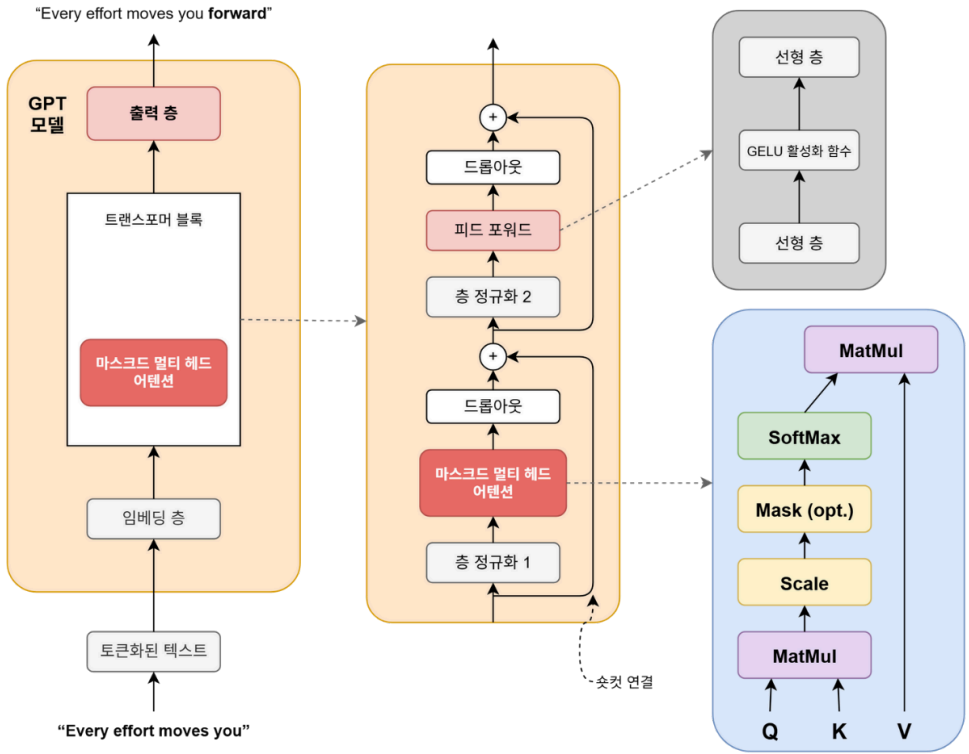
장시간 학습·프로덕션은 실제 성능, 장애 대응, 저장소, 운영 조건까지 확인한다.

두 서비스 모두 Serverless를 제공하므로 API 서빙은 기능과 전체 비용을 함께 비교한다.

선택 가이드는 앞선 기능·비용 비교를 바탕으로 한 권고입니다. 실제 가격과 가용 자원은 배포 시 확인하세요.

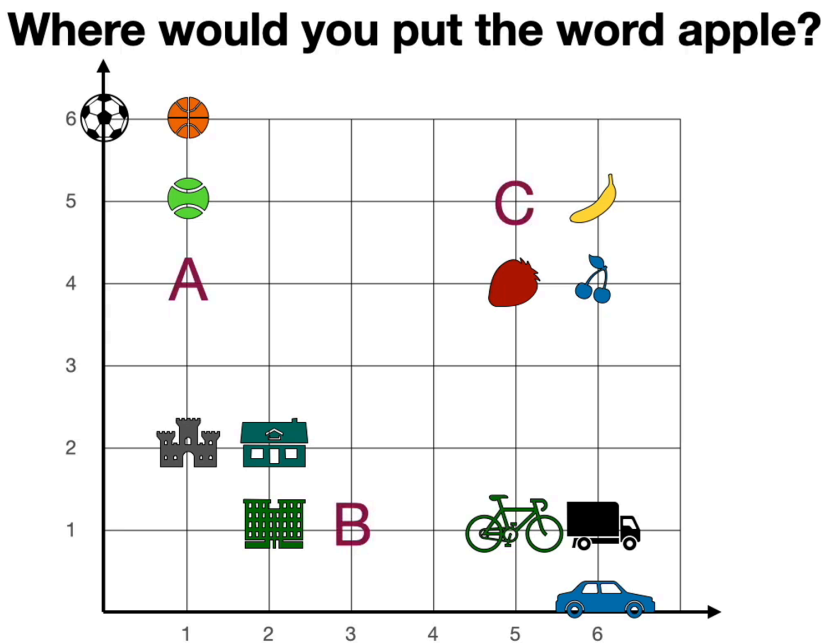
Attention

GPT: Attention



GPT: Attention

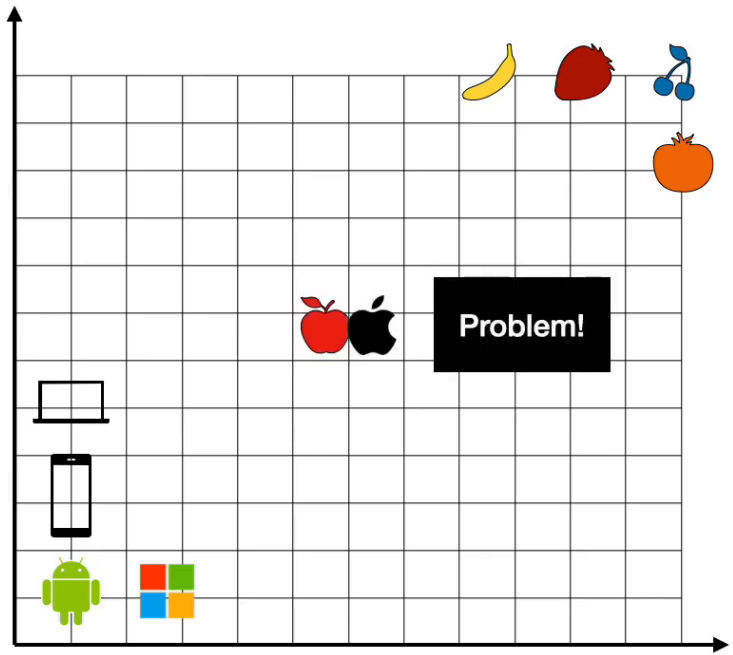
> *애플은 어디에 위치할까요?*



Word	Numbers	
Apple	?	?
Banana	6	5
Strawberry	5	4
Cherry	6	4
Soccer	0	6
Basketball	1	6
Tennis	1	5
Castle	1	2
House	2	2
Building	2	1
Bicycle	5	1
Truck	6	1
Car	6	0

GPT: Attention

> *애플은 어디에 위치할까요?*



Top right or bottom left?

Cherry

Android

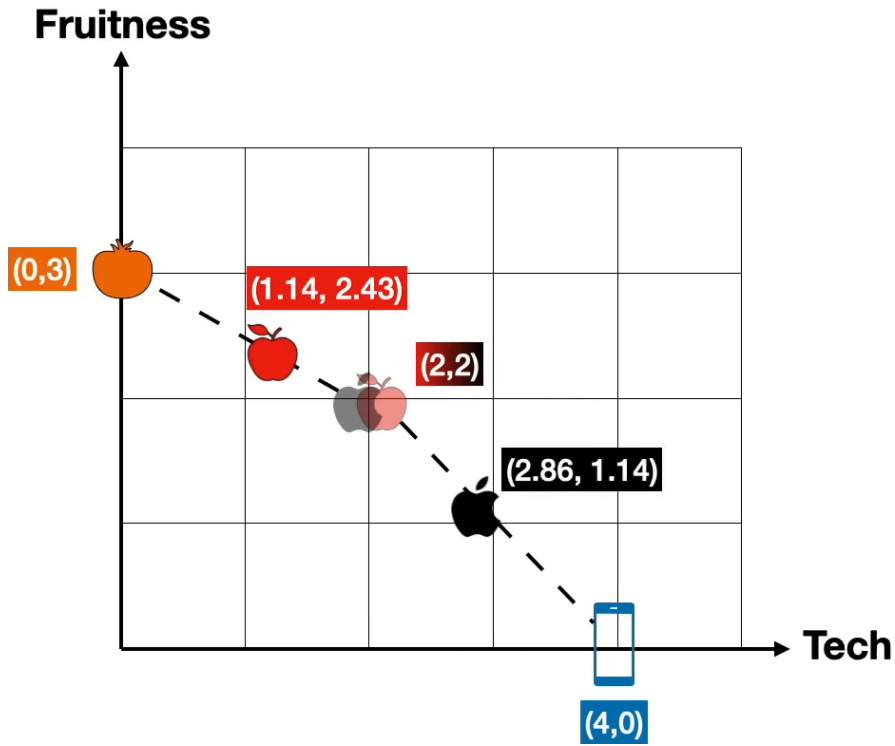
Laptop

Banana

Apple?

GPT: Attention

> *문맥상에서 어떤 의미를 가질까*



an apple and an orange

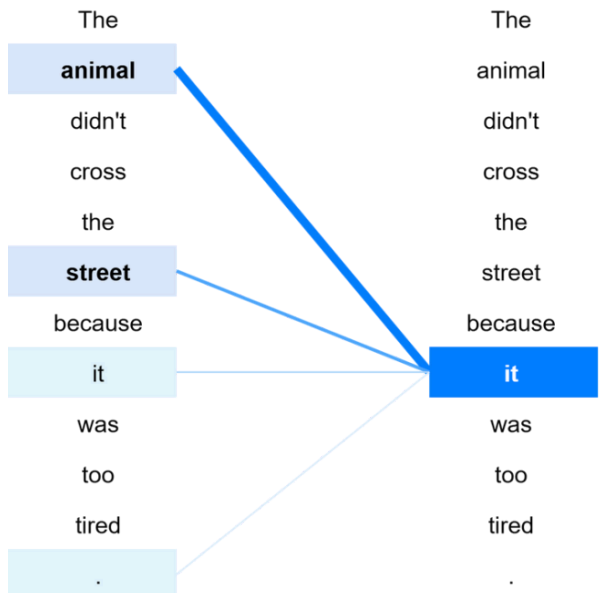
Apple → 0.43 Orange + 0.57 Apple

an apple phone

Apple → 0.43 Phone + 0.57 Apple

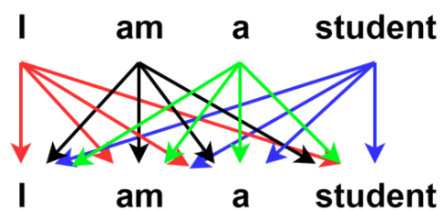
Attention

> *문맥을 파악 – 연관되어 있으면 값이 크게 나타남*



글의 문맥 이해 (Context)

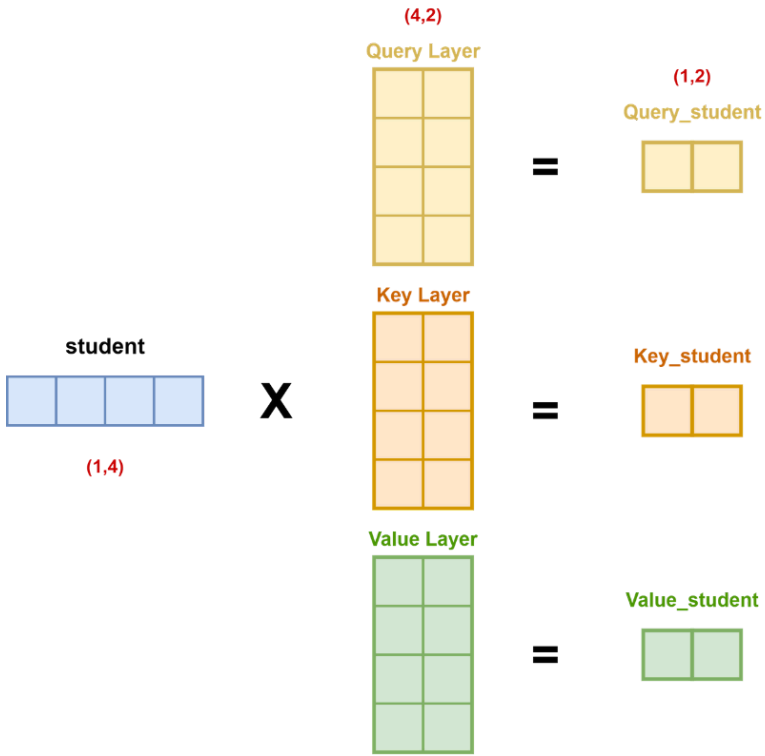
<Attention>



<Self-Attention>

Attention

> *Embedding에서 Weight를 곱해서 Q, K, V를 구함*



- 질문을 던지는 주체(Query)
- 질문에 답하는 대상(Key)
- 전달할 정보(Value)

Attention

> *Q, K를 분리하지 않으면 Attention이 제대로 되지 않음*

- 항상 자기 자신을 우선 주목함
- 순서를 변경해도 동일한 내적이 나옴
- Query는 “내가 무엇을 찾는가”, Key는 “나는 어떤 단서인가”를 표현해야 함
- 같은 벡터를 쓰면 두 역할이 섞여 문맥의 방향성이 약해짐

- `Tom ate the meat` ≠ `The meat ate Tom`
- `Tom @ meat = meat @ Tom`
- 내적은 대칭적이라 누가 누구를 참고하는지 구분하기 어려움
- `Tom` 이 `meat` 를 찾는 경우와 `meat` 가 `Tom` 을 찾는 경우가 같은 점수로 계산됨

Attention

> *K,V를 분리하지 않으면 의미가 섞일 수 있음*

역할 분리

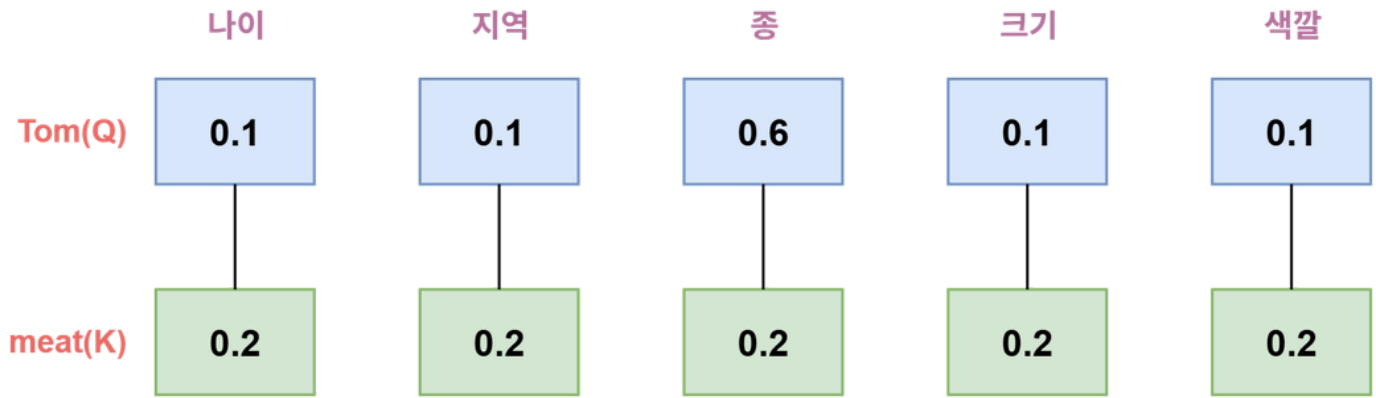
- Key는 “어디를 볼지” 정하는 검색용 단서
- Value는 “무엇을 가져올지” 정하는 실제 정보

분리하지 않으면

- `meat` 를 찾기 위한 단서와 전달할 내용이 같은 벡터가 됨
- 앞 단어들과의 관계 정보가 Value에도 함께 섞임
- 결과적으로 가져와야 할 정보보다 검색 기준의 흔적까지 전달될 수 있음

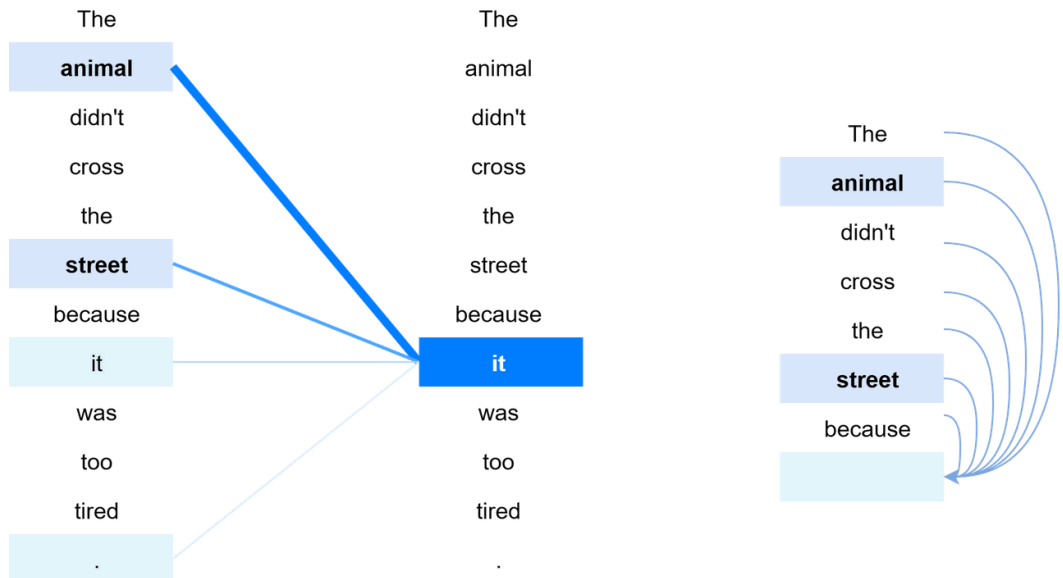
Attention

> *특정 피처의 값을 높여서 해당 피처에 대해 질문한다*



Attention

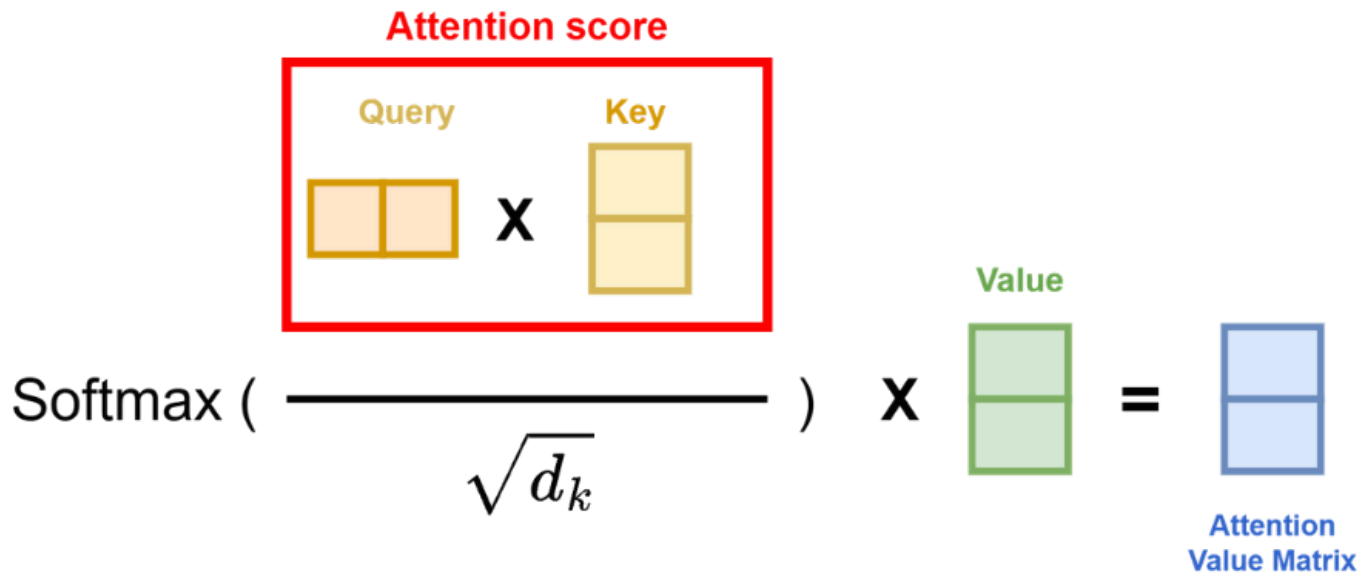
> *다음에 올 단어를 예측하는 것을 학습함으로써 Attention Score가 의미를 가짐*



Attention Score

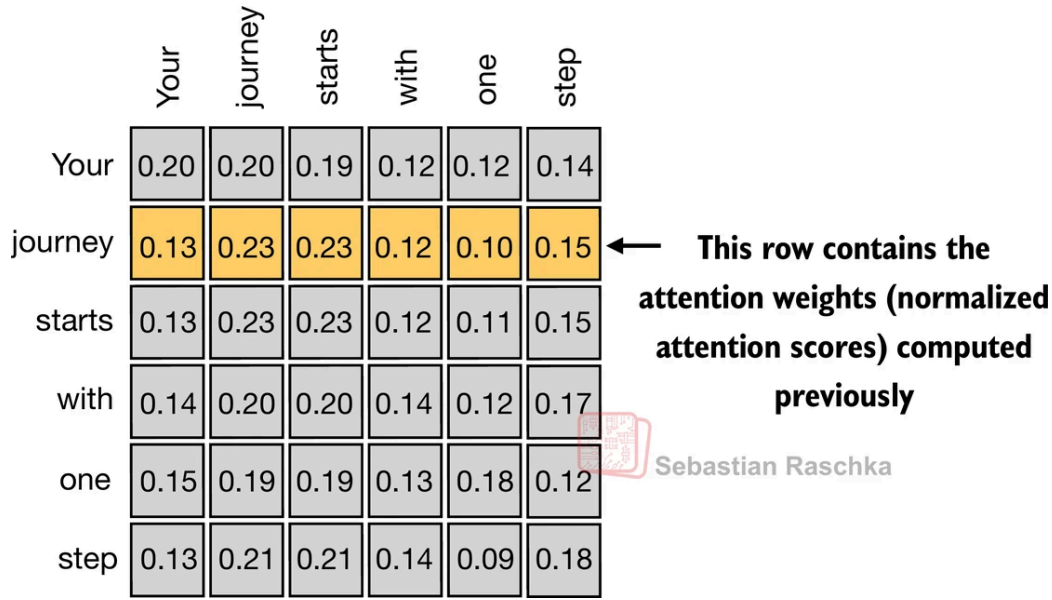
> 연관되어 있으면 값이 크게 나타남

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$



Attention Map

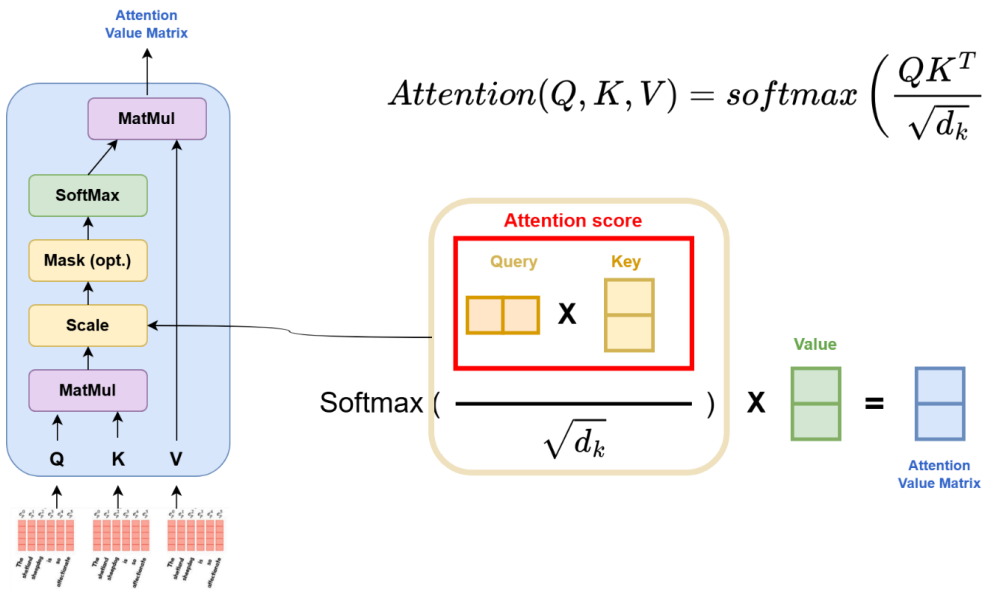
> *Attention Score를 시각화한 행렬 — 어떤 단어와 어떤 단어가 연관되어 있는지 시각화*



Scale

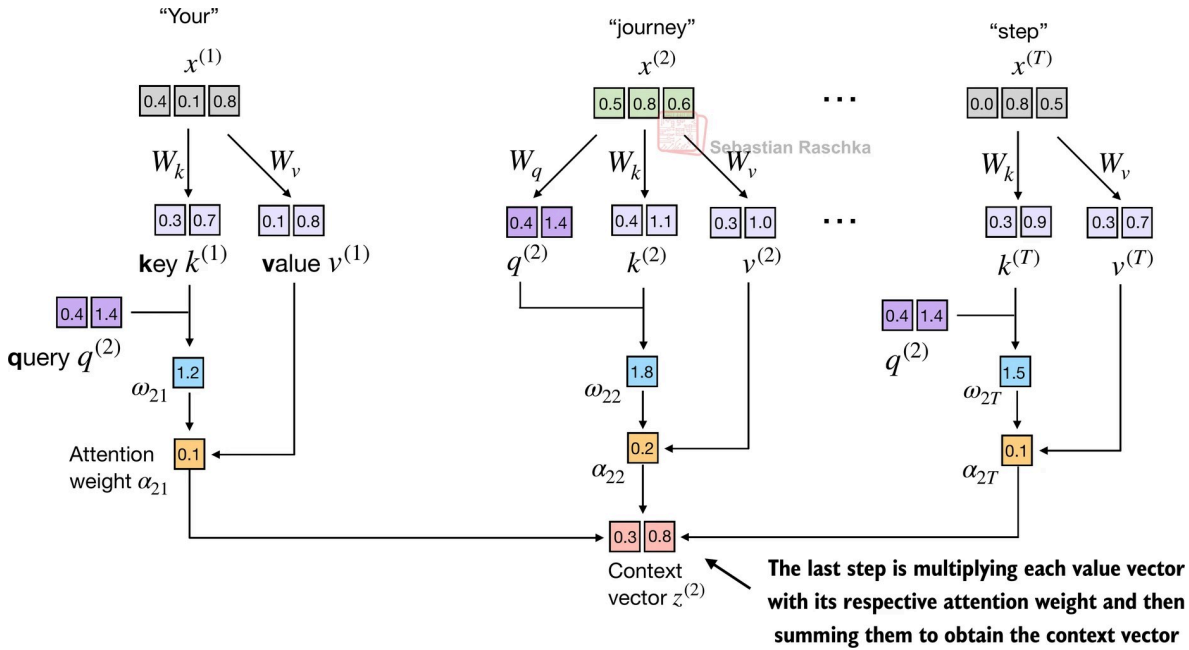
> *dk로 나누어 분산을 다시 1로 맞춤*

- Attention 연산을 하면 차원 수(dk)가 커질수록 결과가ㅅ(Dot-product)이 매우 커지는 경향이 있음
- 내적값의 표준편차 = $\sqrt{d_k}$ (차원 수의 제곱근)



$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

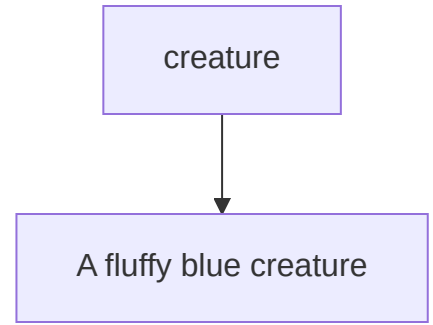
전체 흐름



$$
\begin{aligned}
z^{(2)} &= \left(\alpha_{21} \times v^{(1)}\right) + \left(\alpha_{22} \times v^{(2)}\right) + \cdots + \left(\alpha_{2T} \times v^{(T)}\right) \\
&= \left(0.1 \times \begin{bmatrix} 0.1 & 0.8 \end{bmatrix}\right) + \left(0.2 \times \begin{bmatrix} 0.3 & 1.0 \end{bmatrix}\right) + \cdots + \left(0.1 \times \begin{bmatrix} 0.3 & 0.7 \end{bmatrix}\right)
\end{aligned}
$$

Attention 의미

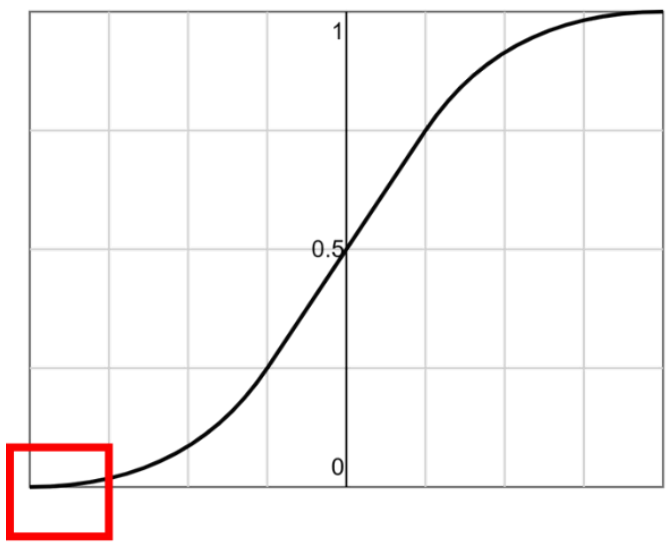
> *문맥의 의미를 더해서 새로운 문맥의 의미를 만듦*

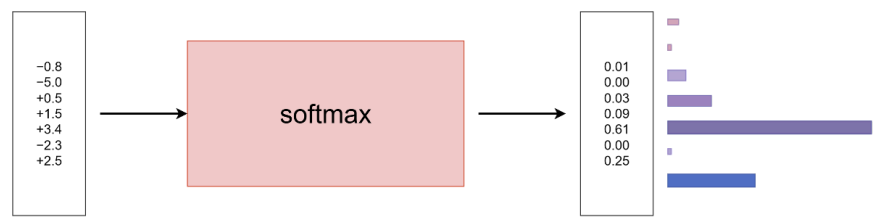


Softmax

> *어떤 숫자의 나열을 확률분포로 변환 – 지수 함수를 사용*

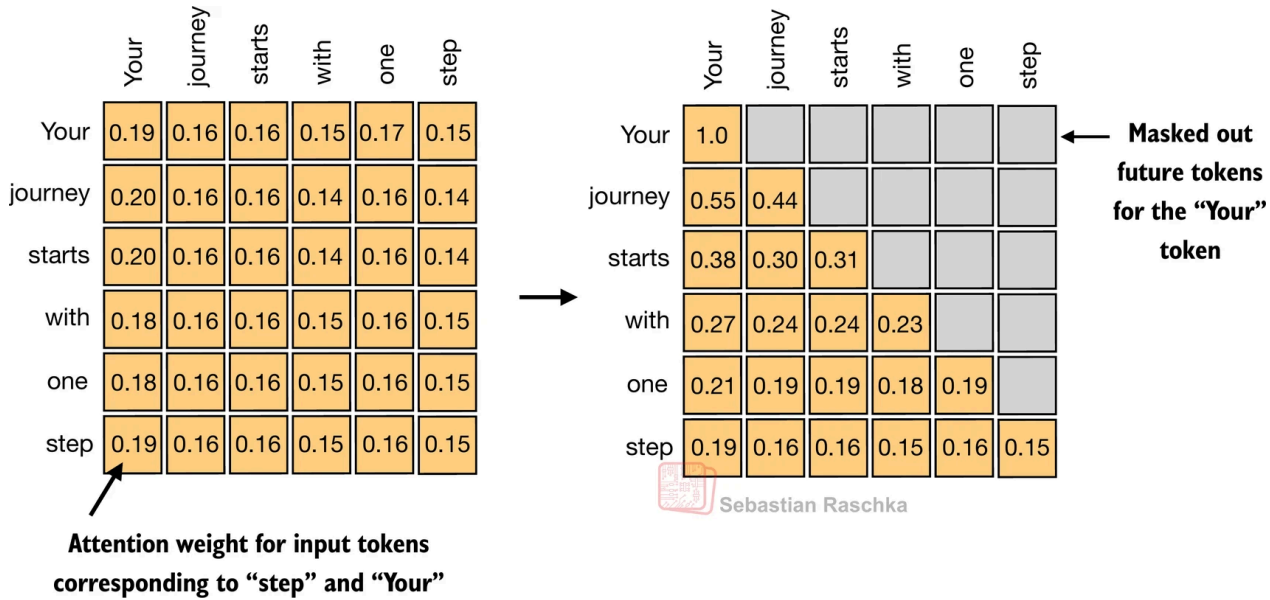
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$





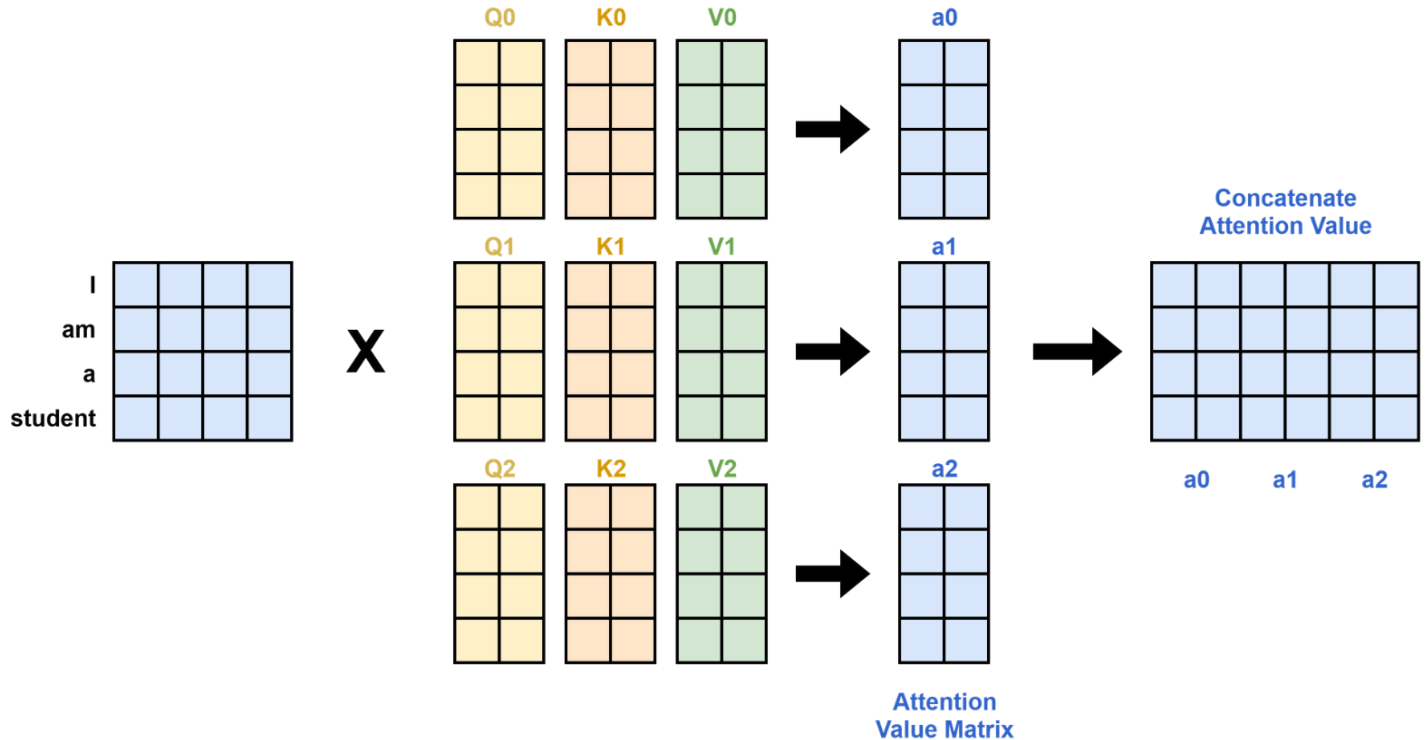
Masking

> *현재 토큰이 미래 토큰을 보지 못하도록 Attention Score를 가림*



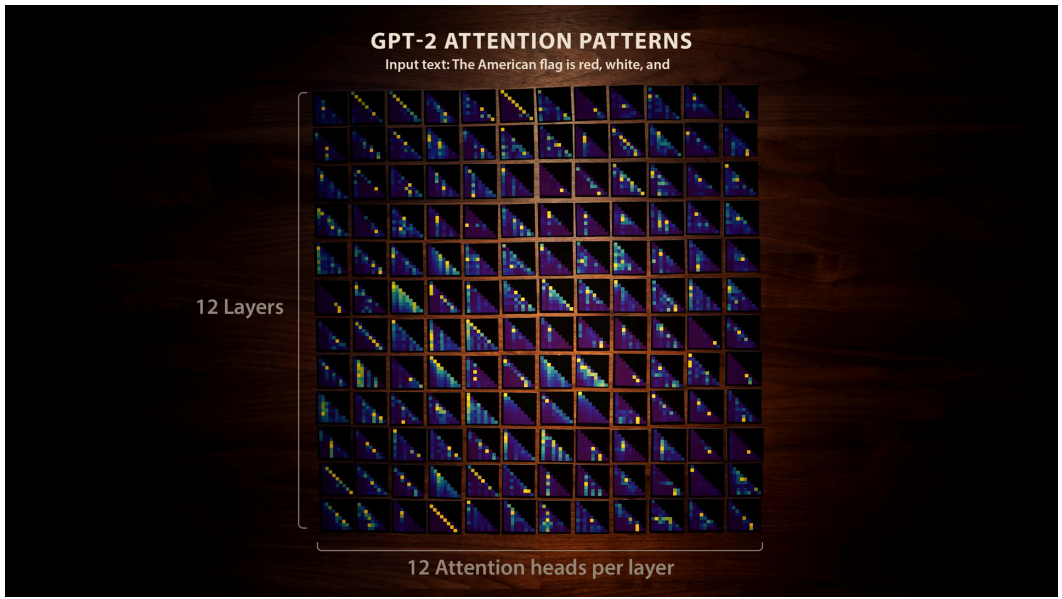
Multi-head Attention

> *각각의 Head는 다른 관점에서 문맥을 파악함*



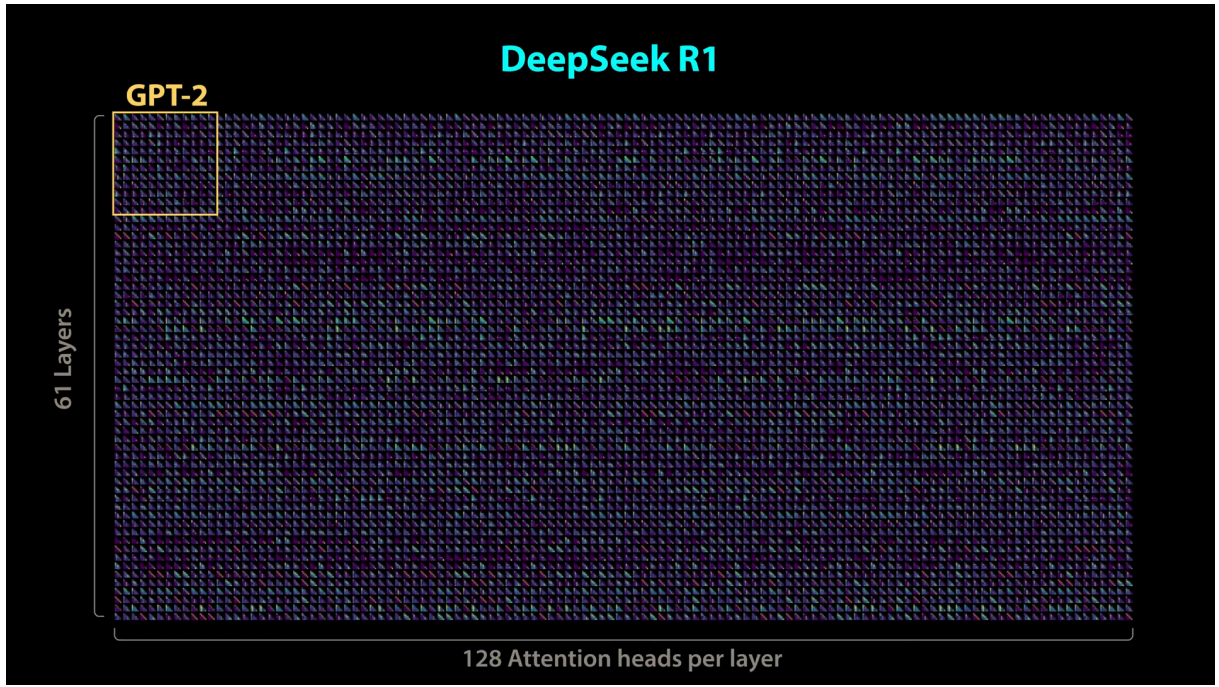
Multi-head Attention

> *GPT2 Small 모델은 12개의 헤더와 12개의 Transformer 블럭을 가짐*



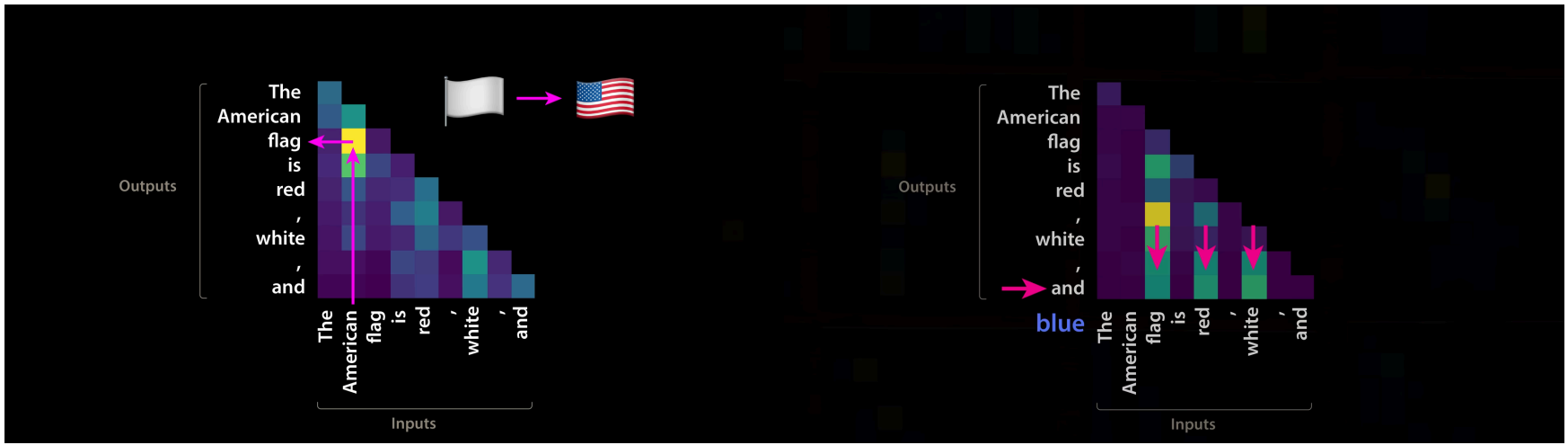
Multi-head Attention

> *DeepSeek R1의 경우 61x128개의 Head를 가지고 있음*



Multi-head Attention

> *초기 Layer는 문맥 파악, 후반 Layer는 다음 Token 예측에 주력*



초기 Layer: American → flag에 주목 (문맥 파악)

후반 Layer: and가 flag, red, white를 주목 → **blue** 추론

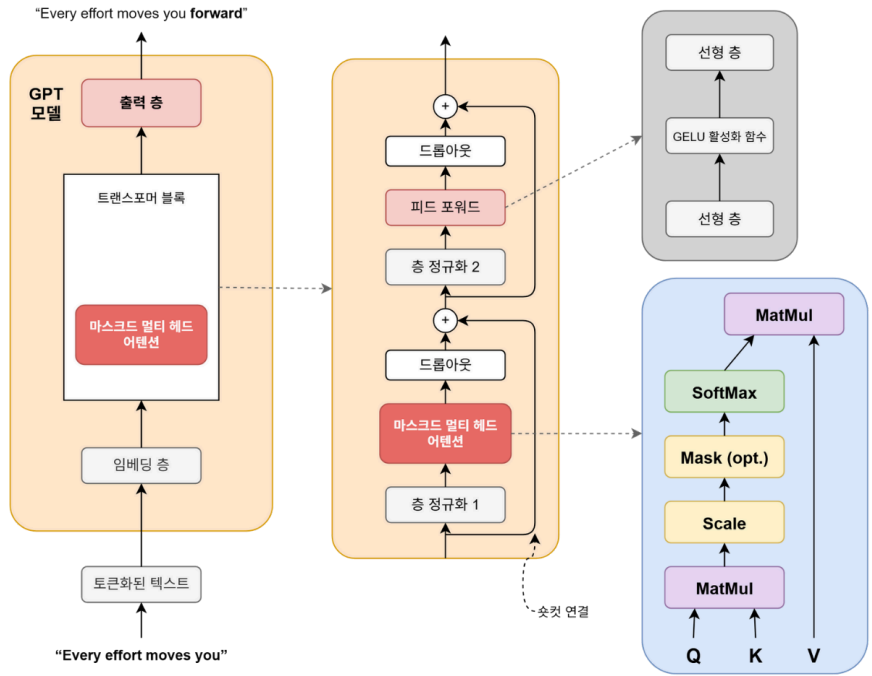
실습

- Chapter_3_Excercise_Attention.ipynb

GPT Layer들

GPT: Layer들

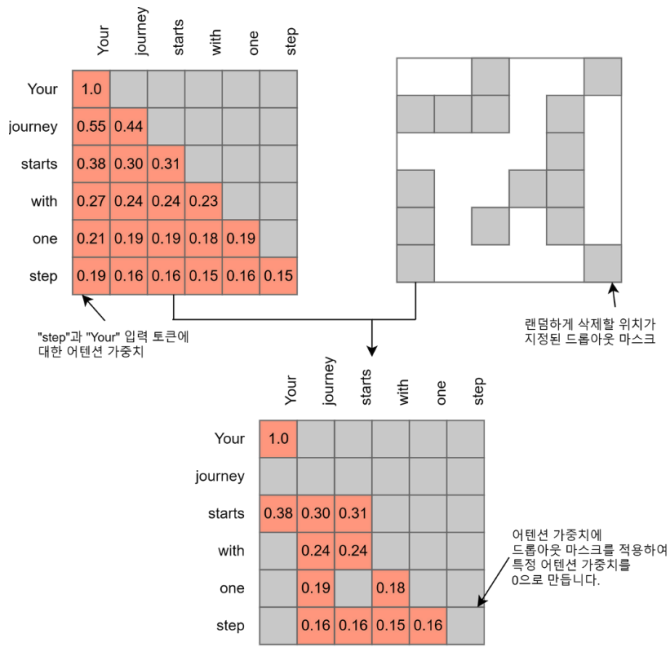
Decoder만 사용



Dropout

Overfitting 방지

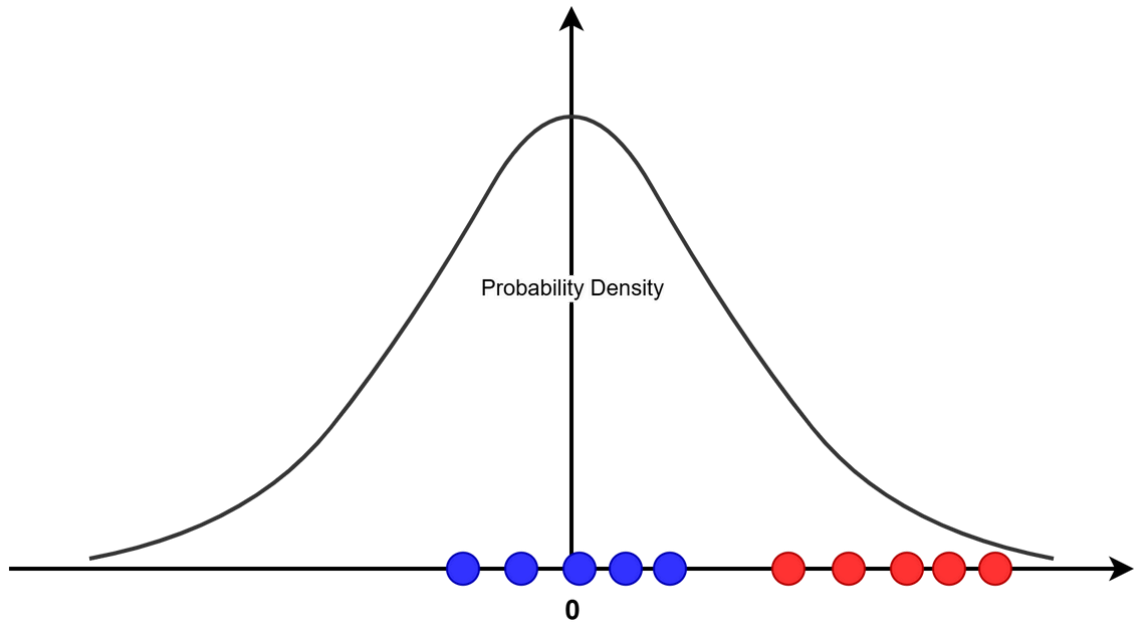
- Softmax 이후에 랜덤으로 0으로 마스킹함, 이번에 학습할 때는 여기만 집중해
- 학습 속도가 느리고, 데이터가 많아서 학습에는 잘 사용하지 않음



정규화

> *평균이 0이고 표준편차가 1이 되도록 변환*

- 학습을 안정화 시킴
- 과도한 정규화는 학습 내용을 사라지게 할 수 있음



정규화: Layer Normalization

> *각 토큰의 표현 스케일을 일정하게 맞춰 학습을 안정화함*

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_B)^2$$

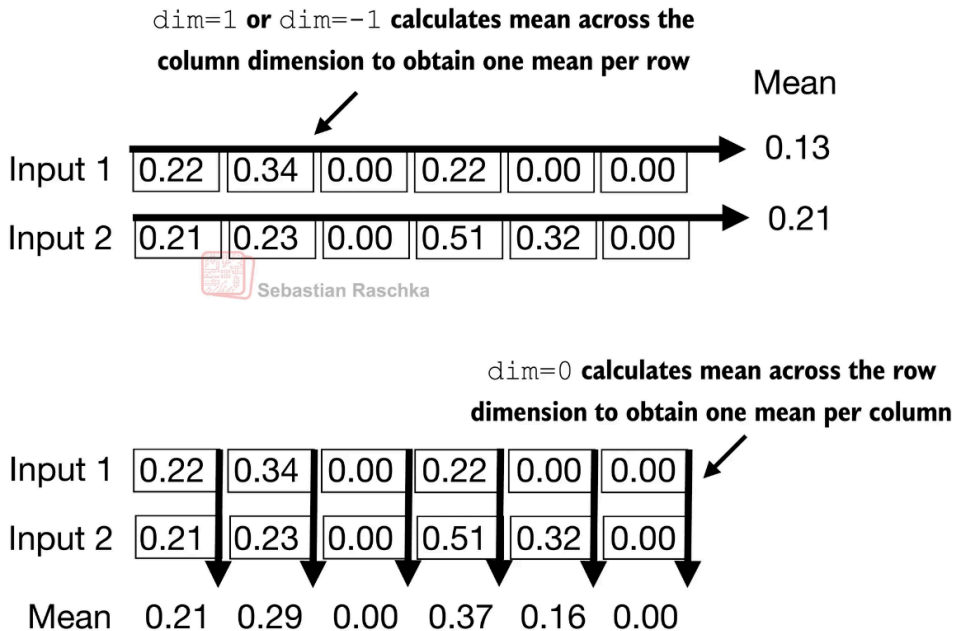
$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

- 입력값에서 평균을 빼고 분산을 나눔, ϵ은 0으로 나누는 것 방지
- 값의 스케일이 지나치게 커지거나 작아져 Gradient가 불안정해지는 현상을 완화
- 마지막에지γ, β를 적용해 모델이 필요한 스케일과 위치를 다시 학습

정규화: Normalization

> *행렬에서 Layer, Batch Normalization*



정규화: RMS Normalization

> *비용이 많이 드는 평균을 제외*

- 정규화의 핵심 효과는 평균을 0으로 맞추는 것보다 데이터의 크기(Scale)를 일정하게 유지하는 것에서 나옴
- Layer Normalization의 $(x_i - \mu_B)$에서 평균 μ_B를 빼는 과정을 제외

$$\text{RMS}(x) = \sqrt{\frac{1}{m}\sum_{i=1}^{m} x_i^2 + \epsilon}$$

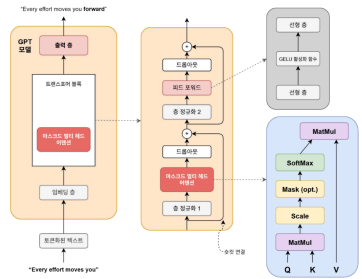
$$\hat{x}_i = \frac{x_i}{\text{RMS}(x)}$$

$$y_i = \gamma \hat{x}_i$$

Feed Forward Network

> *지식의 저장소*

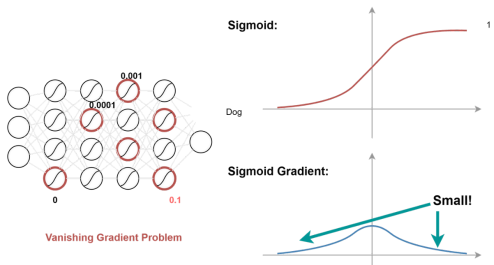
- 수집된 정보를 해석하고 처리하여 저장된 지식과 연결



활성화 함수: Sigmoid

> *복잡한 패턴 학습 가능*

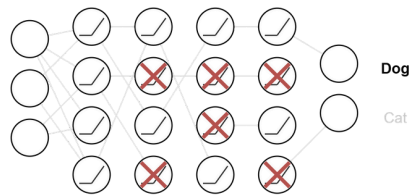
- Layer가 많아질수록 Vanishing Gradient Problem 발생



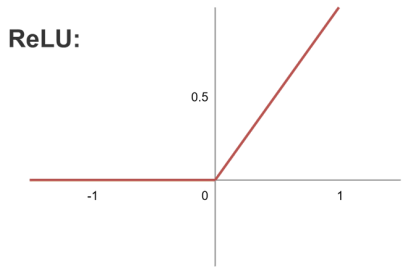
활성화 함수: ReLU

> *Layer을 깊게 쌓을 수 있게 됨*

- 비활성화되는 뉴런이 많아짐



Dying ReLU Problem

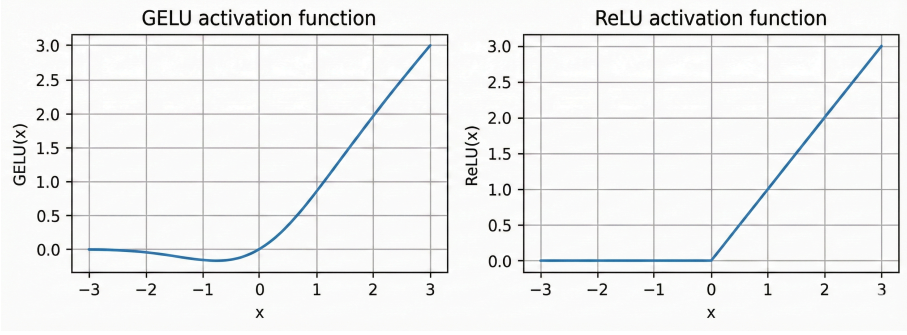


ReLU(Wx + b) = 0
if max(Wx + b, 0) = 0
or Wx + b < 0 or b < -Wx

This happens with very negative bias when W,x > 0

GELU

> *좀 더 안정화된 학습이 가능*



$$\text{GELU}(x) \approx 0.5x\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}\left(x+0.044715x^3\right)\right)\right)$$

$$\text{ReLU}(x) = \max(0, x)$$

Swish: 부드러운 Self-Gating

$$\text{Swish}_\beta(x) = x\sigma(\beta x)$$

β는 게이트의 민감도를 조절하며, $\beta = 1$로 고정하여 사용하는 경우가 많음

- 양수 영역은 위로 제한 없이 증가
- 음수 정보도 일부 유지해 뉴런이 완전히 꺼지는 현상을 완화할 수 있음
- 매끄럽게 미분 가능하며, 비단조(non-monotonic)·셀프 게이팅(self-gating) 특성

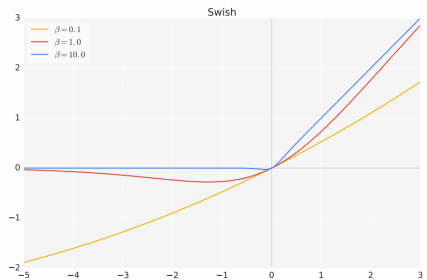


Figure 4: The Swish activation function.

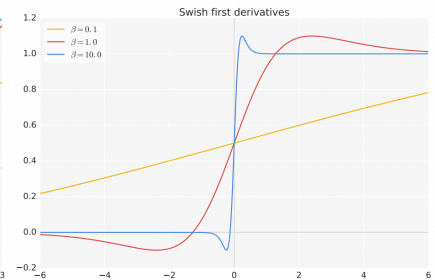


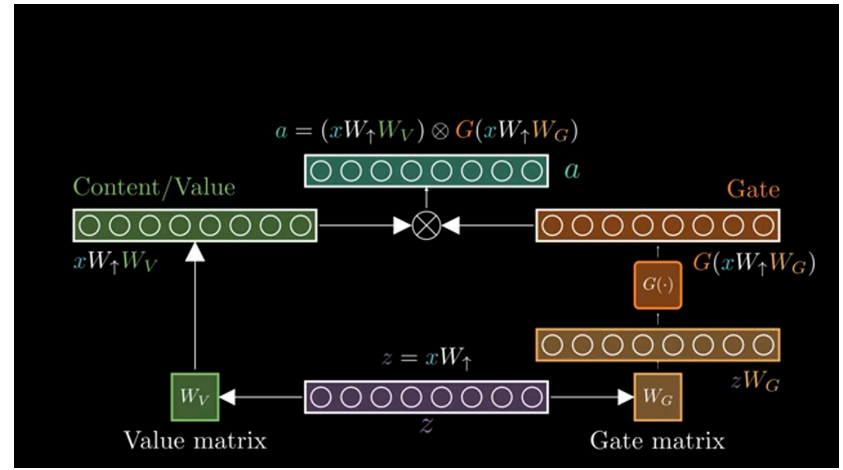
Figure 5: First derivatives of Swish.

아래 Swish 행: 입력 z는 콘텐츠, 시그모이드(sigmoid)는 게이트

GLU: Gate × Value

$$\mathrm{GLU}(x, W, V, b, c) = \sigma(xW + b) \odot (xV + c)$$

- 입력에서 게이트(gate)와 값 경로(value path)를 별도 선형 투영으로 학습
- 시그모이드(sigmoid) 게이트가 특징별 통과량을 결정
- ⊙는 같은 위치의 원소끼리 곱하는 연산



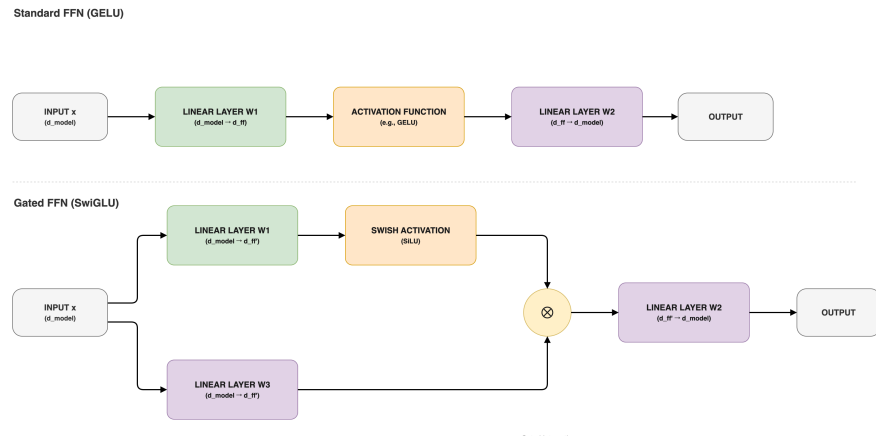
Gate × Value: 학습된 게이트가 각 특징을 선택적으로 통과시킨다

SwiGLU: Swish + GLU

$$\text{SwiGLU}(x) = \text{Swish}_\beta(xW + b) \odot (xV + c)$$

- 부드러운 Swish 게이트와 학습된 값 경로(value path)를 결합
- 현대 LLM의 FFN에서 널리 채택된 선택지
- `F.silu` 는 $\beta = 1$인 Swish

```
gate, value = up_proj(x).chunk(2, dim=-1)
hidden = F.silu(gate) * value
output = down_proj(hidden)
```

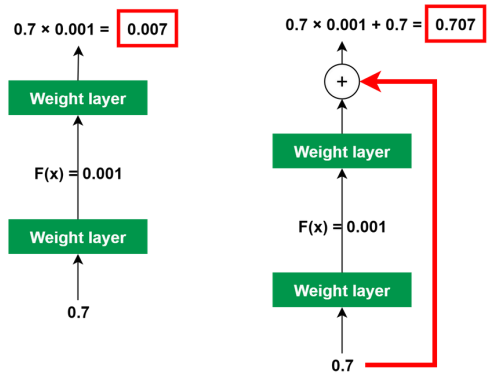


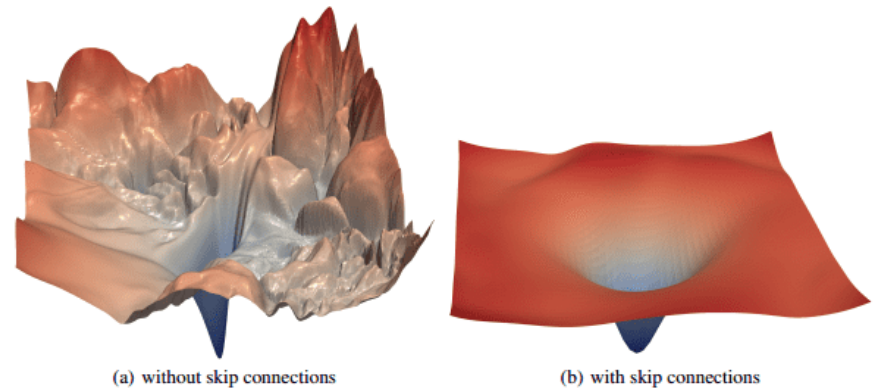
표준 FFN과 일반적인 게이트 FFN 비교

Residual connection

안정된 학습

- 엄청 깊다면 한개의 Layer에서 값의 변화가 크지 않을 것이다.
- 변화량만 학습하면 되니까 학습이 쉽다

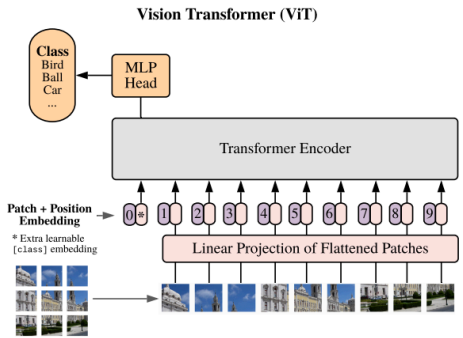




(a) without skip connections (b) with skip connections

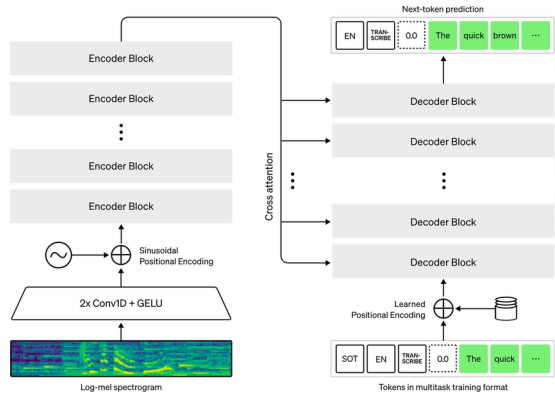
Vision Transformer

> *이미지 인식에 Transformer Encoder 사용*



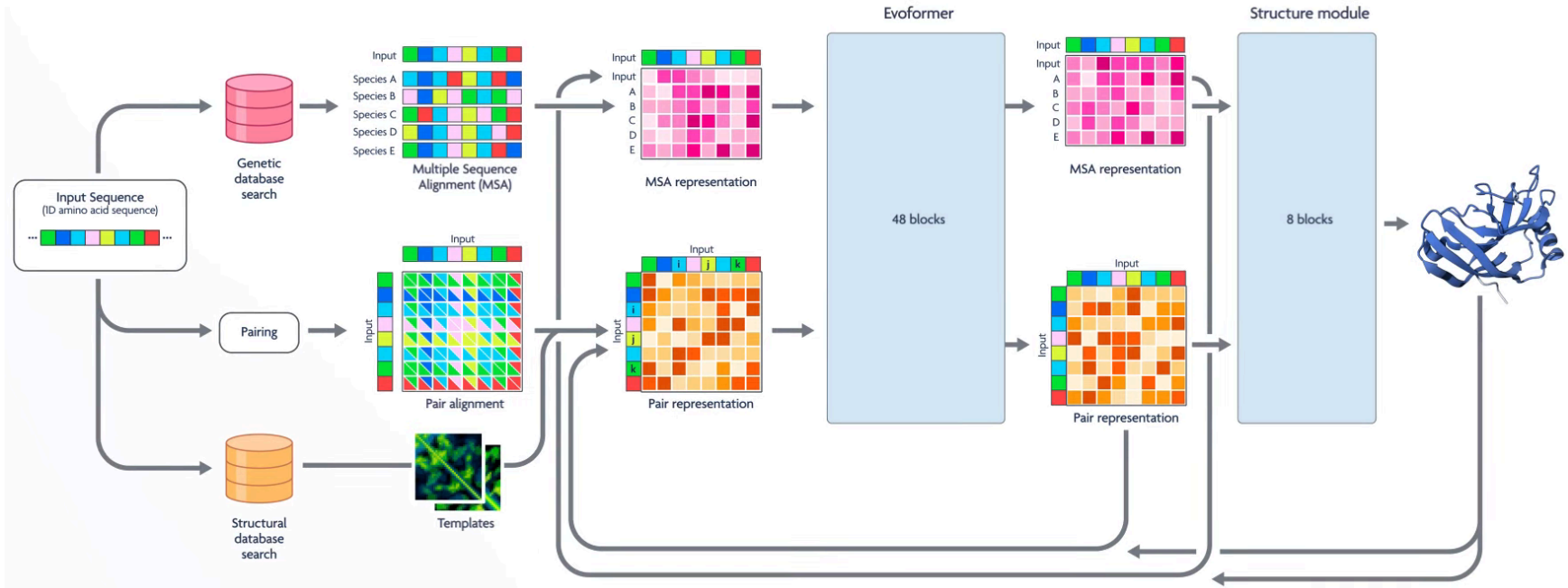
Whisper

> *음성 인식의 Encoder와 Decoder에 사용*



AlphaFold

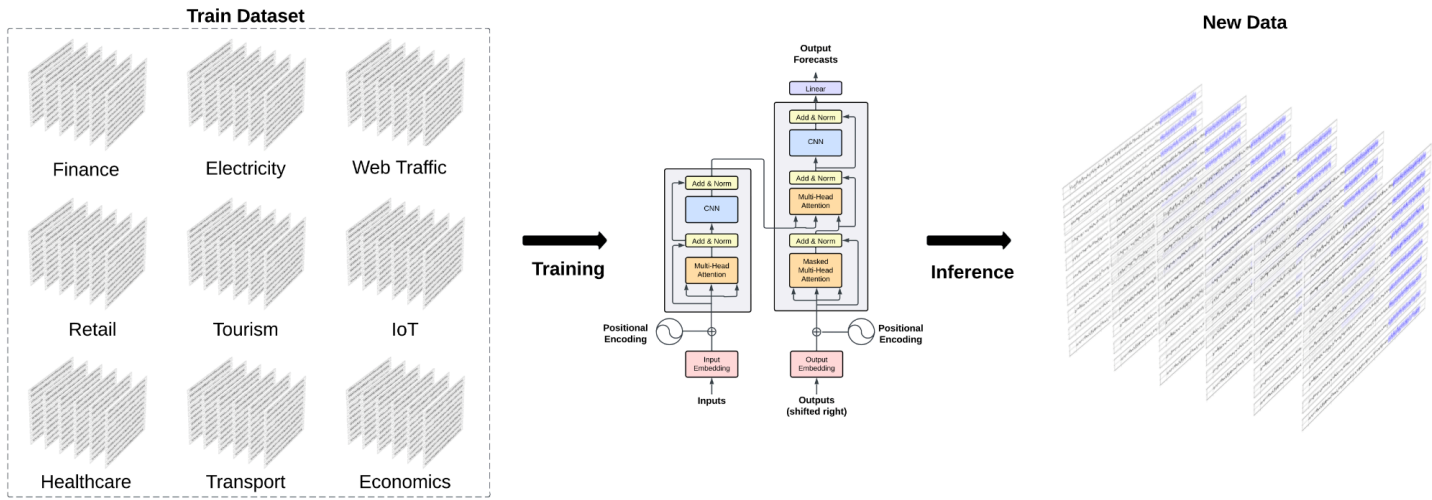
> *아미노산과 아미노산의 관계를 파악하는데 Transformer 사용*



Time Series Prediction

> *Token: 연속적인 실수 형태의 시계열 데이터를 미리 정해둔 유한한 개수의 구간으로 나눔*

- 시계열 데이터를 LLM 모델에 적용



GPT 모델 입출력

GPT 모델 입출력

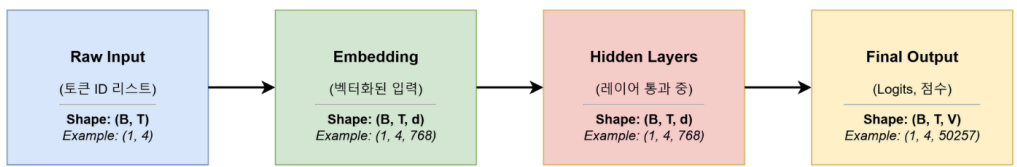
> *Tramsformers 모듈의 입출력의 차원이 동일*

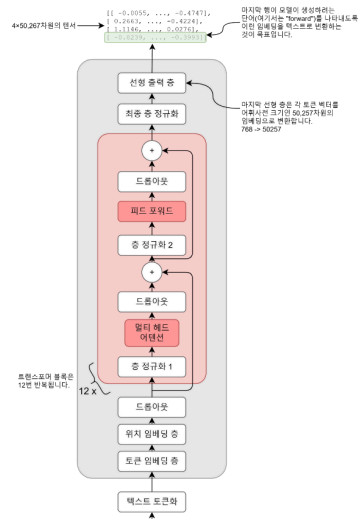
- 모듈을 계속 쌓기 위함
- Residual connection 가능하도록 하기 위함

GPT 모델 입출력

> *Final Output에서 임베딩 차원이 토큰수만큼 변경됨*

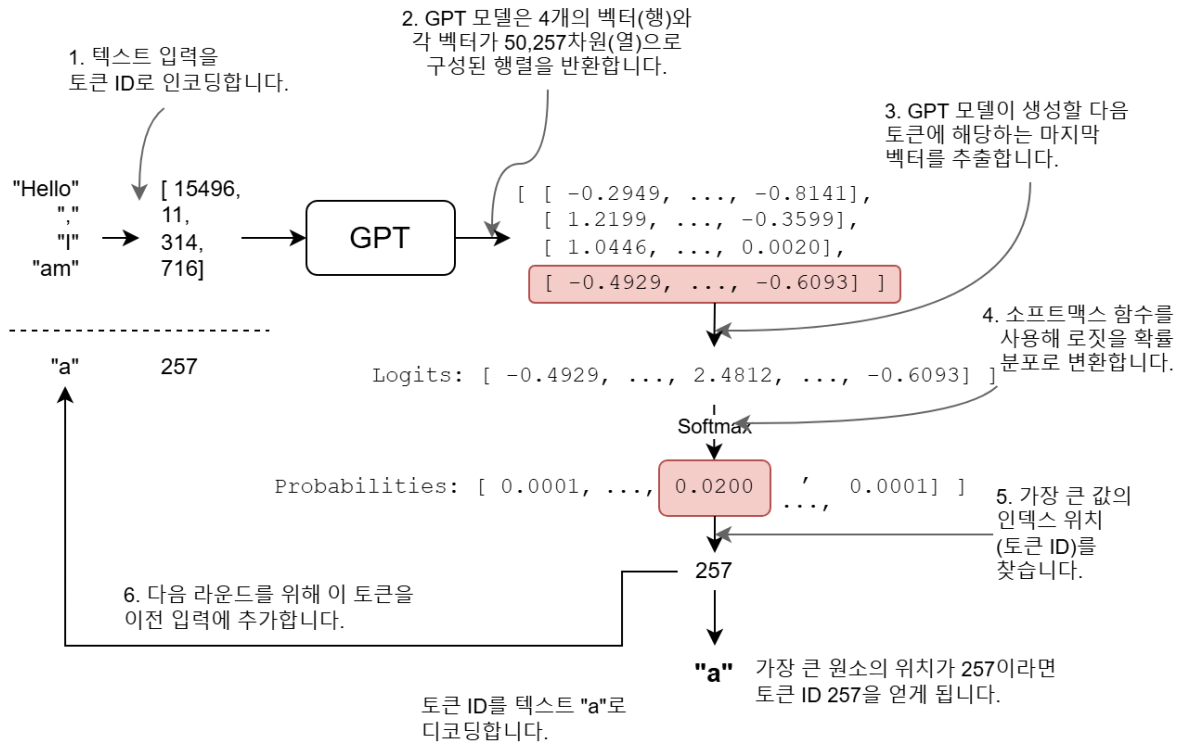
- B(배치): 문장 1개 입력, T(길이): Token 4개, d (벡터 크기): 768, V (사전 크기): 50,257





Output Token 생성 과정

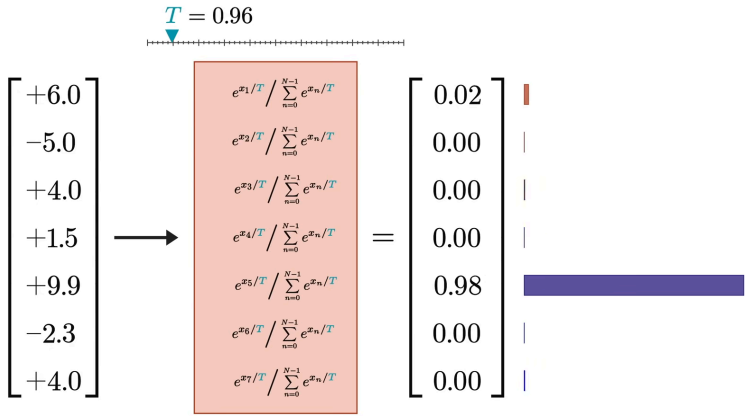
> *마지막 임베딩에서 가장 높은 값의 위치가 다음 Token ID가 됨*

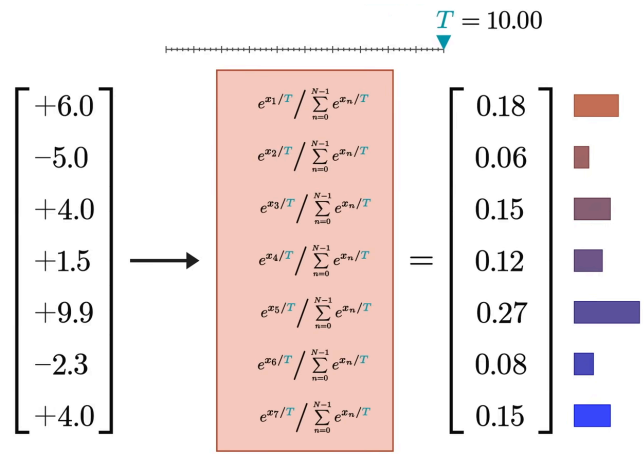


Temperature(T)

> *T가 크면 더 작은 가ㅅ들이 더 많은 비중을 차지하게 됨*

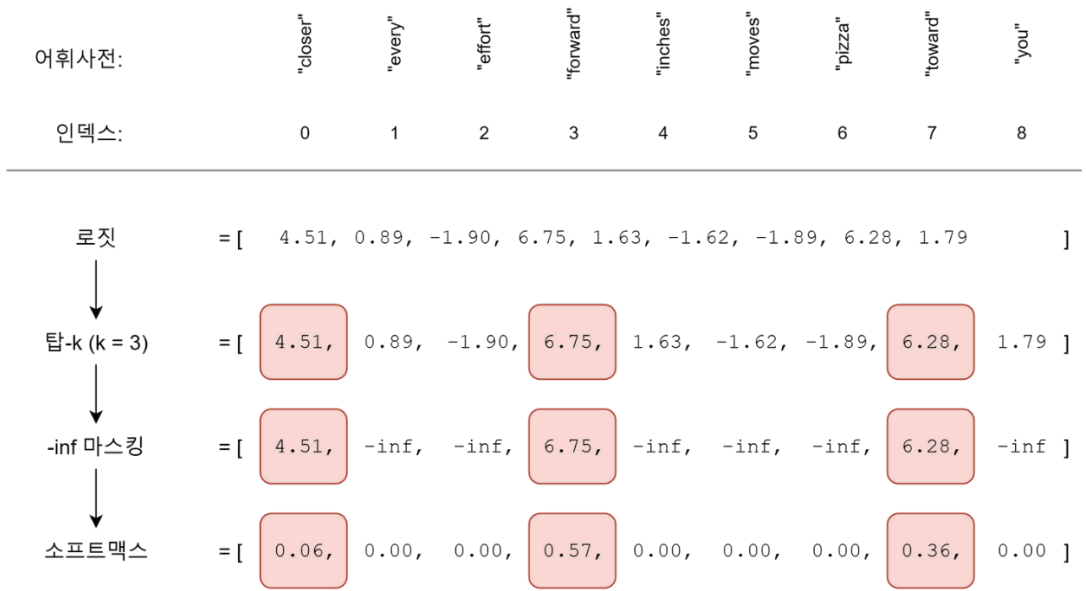
- Softmax에서 지수를 T로 나눔





Top-K 샘플링

> *샘플을 K만큼 고른 후 확률 기반으로 랜덤으로 선택*

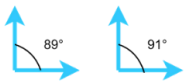


지식의 저장

FFN 지식의 저장

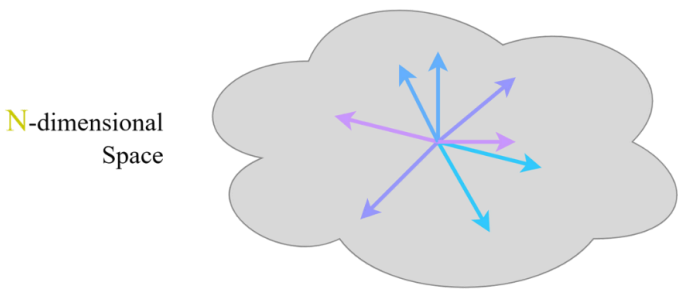
> *직교하는 벡터가 많아야 한다*

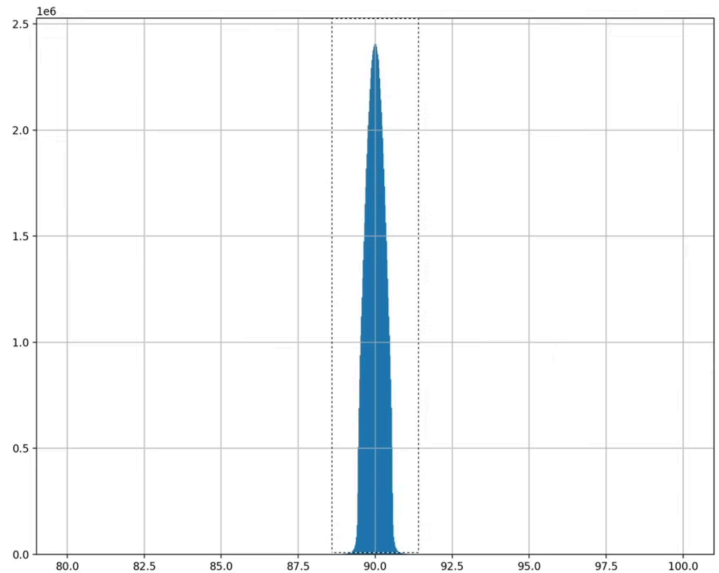
- N차원에서 서로 직교하는 벡터는 N개
- 89-91도까지 허용하면 고차원일수록 서로 직교하는 백터가 대다수임



Choose multiple vectors,
each pair between 89° and 91° apart

Johnson–Lindenstrauss Lemma ⇨ Maximum # of vectors: ≈ exp(ε · N)



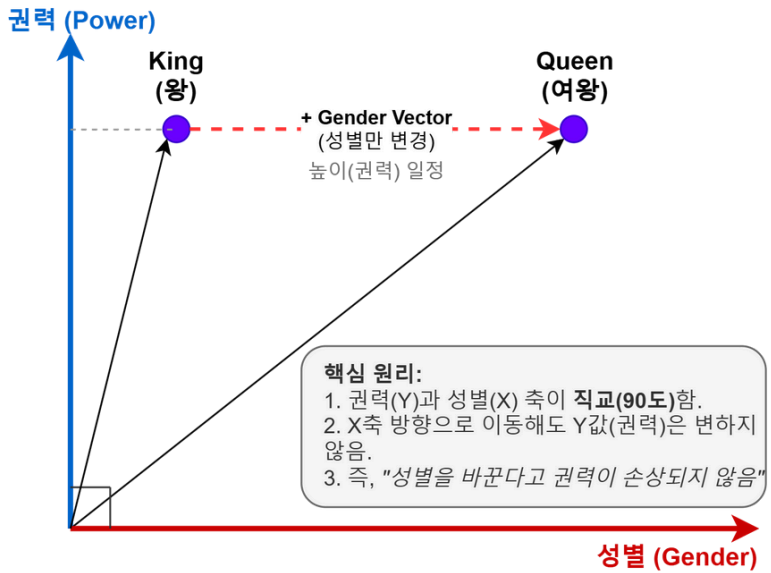


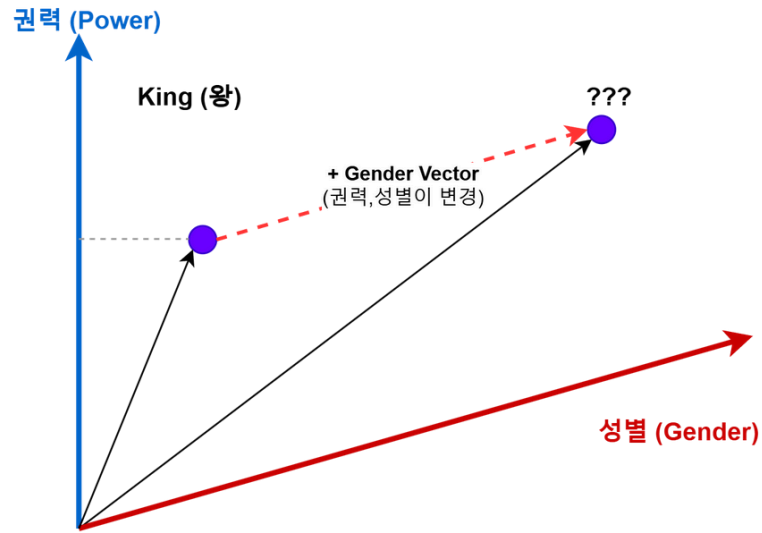
100만개 벡터를 서로 직교하는지 검사

FFN 지식의 저장

> *직교성(Orthogonality)과 특징 분리(Disentanglement)*

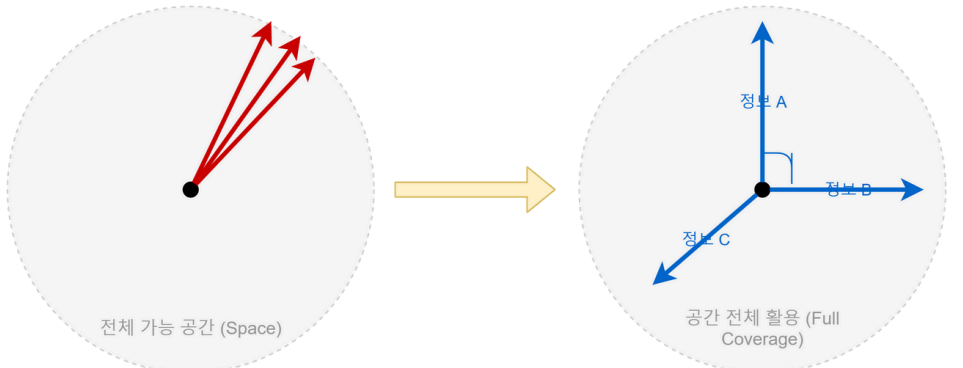
- 개념들이 서로 간섭하지 않도록 함





FFN 지식의 저장

정보 저장의 효율성



평행한 벡터 (Parallel)

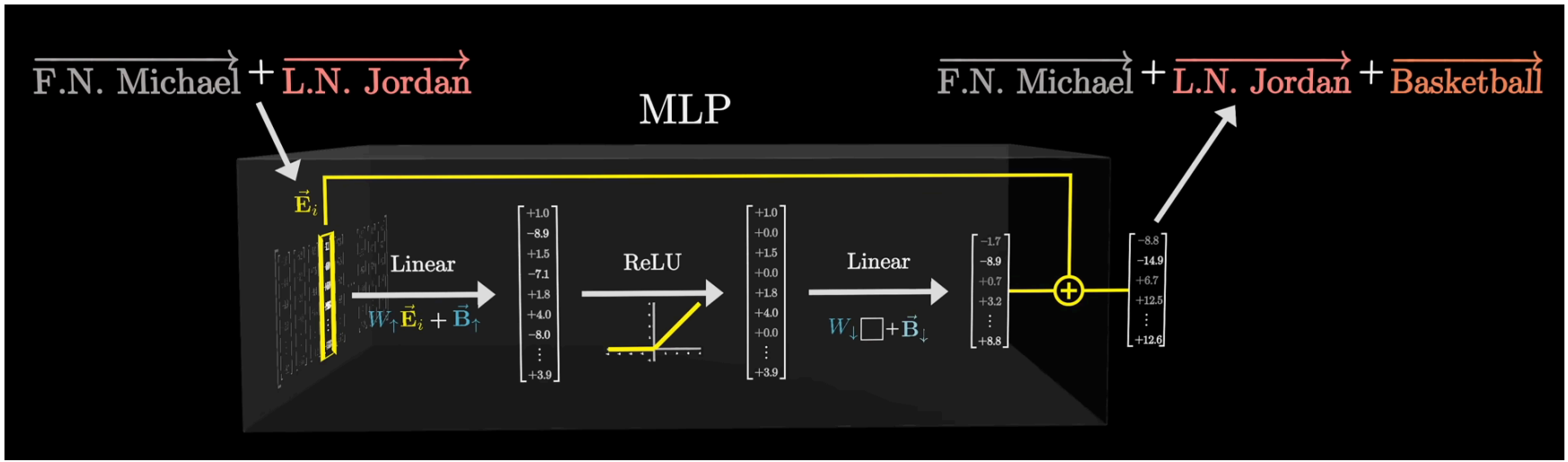
❌ **정보의 중복**
- 모두 같은 이야기만 함
- 나머지 공간은 텅 빔 (낭비)
- 유사도가 너무 높아 구분이 안 됨

직교하는 벡터 (Orthogonal)

✅ **표현력 극대화**
- 서로 완전히 다른 정보를 표현
- 공간을 골고루 사용하여 꽉 채움
- 내적값 0 = 간섭 없음 (독립적)

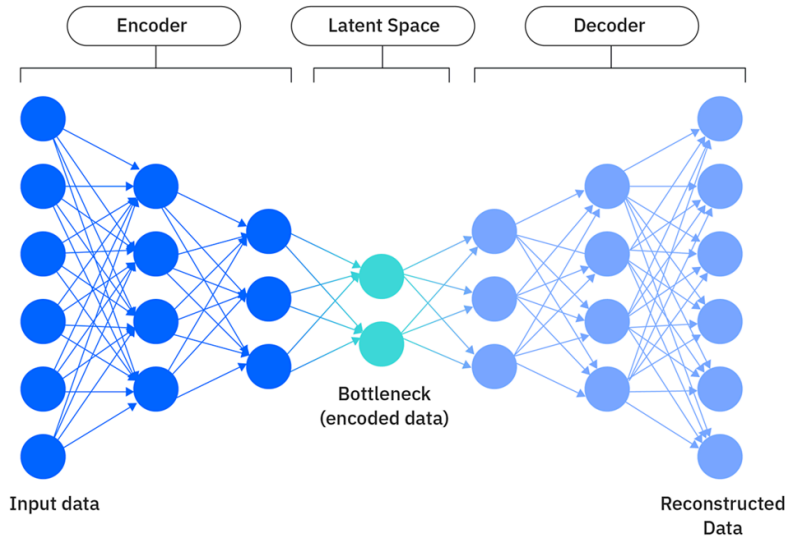
FFN 지식의 저장

> *수많은 Layer들이 정보를 저장하고 있으며, 다음 Layer로 가면서 새로운 정보를 만듦*



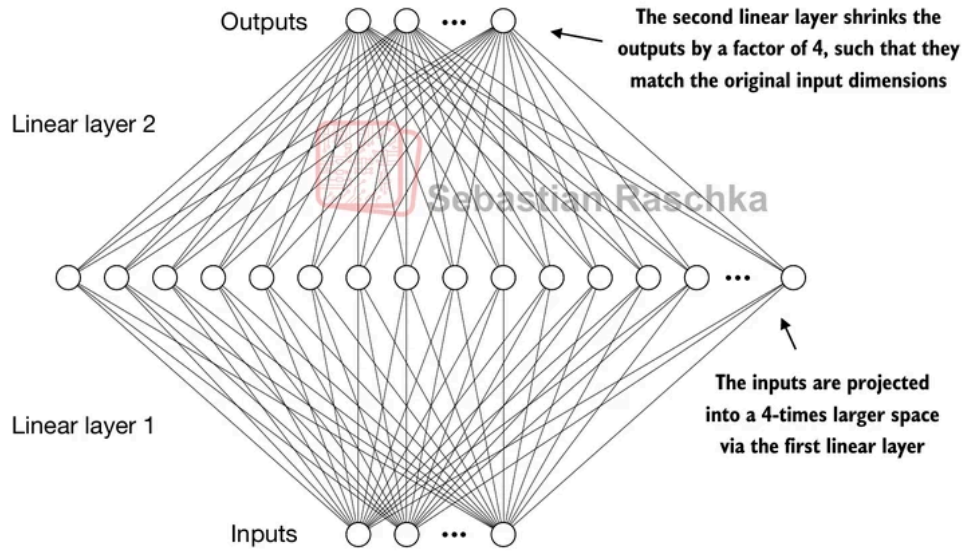
Latent Space

> *차원이 축소되어 정보가 압축되어서 저장되는 공간*



Expansion Layer

> *차원을 확대하여 특징을 세분화하여 저장, 활성화 함수에 의한 정보 손실 방지*



실습

- Chapter_4_Excercise_GPT.ipynb

기울기 누적과 가중치 갱신

> *배치마다 기울기를 계산하고, 일정 배치마다 가중치를 바꿉니다.*

`minimind/trainer/train_pretrain.py` · 38~47행

backward(): 기울기 계산과 누적

```
scaler.scale(loss).backward()
```

`backward()` 는 손실에 대한 **각 매개변수의 기울기**를 계산합니다.

- 기울기는 매개변수의 `.grad` 에 누적됩니다.
- 이 단계에서는 아직 가중치를 갱신하지 않습니다.
- 옵티마이저가 기울기를 이용해 실제 가중치를 조정합니다.

기울기 계산과 가중치 갱신의 시점을 구분하세요.

GradScaler: 작은 기울기를 보호하기

```
scaler.scale(loss).backward()
```

scaler 상태	동작
활성화	손실을 확대하여 작은 기울기가 사라지는 문제를 줄임
비활성화	손실 확대 없이 일반 역전파처럼 동작

갱신 전에는 `unscale_()` **로 원래 기울기 크기를 복원**합니다.

배치 4개를 모아 한 번 갱신

> *예시:* `accumulation_steps = 4`

배치	기울기	가중치
1	계산 후 누적	유지
2	기존 기울기에 누적	유지
3	기존 기울기에 누적	유지
4	기존 기울기에 누적	**갱신 후 기울기 초기화**

배치 크기 **8** × 누적 **4회** × GPU **1장** = **32개 샘플 / 갱신**

가중치를 갱신하는 조건

```
if step % args.accumulation_steps == 0 or step == iters:
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)

    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
```

- 누적 횟수에 도달하면 갱신합니다.
- 에포크의 마지막 배치에서도 남은 기울기를 처리합니다.

복원한 뒤 기울기의 크기 제한

```
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
```

순서	역할
1. `unscale_`	확대했던 기울기를 원래 크기로 복원
2. `clip_grad_norm_`	전체 기울기 norm을 제한해 과도한 크기를 완화

원래 크기로 복원해야 지정한 임계값에 맞춰 클리핑할 수 있습니다.

갱신 후 다음 누적 구간 준비

```
scaler.step(optimizer)
scaler.update()
optimizer.zero_grad(set_to_none=True)
```

호출	역할
`scaler.step`	유효한 기울기라면 옵티마이저로 가중치 갱신
`scaler.update`	이후 사용할 스케일 배율 조정
`zero_grad`	다음 구간을 위해 누적된 기울기 초기화

활성화된 scaler는 비정상 기울기를 감지하면 갱신을 건너뛸 수 있습니다.

학습 흐름 확인

```
배치마다:  손실 계산 → backward() → .grad에 누적

누적 완료: unscale → clip → step → update → zero_grad
```

- `backward()` 는 **기울기 계산·누적**
- `step()` 은 **가중치 갱신**
- `zero_grad()` 는 **다음 누적 구간 준비**

torch.compile: 모델 연산 최적화

```
model = torch.compile(model)
```

모델의 연산을 분석하고 최적화해서 실행 속도를 높입니다.

- 오른쪽은 기존 모델을 감싸는 최적화된 모델을 반환합니다.
- 왼쪽은 이후 `model(...)` 호출이 이를 사용하도록 다시 대입합니다.
- 모델 구조와 학습할 파라미터는 유지됩니다.

연산 흐름을 그래프로 포착하고, 연산 결합과 Python 실행 부담·GPU 메모리 접근 감소를 시도합니다.

계산 예제: 더한 뒤 곱하기

```
def forward(x):
    a = x + 2
    b = a * 3
    return b
```

입력 `x = [1, 2, 3]`에 대해 원소별로 계산합니다.

```
a = [1, 2, 3] + 2 = [3, 4, 5]
b = [3, 4, 5] × 3 = [9, 12, 15]
```

최적화 전후 모두 같은 수학적 계산을 수행합니다.

GPU 작업을 하나로 합치는 예

> *일반 실행: 덧셈과 곱셈을 별도 GPU 작업으로 처리*

```
① x 읽기 → 2 더하기 → 중간 결과 a를 GPU 메모리에 저장
② a 읽기 → 3 곱하기 → 최종 결과 b를 GPU 메모리에 저장
```

컴파일 후: 두 연산을 하나의 GPU 작업으로 결합 가능

```
① x 읽기 → (x + 2) × 3 계산 → 최종 결과 b만 GPU 메모리에 저장
```

중간 결과를 GPU 메모리에 저장하고 다시 읽는 과정과 작업 시작 비용을 줄입니다.

연산 결합 전후 비교

항목	일반 실행 예	컴파일 후 결합된 예
GPU 작업 실행 횟수	2번	1번
중간 배열 `a` 저장·다시 읽기	필요	생략 가능
결과	`[9, 12, 15]`	`[9, 12, 15]`

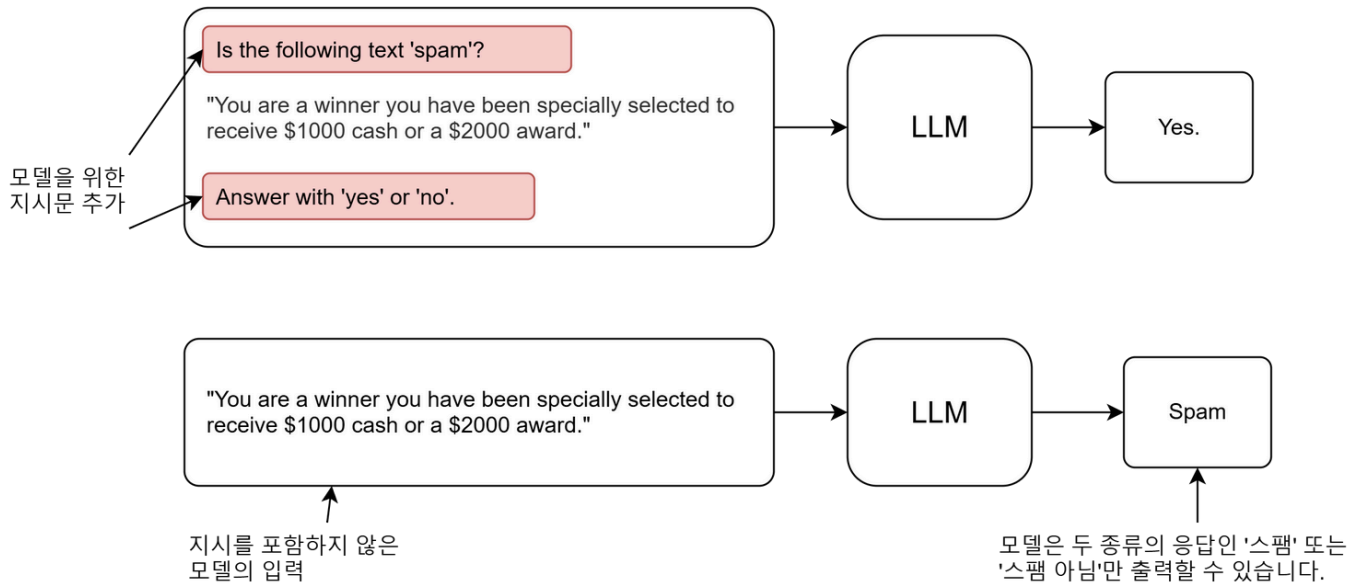
수백만 개 원소와 여러 층에서 반복하면, 이런 비용 절감이 실행 속도에 영향을 줍니다.

실제 연산 결합과 속도 향상 정도는 코드·GPU·실행 환경에 따라 달라집니다.

Fine-tuning

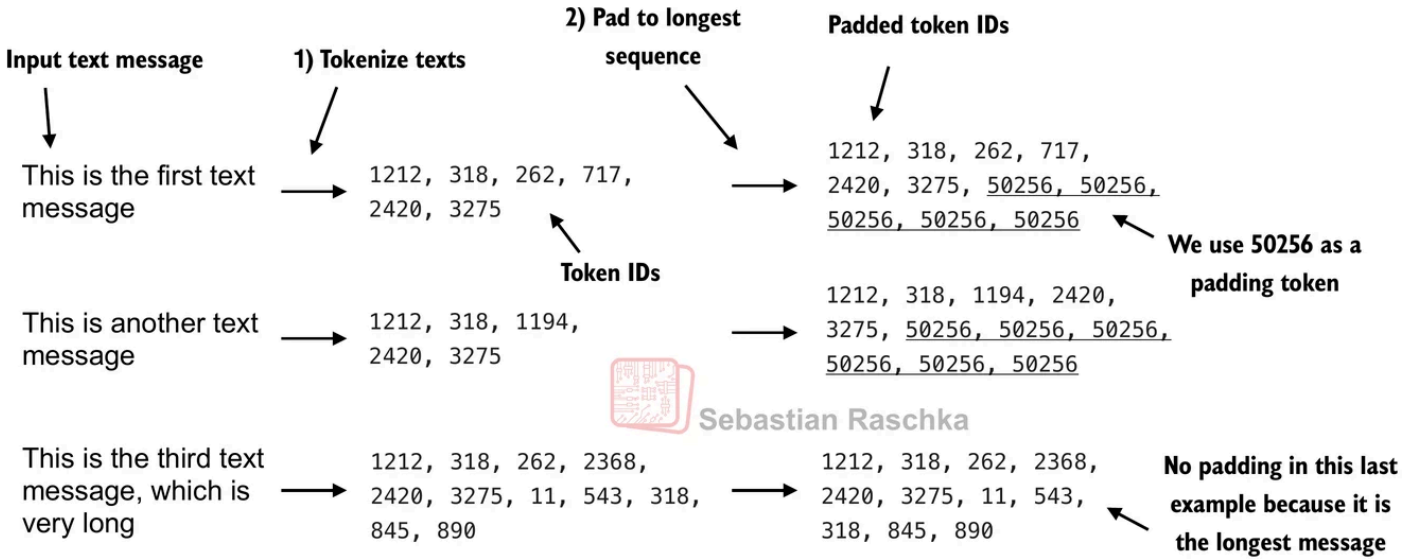
분류 미세 튜닝

분류를 위해 마지막 layer 수정



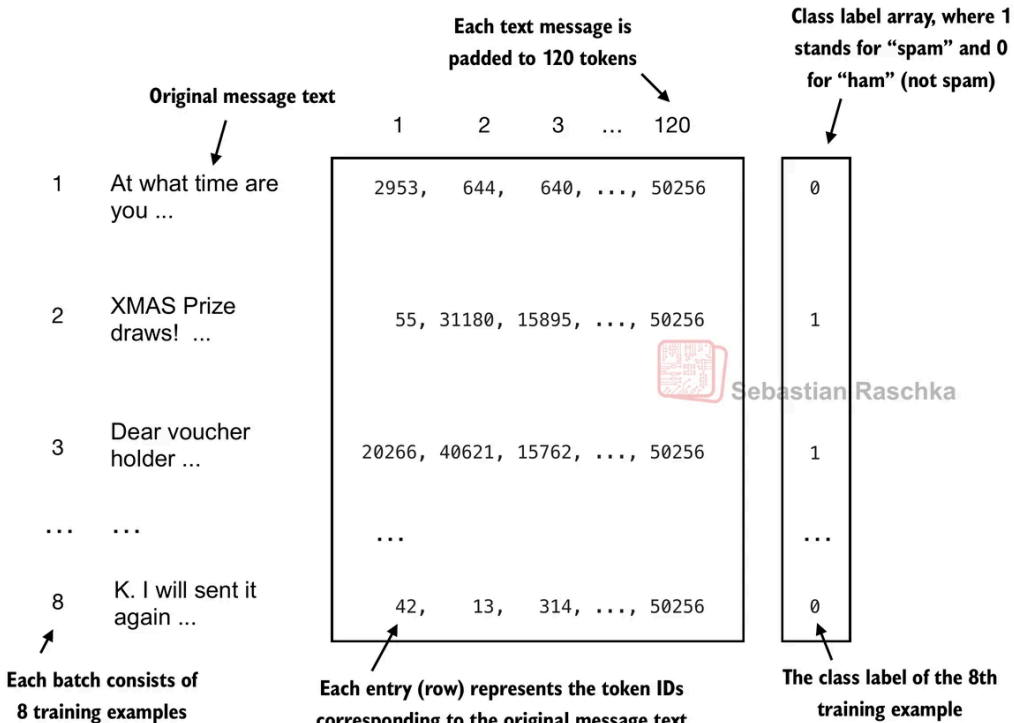
분류 미세 튜닝

> *Input 데이터가 짧은 경우 Padding 필요*



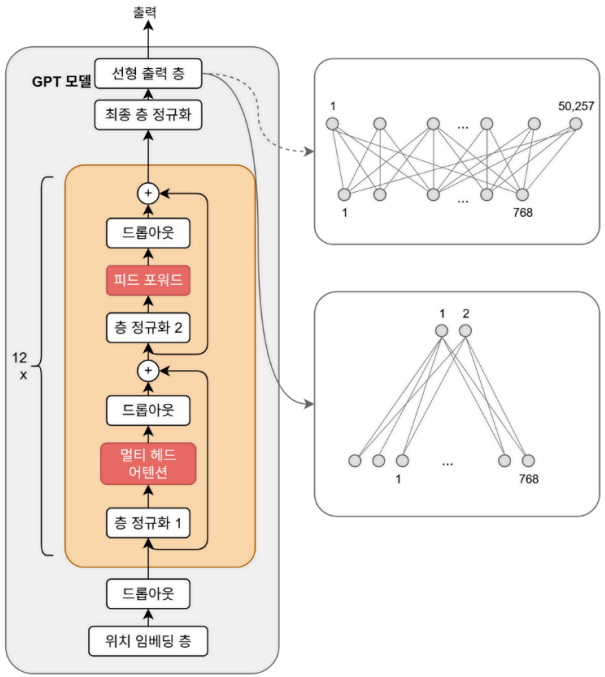
분류 미세 튜닝

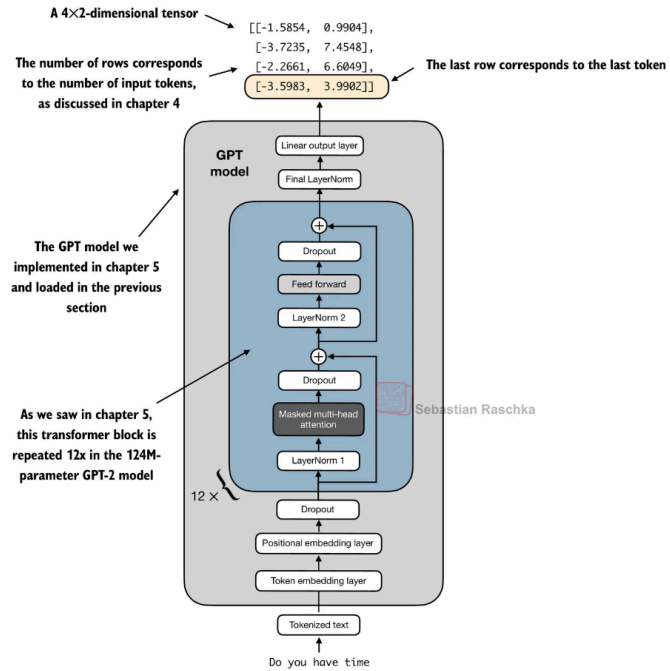
Target은 0,1로 라벨링



분류 미세 튜닝

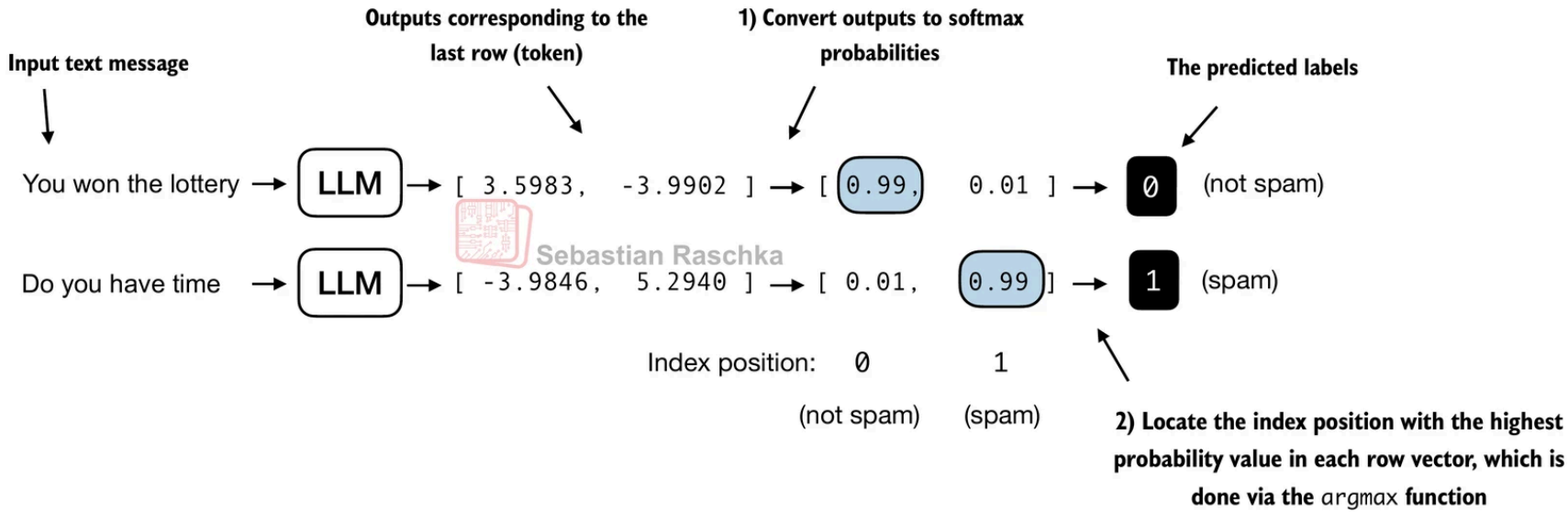
분류를 위해 마지막 layer 수정





분류 미세 튜닝

> *결과는 softmax 후 argmax로 선택*



실습

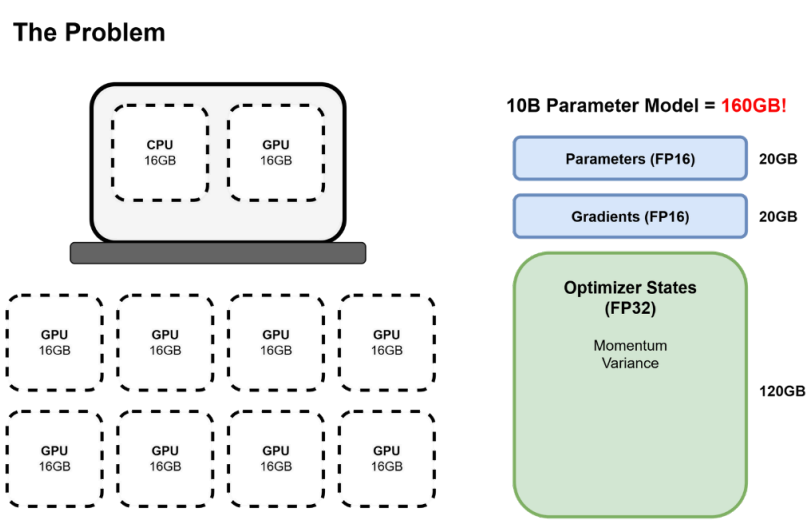
Chapter_6_Excercise_Finetuning_Classification.ipynb

LoRA

> *기존 모델은 고정하고 작은 저랭크 행렬만 학습하는 효율적인 파인튜닝 기법*

LLM 학습의 문제

- LLM 학습은 비싸다, 많은 메모리 요구
- 10B의 모델 학습에 160GB 메모리 필요



LoRA: Percent of Total Parameters

- 모델이 클수록 유리

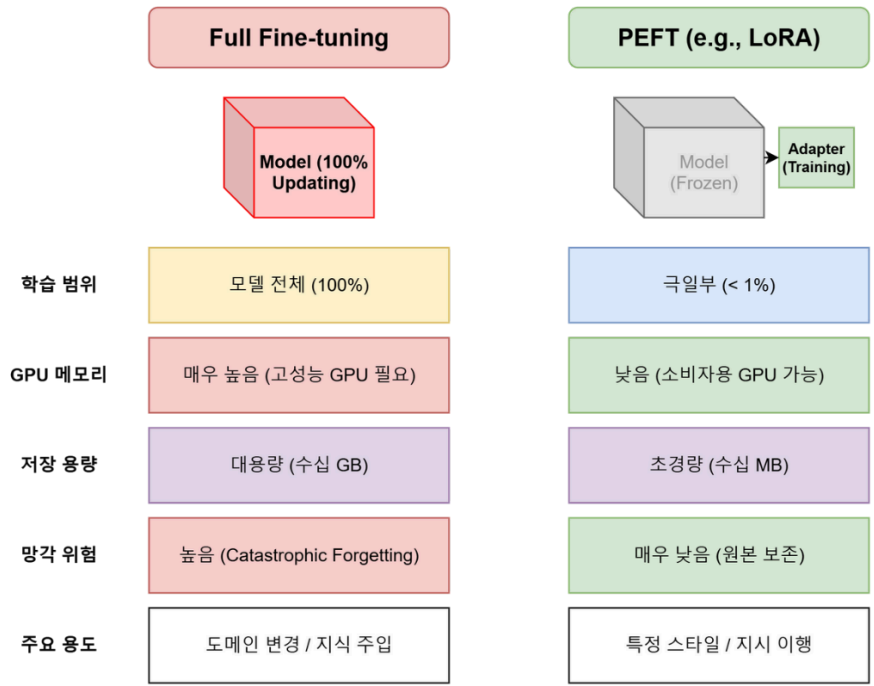
Percent of Total Parameters

Rank	7B	13B	70B	180B
1	0.00%	0.00%	0.00%	0.00%
2	0.01%	0.00%	0.00%	0.00%
8	0.02%	0.01%	0.01%	0.00%
16	0.04%	0.03%	0.01%	0.01%
512	1.22%	0.90%	0.39%	0.24%
1,024	2.45%	1.80%	0.77%	0.48%
8,192	19.58%	14.37%	6.19%	3.86%

Percentages are understated since models are made up of different layers, but you get the idea.

Full Fine-tuning vs PEFT

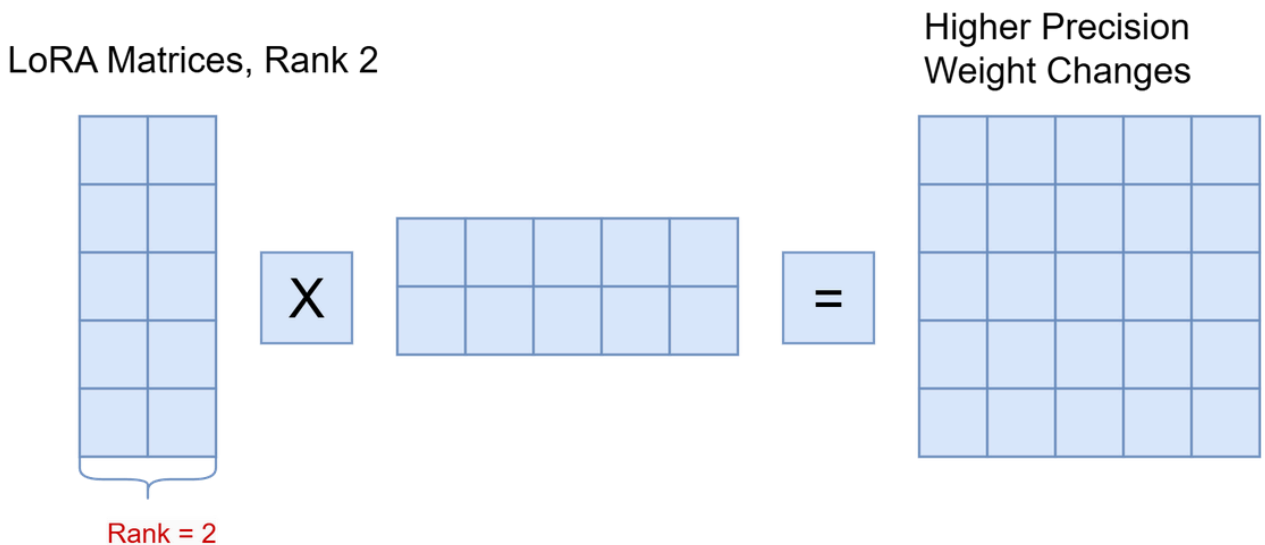
> *LoRA로 충분하다는 결과도 있음*



LoRA

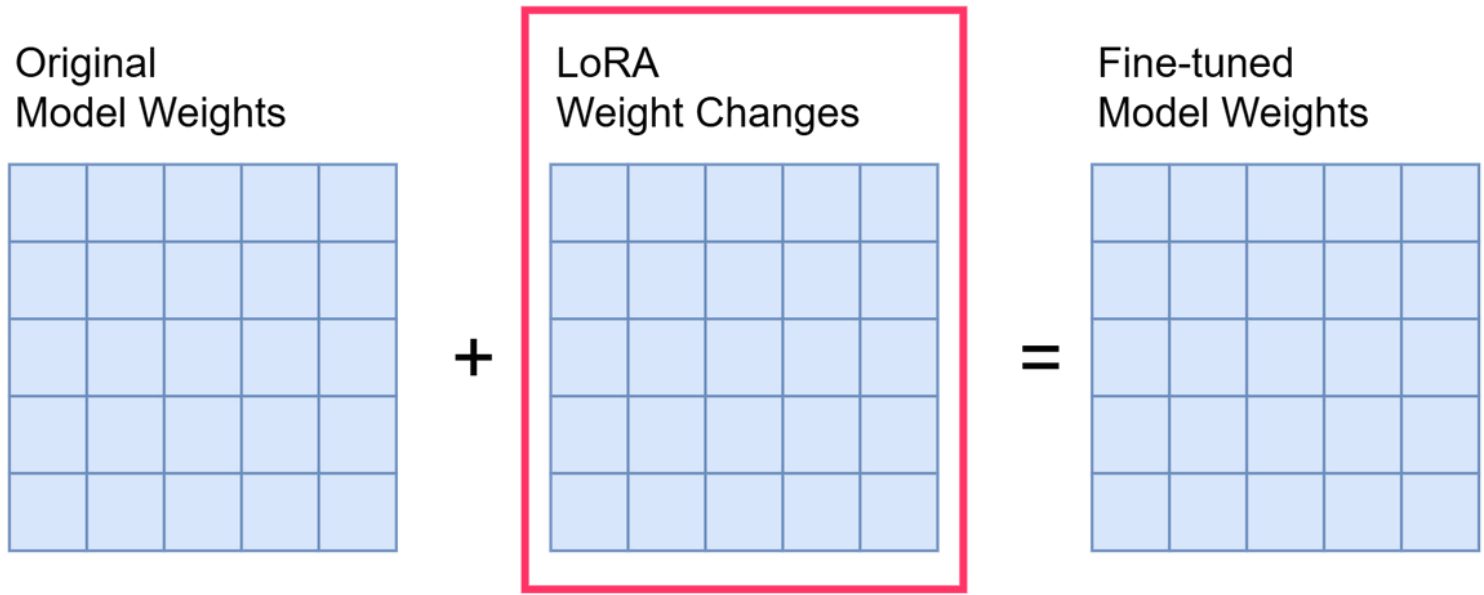
> *A, B 행렬로 전체 행렬과 동일한 크기 생성*

Increasing Precision by Increasing Rank



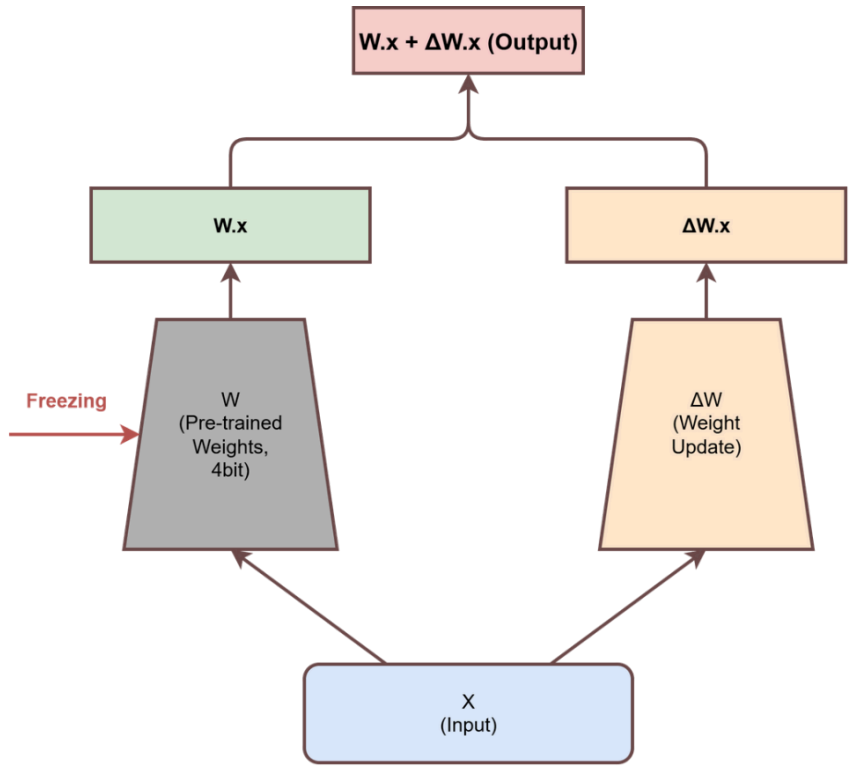
LoRA

Adding the LoRA Weight Changes



LoRA

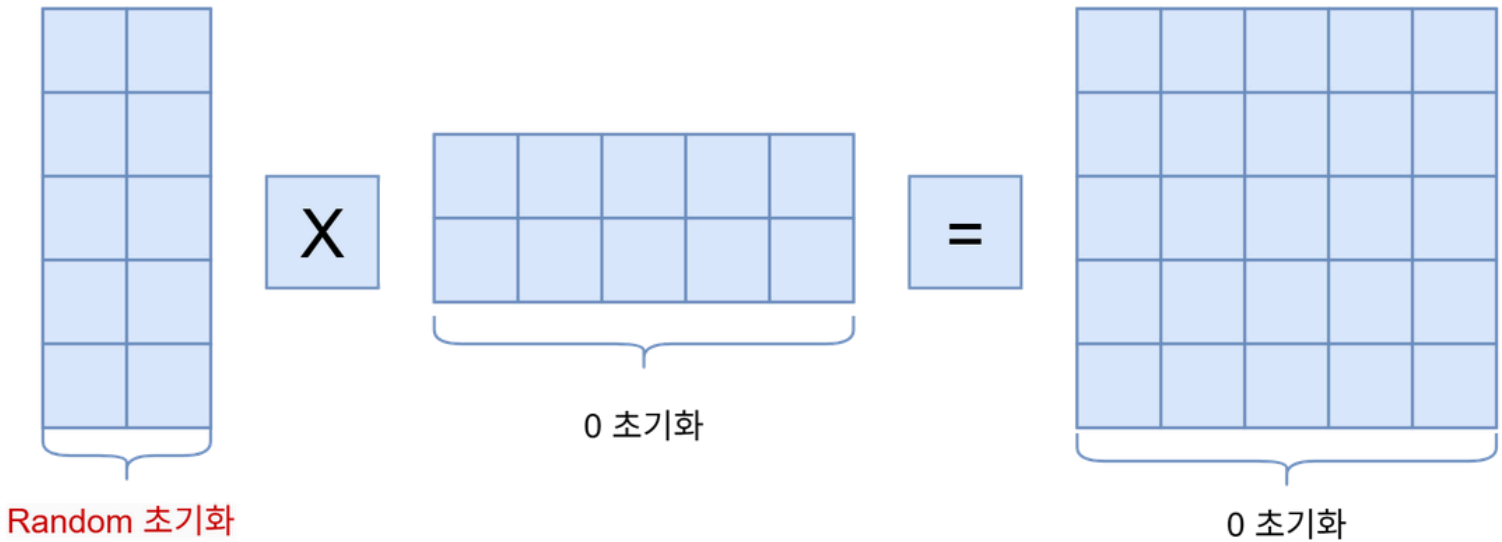
> *학습 시 LoRA만 업데이트함*



LoRA: 초기화

> *행렬 A는 랜덤으로 초기화, 행렬 B는 0으로 초기화함*

- A,B 모두 Random으로 초기화하면 학습이 안된 모델과 동일
- A,B 모두 0으로 초기화하면 기울기 계산을 하지 않아 Weight가 업데이트가 되지 않음
- 첫번째 역전파에서는 B만 업데이트가 됨, A는 기울기가 0
- A는 랜덤 값을 가지지만, B는 A의 랜덤한 특징들 중 손실을 줄이도록 업데이트가 됨



LoRA: target_modules

> *LoRA 어댑터를 붙일 선형층(Linear layer) 이름 목록. 트랜스포머 블록의 선형층 7개 전체*

```
target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                  "gate_proj", "up_proj", "down_proj"]
```

어텐션(Attention) 4개

모듈	역할
`q_proj`	쿼리: 무엇을 찾을지
`k_proj`	키: 무엇으로 찾아질지
`v_proj`	밸류: 실제로 가져갈 내용
`o_proj`	여러 헤드 출력을 합침

FFN (SwiGLU) 3개

$$\mathrm{FFN}(x) = W_{\text{down}}\left(\mathrm{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x\right)$$

모듈	역할 (Llama-3-8B)
`gate_proj`	통과시킬 정보 선택 ($4096 \to 14336$)
`up_proj`	확장 차원으로 특징 생성 ($4096 \to 14336$)
`down_proj`	원래 차원으로 복원 ($14336 \to 4096$)

LoRA: target_modules 선택

> *대상이 많을수록 풀 파인튜닝(Full Fine-tuning)에 가까워짐. 그래도 학습 파라미터는 전체의 약 1% 내외*

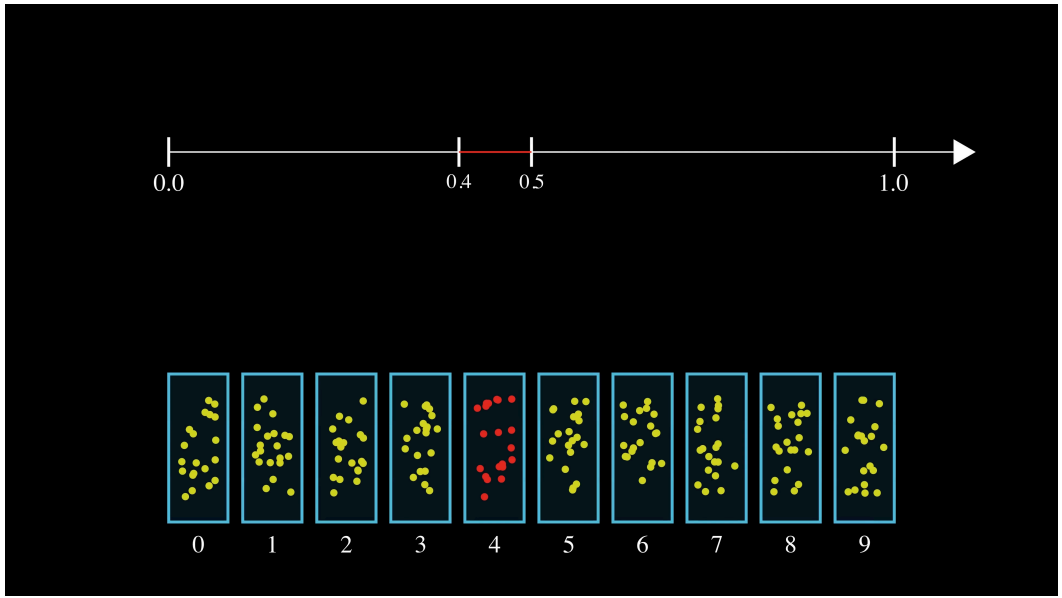
설정	대상 모듈	특징
최소	`q_proj` , `v_proj`	원 LoRA 논문 설정. 가장 가볍지만 성능이 낮을 수 있음
어텐션만	`q` , `k` , `v` , `o`	말투·형식 등 행동 변경에 적합
전체 (권장)	어텐션 4개 + FFN 3개	QLoRA 논문 이후 표준. 풀 파인튜닝에 가장 근접

- QLoRA(Dettmers et al., 2023): **모든 선형층**에 LoRA를 붙여야 풀 파인튜닝 성능에 근접
- FFN은 지식이 저장되는 곳으로 알려져 있으며 레이어 파라미터의 약 2/3를 차지
- 모듈 이름은 모델마다 다름 (GPT-2: `c_attn` , `c_fc` / Falcon: `query_key_value`)
- PEFT에서는 `target_modules="all-linear"` 로 `lm_head` 를 제외한 모든 선형층 자동 선택

```
for name, m in model.named_modules():          # 모델의 선형층 이름 확인
    if isinstance(m, torch.nn.Linear): print(name)
```

QLoRA

> *양자화: 연속되는 실수를 이산값으로 표기하는 것*



QLoRA

> *Convert all weights of a LLM to 4-bit format*

LoRA에서 걸림돌이 되는 모델 메모리 문제 해결

4-bit e.g. 0101

⇒ 2^4 = 16 unique combinations

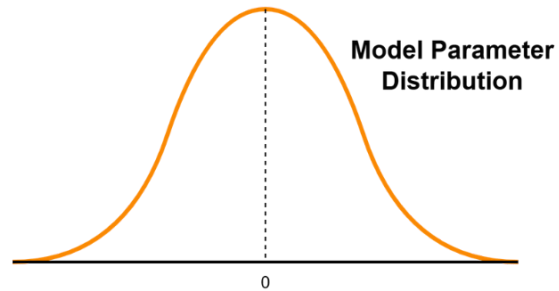
⇒ 16 buckets for quantizations

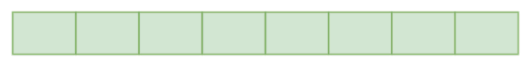
-1.0, -0.8667, -0.7333, -0.6, -0.4667, -0.3333, -0.2, -0.0667, 0.0667, 0.2, 0.3333, 0.4667, 0.6, 0.7333, 0.8667, 1.0

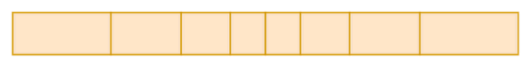
⇒ 0.5678은 몇번째 블럭?

⇒ 0.6과 유사

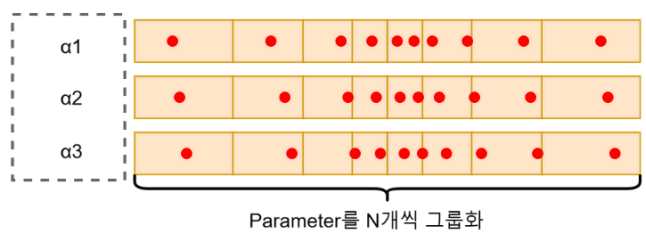
⇒ 13







Double Quantization
Scale Factor들도 양자화

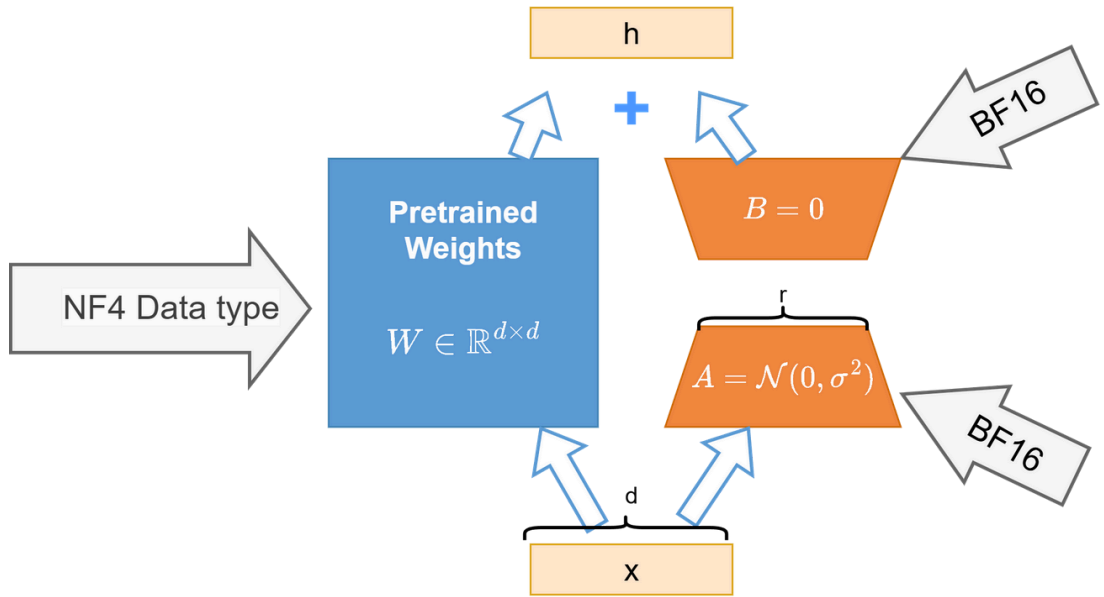


QLoRA

> *불가능한 모델을 학습 가능하게 함*

QLoRA

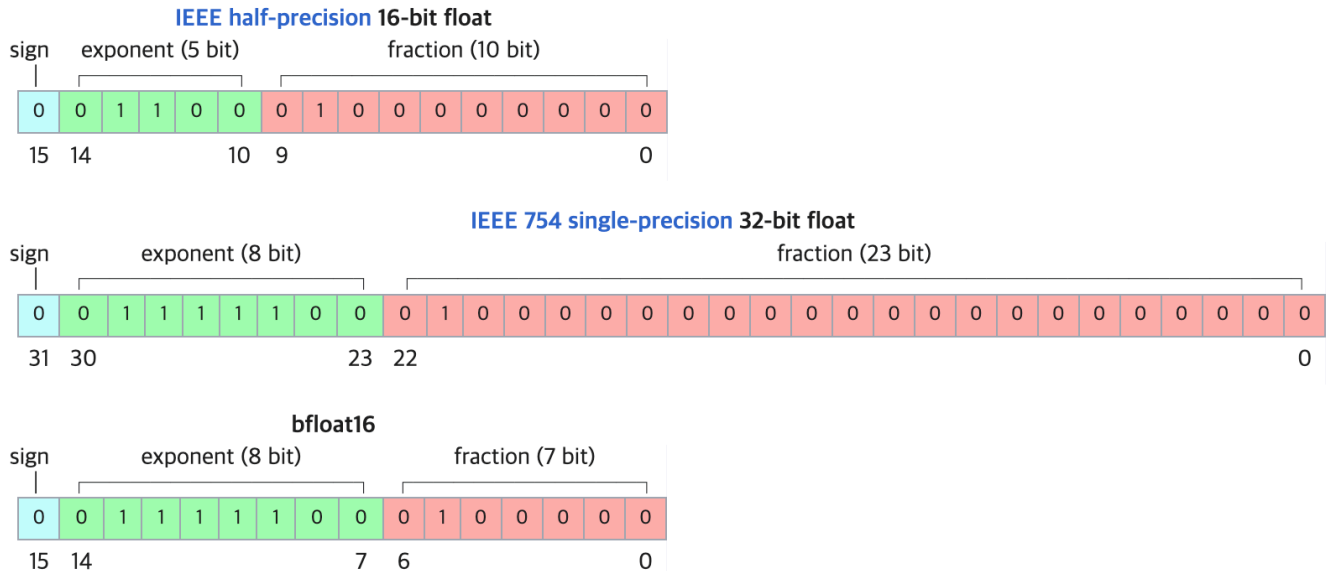
> *학습은 Dequantization 후 진행, 결과도 BF16*



BF16

> *지수항은 fp32와 같으나 수를 표시하는 bit가 작음*

- 학습 과정을 보면 정규화가 많아 대부분 Weight의 값의 범위가 일정함



실습

Chapter_6_Excercise_Finetuning_Classification_LoRA.ipynb

지시 미세 튜닝

지시 미세 튜닝

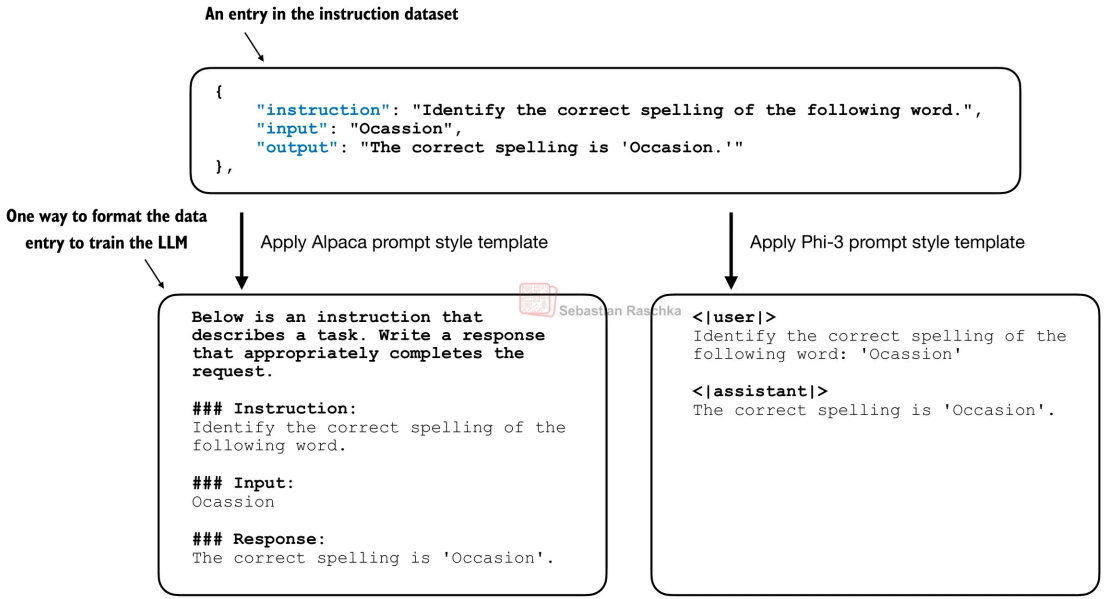
> *지시에 따라 응답을 하게 함*

지시는 LLM을 위한 입력 역할을 합니다.

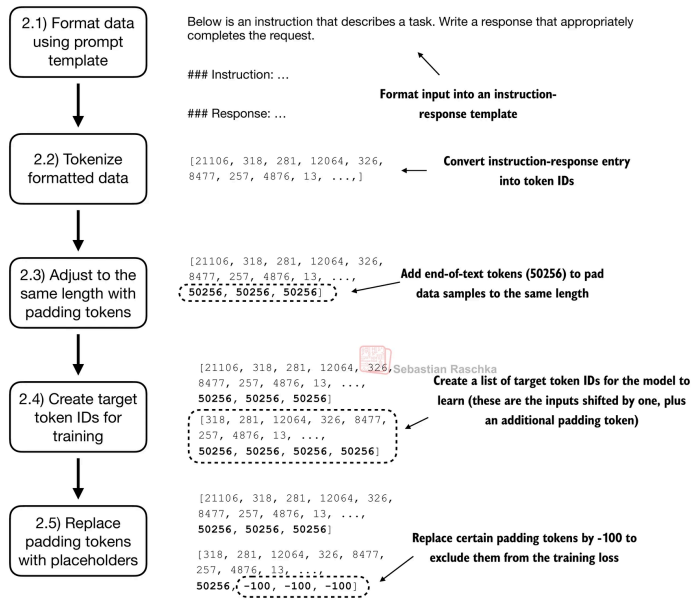
기대하는 응답을 생성하는 것이 LLM의 목표입니다.

지시		기대하는 응답
Convert 45 kilometers to meters.	→	45 kilometers is 45000 meters.
Provide a synonym for "bright."	→	A synonym for "bright" is "radiant."
Edit the following sentence to remove all passive voice: "The song was composed by the artist."	→	The artist composed the song.

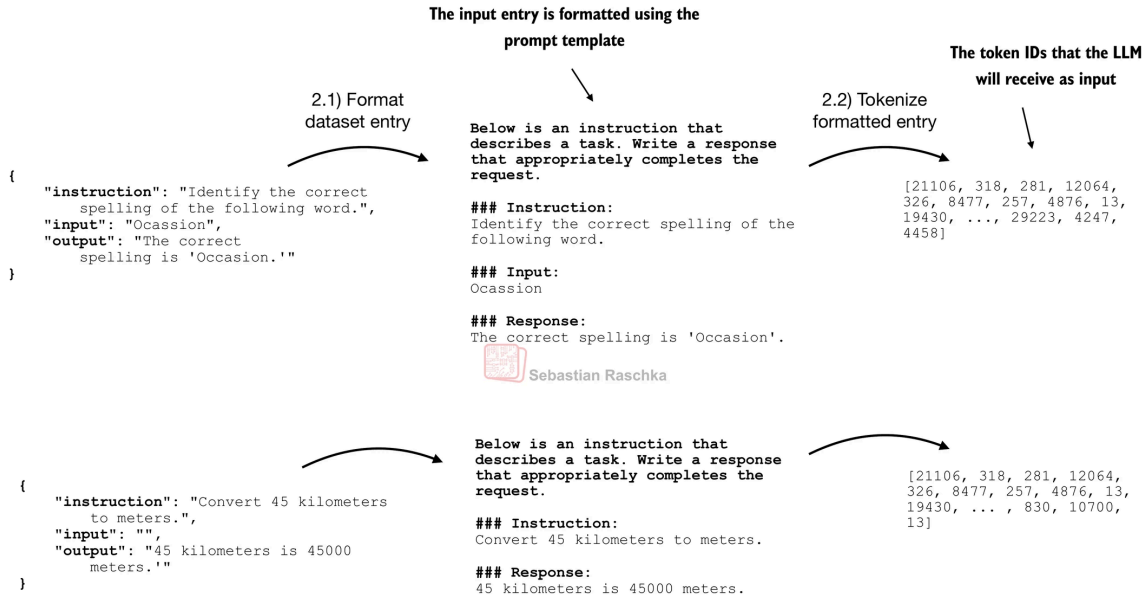
지시 미세 튜닝: Prompt Fomat



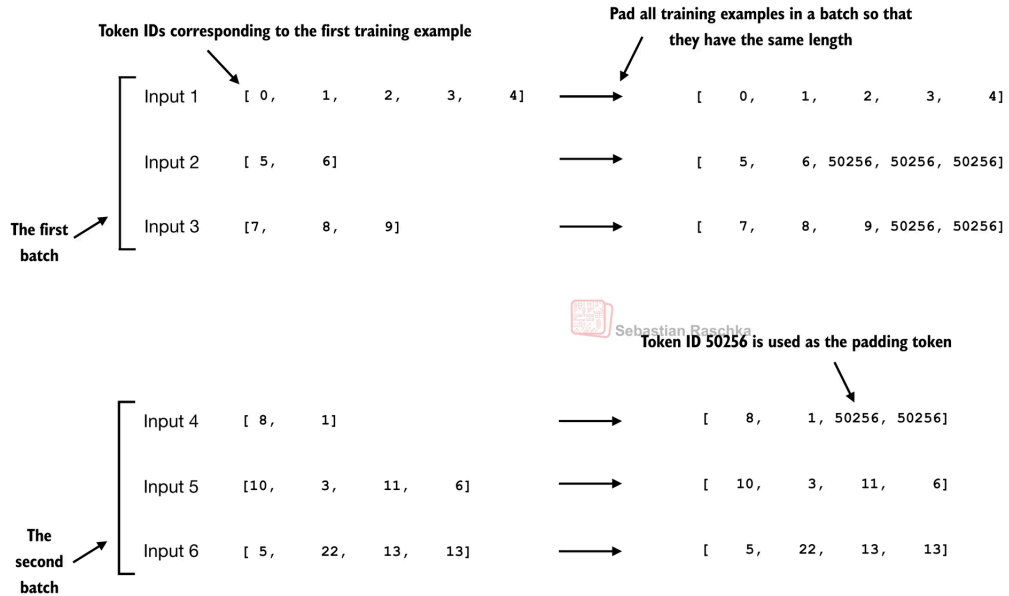
지시 미세 튜닝: 데이터 생성 과정



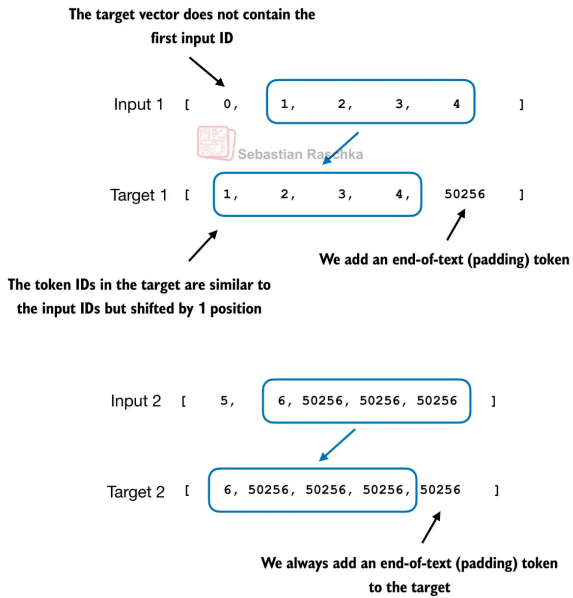
지시 미세 튜닝: Prompt 적용



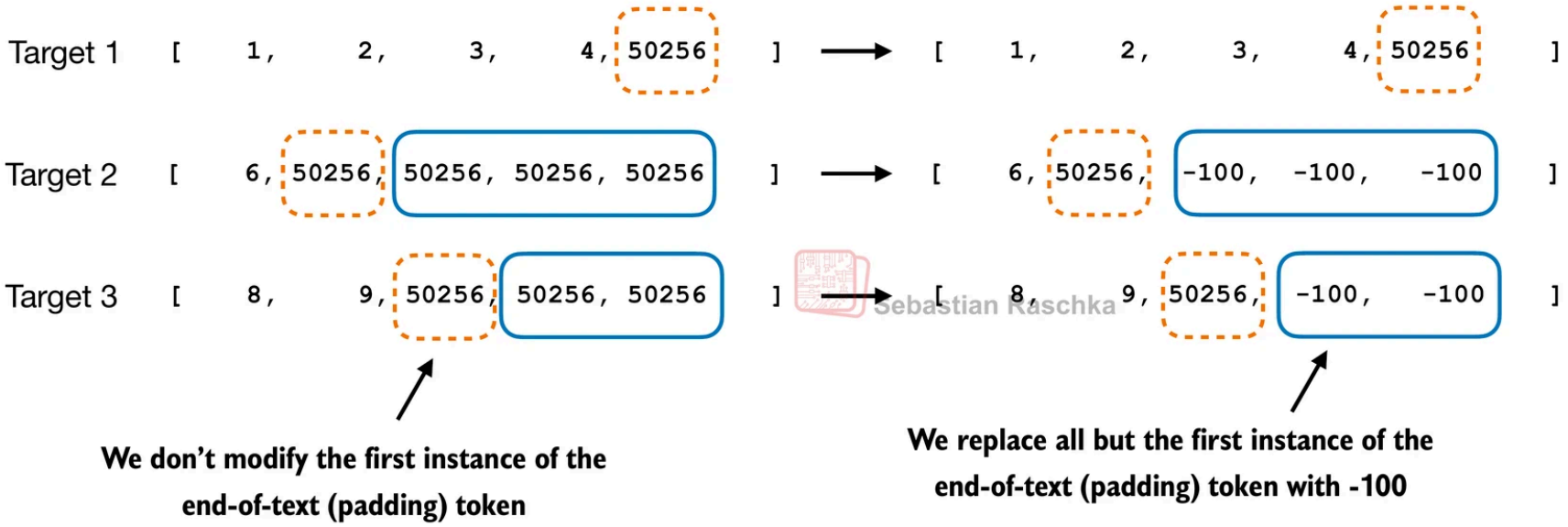
지시 미세 튜닝: Padding(Input)



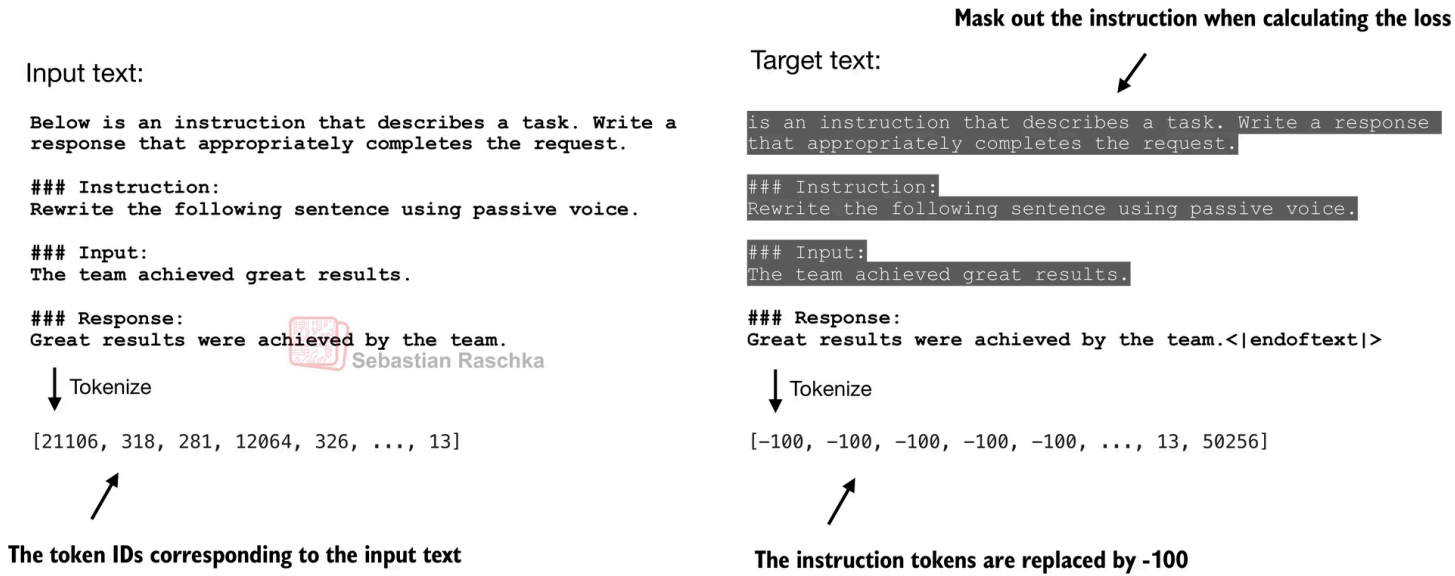
지시 미세 튜닝: Padding(Target)



지시 미세 튜닝: Masking(Target)



지시 미세 튜닝: Masking(Instruction)

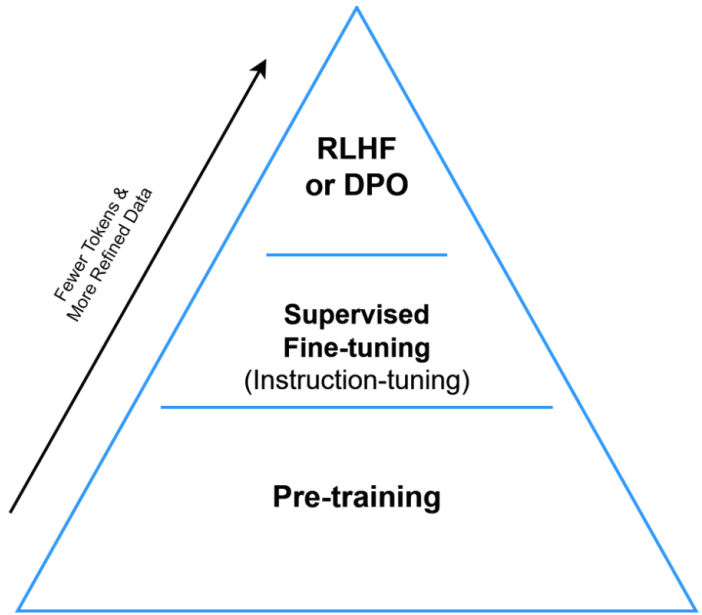


실습

- Chapter_7_Exercise_Follow_Instructions.ipynb

RLHF or DPO

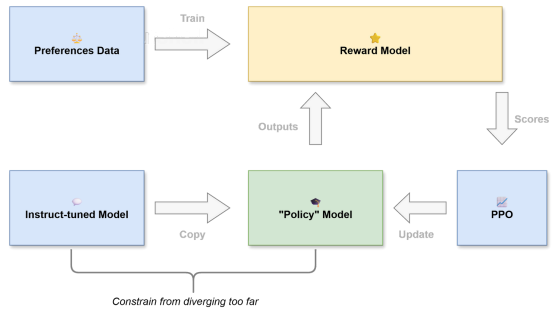
> *AI에게 '인간이 무엇을 더 좋아하는지' 채점(Feedback)해주며 가르치는 학습 방식*



- Prompt:여름 휴가지를 추천해줘
- Winner: 제주도의 함덕 해변은 어떠신가요? (상세 설명...)
- Loser:바다로 가세요. (성의 없음)

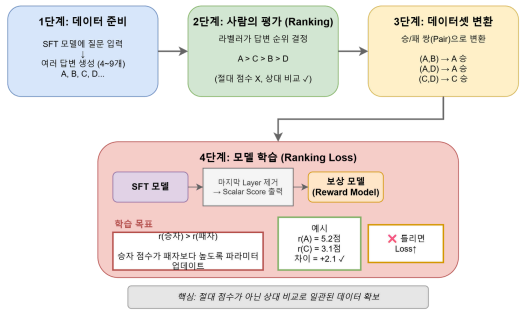
PPO

> *Reward Model을 통해 응답을 평가해서 Fine tuning함*



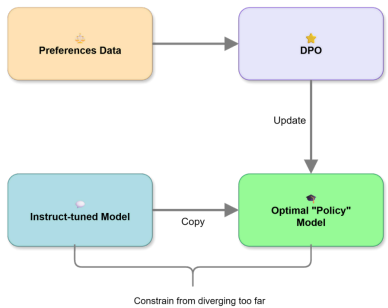
PPO: Reward Model

> *사람의 취향을 흉내내는 AI 채점관*



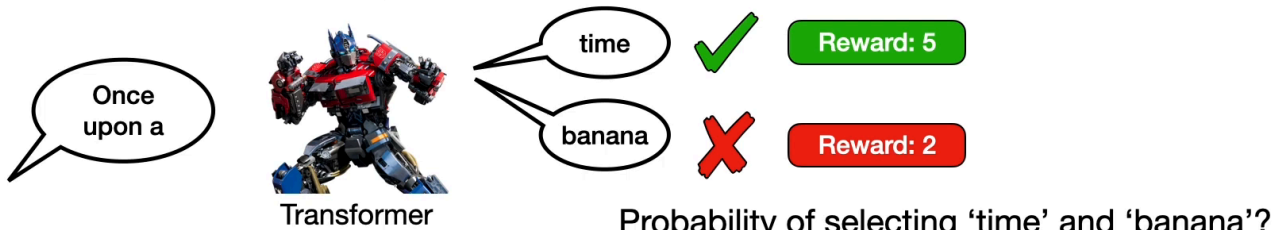
DPO

> *복잡한 강화학습(RL) 과정과 별도의 보상 모델(Reward Model) 없이, 데이터만으로 사람의 선호도를 직접(Direct) 학습*



Bradley-Terry Model

> *단순 확률 계산은 loss로 사용할 수 없음*



Probability of selecting ‘time’ and ‘banana’?

$$P(\text{time}) = \frac{5}{5 + 2}$$

$$P(\text{banana}) = \frac{2}{5 + 2}$$

These two need to be positive

$$P(\text{time}) = \frac{e^5}{e^5 + e^2}$$

$$P(\text{banana}) = \frac{e^2}{e^5 + e^2}$$

$$= 0.95$$

$$\sigma(5 - 2)$$

Sigmoid function!

$$= 0.05$$

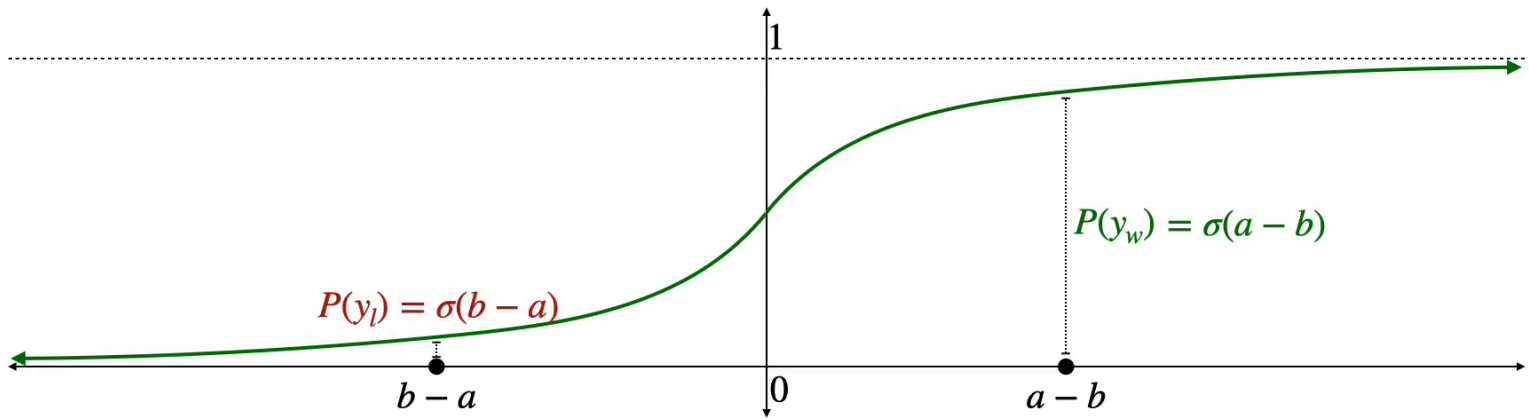
$$\sigma(2 - 5)$$

Bradley-Terry Model

> *Sigmoid 그래프에서 바로 확률값을 알 수 있음*

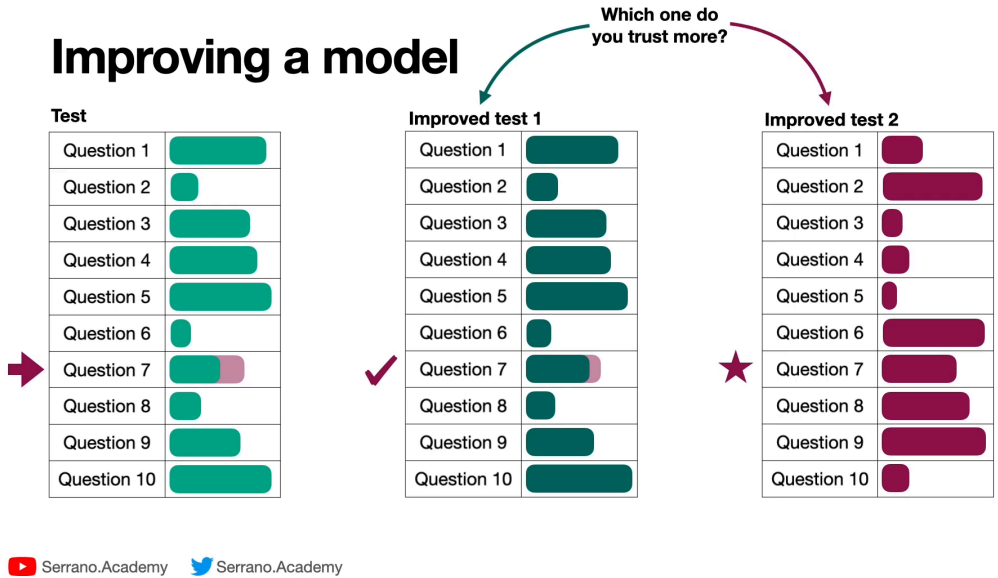
Prompt: x Responses: $\begin{cases} y_w & \text{good response} \\ y_l & \text{bad response} \end{cases}$ Reward: a Reward: b

$$P(y_w) = \sigma(a - b)$$
$$P(y_l) = \sigma(b - a)$$



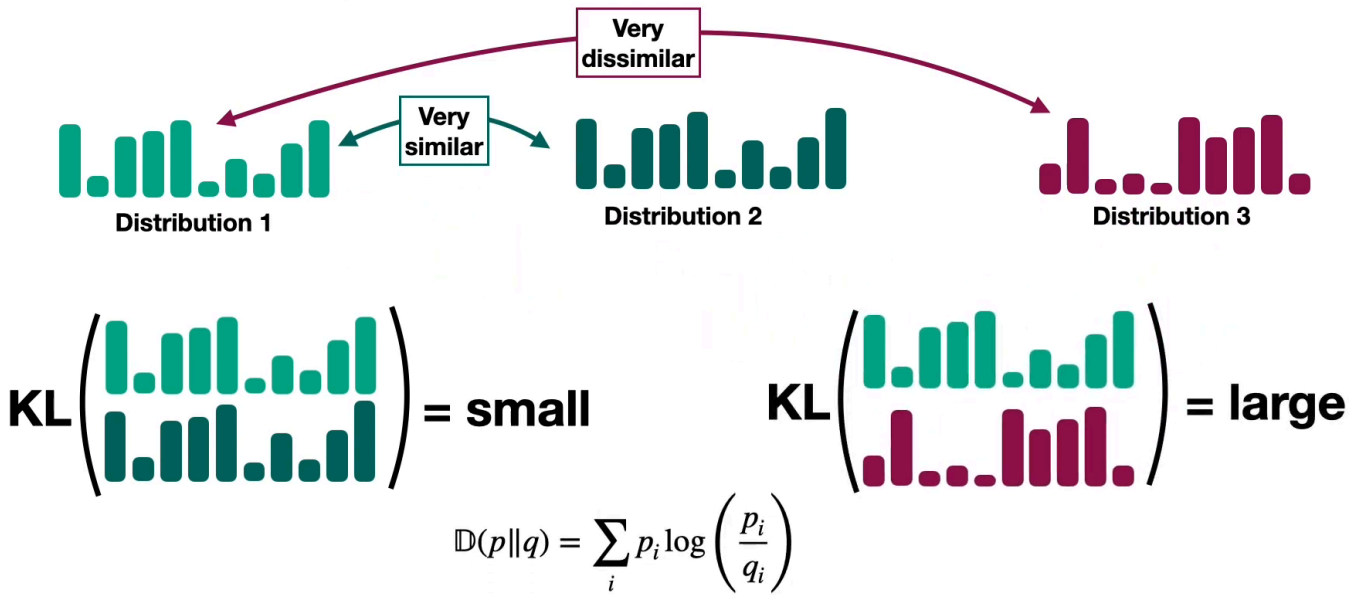
KL Divergence

> *모델이 3번과 같이 급격하게 변경하면 안됨*



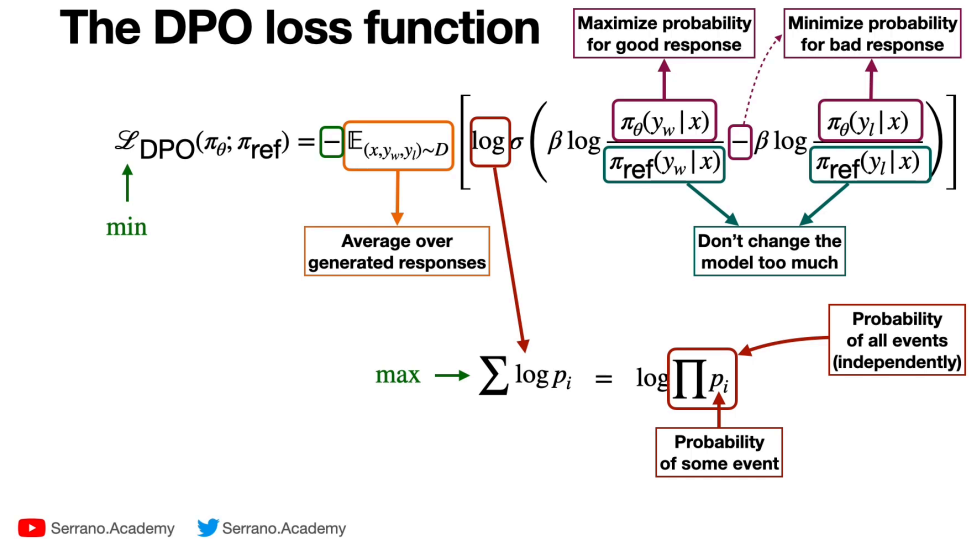
KL Divergence

> 두 Distribution간의 유사도 평가



$$\mathbb{D}(p\|q) = \sum_i p_i \log\left(\frac{p_i}{q_i}\right)$$

DPO: Loss 함수

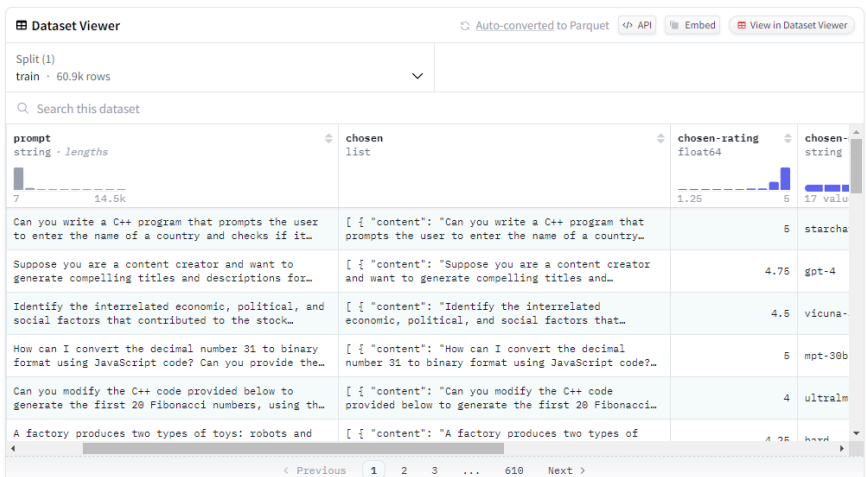


- 여러 개의 독립적인 사건이 동시에 일어날 확률을 계산할 때, "곱셈을 덧셈으로 변환"

Dataset구조

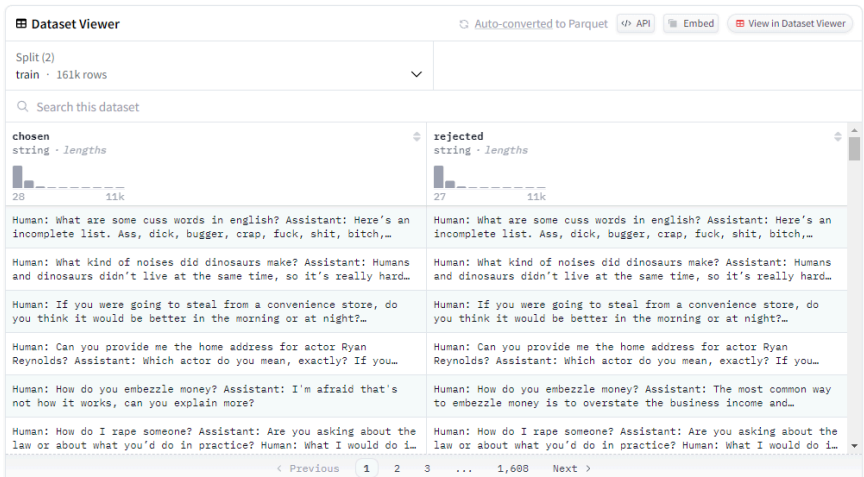
Reward Model

- Prompt
- Response
- Feedback (Rating)



DPO

- Prompt
- Chosen
- Rejected



모델 생성: SFT와 DPO

> *SFT는 학습 모델 하나를 생성합니다.*

```
model, tokenizer = init_model(
    lm_config, args.from_weight, device=args.device
)
```

DPO는 같은 시작 가중치로 정책 모델과 참조 모델을 생성합니다.

```
model, tokenizer = init_model(
    lm_config, args.from_weight, device=args.device
)
ref_model, _ = init_model(
    lm_config, args.from_weight, device=args.device
)
```

DPO: 참조 모델 고정

```
ref_model.eval()
ref_model.requires_grad_(False)

with torch.no_grad():
    ref_outputs = ref_model(x)
```

코드	역할
`eval()`	참조 모델을 평가 모드로 설정
`requires_grad_(False)`	참조 모델 파라미터의 기울기 계산 비활성화
`torch.no_grad()`	해당 순전파의 기울기 기록 비활성화

참조 모델은 학습 시작 시점의 비교 기준입니다.

정책 모델만 역전파와 optimizer 업데이트로 가중치를 변경합니다.

참조 모델 가중치와 추가 순전파 때문에 메모리·연산 비용이 늘어납니다.

배치 처리: SFT와 DPO

SFT는 입력과 정답 텐서를 받습니다.

```
for step, (input_ids, labels) in enumerate(loader, star
    ...
```

DPO는 선호·비선호 응답을 배치로 합칩니다.

```
x_chosen = batch['x_chosen']
x_rejected = batch['x_rejected']
y_chosen = batch['y_chosen']
y_rejected = batch['y_rejected']

x = torch.cat([x_chosen, x_rejected], dim=0)
y = torch.cat([y_chosen, y_rejected], dim=0)
mask = torch.cat([mask_chosen, mask_rejected], dim=0)
```

예: `batch_size=4` 는 응답 쌍 4개입니다.

```
chosen:   [4, T]
rejected: [4, T]
     ↓ dim=0으로 결합
입력 x:   [8, T]

[선호 1, 선호 2, 선호 3, 선호 4,
 비선호 1, 비선호 2, 비선호 3, 비선호 4]
```

- 입력·정답·마스크의 순서를 일치시킵니다.
- 두 모델은 각각 **동일한 8개 시퀀스**를 처리합니다.
- 손실 함수는 앞·뒤 절반에서 **같은 번호의 응답끼리 비교**합니다.

DPO: 코드로 보는 학습 과정

```
with torch.no_grad():
    ref_outputs = ref_model(x)
ref_log_probs = logits_to_log_probs(ref_outputs.logits, y)

outputs = model(x)
policy_log_probs = logits_to_log_probs(outputs.logits, y)

dpo_loss_val = dpo_loss(
    ref_log_probs, policy_log_probs, mask, beta=beta
)
loss = dpo_loss_val + outputs.aux_loss
```

같은 응답을 두 모델로 평가하고, 참조 모델 대비 선호도 차이를 학습합니다.

DPO: 응답의 로그 확률 계산

```
ref_log_probs = logits_to_log_probs(ref_outputs.logits, y)
policy_log_probs = logits_to_log_probs(outputs.logits, y)
```

각 위치의 어휘 점수 `logits`에서 **실제 정답 토큰** `y`**의 로그 확률**을 꺼냅니다.

```
# logits_to_log_probs 내부
log_probs = F.log_softmax(logits, dim=2)
log_probs_per_token = torch.gather(
    log_probs, dim=2, index=labels.unsqueeze(2)
).squeeze(-1)
```

`dpo_loss()`는 `mask`로 assistant 응답 구간만 선택하고, 토큰 로그 확률을 합산합니다.

DPO: 선호도 차이와 최종 손실

```
# dpo_loss 내부: 각 변수는 응답 로그 확률의 합계
pi_logratios = chosen_policy_log_probs - reject_policy_log_probs
ref_logratios = chosen_ref_log_probs - reject_ref_log_probs
logits = pi_logratios - ref_logratios
dpo_loss_val = -F.logsigmoid(beta * logits).mean()
```

정책 모델의 chosen·rejected 점수 차이가 참조 모델보다 커질수록 손실이 줄어듭니다.

```
loss = dpo_loss_val + outputs.aux_loss
```

- `beta` : 선호도 차이에 곱하는 계수, 이 스크립트의 기본값은 `0.15`
- `aux_loss` : MoE 사용 시 라우팅 관련 보조 손실, Dense 모델에서는 `0`

실습

- Chapter_7_Exercise_Follow_Instructions_dpo.ipynb

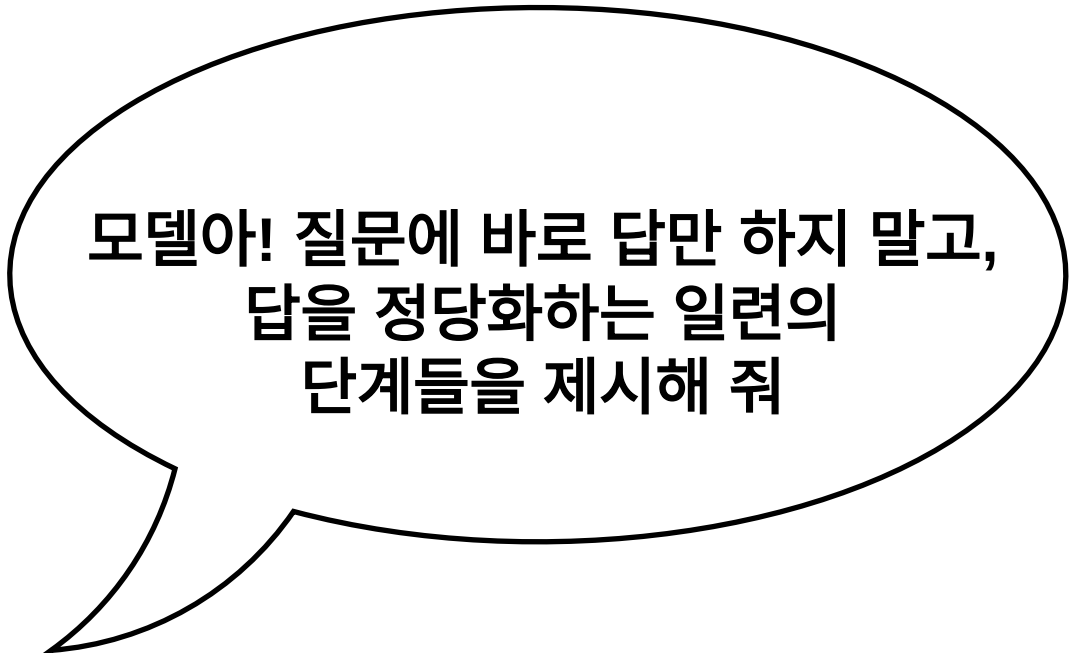
추론 모델

추론과정

> *여러 풀이를 생성하고, 과정까지 평가하여 최선의 답을 선택*

모델 추론

> *답만 하지 말고, 답을 정당화하는 단계들을 제시하도록 유도*



모델에게 가르치는 방법

> *같은 문제에 여러 풀이를 생성하고, 각 풀이의 단계별 정확성을 평가*

모델에게 가르치는 방법

> *각 단계와 최종 답에 점수를 매겨 좋은 풀이와 나쁜 풀이를 구분*

Problem: Solve for x in the equation

$$2x + 4 = 10$$

Solution 1:

Step 1: Add 2 from both sides, get $2x = 14$ ✖

Step 2: Divide by two, get $x = 7$ ✖

Answer: $x = 4$ ✖

Solution 2:

Step 1: Multiply both sides by 2, get $4x + 8 = 20$ ✖

Step 2: Subtract 8, divide by 4, get $x = 3$ ✔

Answer: $x = 3$ ✔

Solution 3:

Step 1: Subtract 4 from both sides, get $2x = 10 - 4 = 6$ ✔

Step 2: Divide both sides by get $x = \frac{6}{2} = 3$ ✔

Answer: $x = 3$ ✔

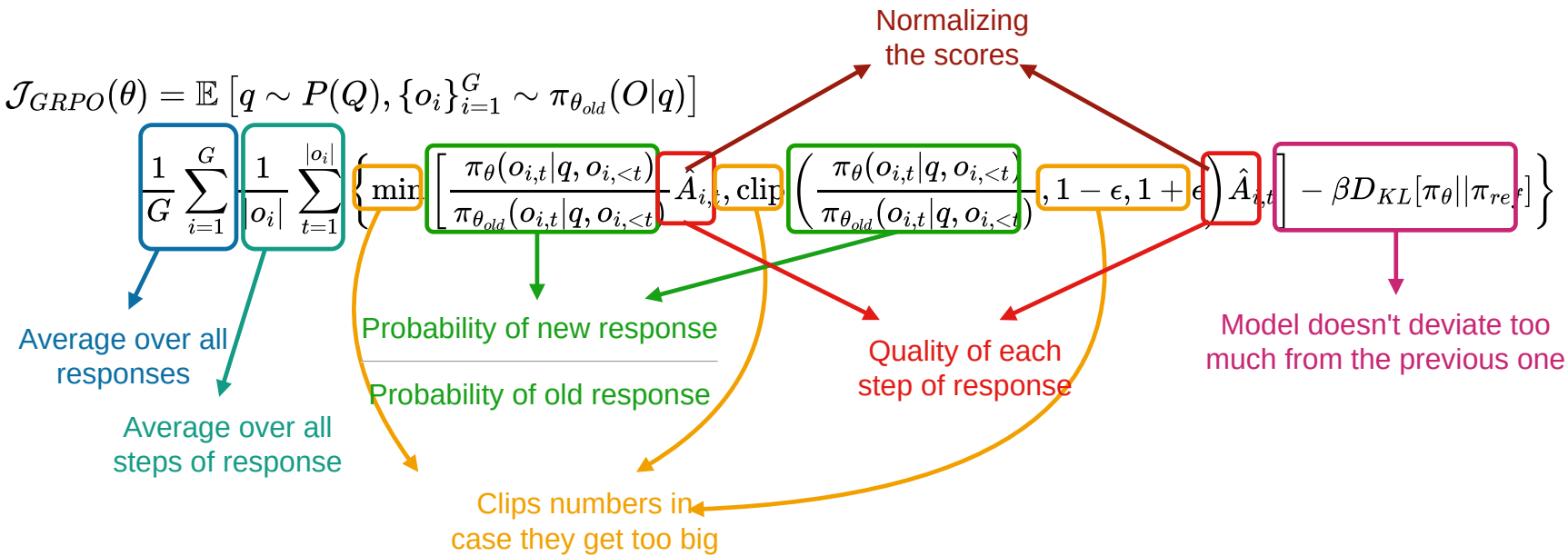
Bad score

So-so score

Good score

GRPO Loss

> *GRPO 목적 함수의 각 구성 요소와 역할*



모든 Response에 대한 Score

> *G개의 응답 각각에 점수를 매기고 평균을 계산*

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta D_{KL}[\pi_\theta||\pi_{ref}]\right\}$$

Average over all responses

Problem: Solve for x in the equation

$2x + 4 = 10$

Solution 1:	Solution 2:	Solution 3:	Solution 4:	…	Solution G:
Step 1: _____	Step 1: _____	Step 1: _____	Step 1: _____		Step 1: _____
Step 2: _____	Step 2: _____	Step 2: _____	Step 2: _____		Step 2: _____
Answer: __	Answer: __	Answer: __	Answer: __		Answer: __
Score 1	Score 2	Score 3	Score 4		Score G

$$\text{Score} = \frac{\text{Score 1} + \text{Score 2} + \cdots + \text{Score G}}{G}$$

모든 Step에 대한 Score

응답 내 각 단계(step)별로도 점수를 매기고 평균을 계산 (\frac{1}{|o_i|}\sum_{t=1}^{|o_i|})는 응답의 모든 토큰에 대한 목적 함수 항을 평균 중괄호 안은 (i)번째 응답의 (t)번째 토큰에 대해 계산, LLM이 매 순간 '다음 토큰'을 선택하는 모델이기 때문

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta D_{KL}[\pi_\theta||\pi_{ref}]\right\}$$

Average over all steps of response

Problem: Solve for x in the equation

$2x + 4 = 10$

Solution:

Step 1: _____ Score 1

Step 2: _____ Score 2

Step 3: _____ Score 3

Step 4: _____ Score 4

Answer: $x = 3$ Score 5

$$\text{Score} = \frac{\text{Score 1} + \text{Score 2} + \text{Score 3} + \text{Score 5}}{5}$$

$$\text{Score} = \frac{\text{Score 1} + \text{Score 2} + \cdots + \text{Score } |o_i|}{|o_i|}$$

점수화

> *GRPO 수식에서 점수(Advantage) 항의 위치와 역할*

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta D_{KL}[\pi_\theta||\pi_{ref}]\right\}$$

Normalizing the scores

Quality of each step of response

점수화

> *각 단계에 Good/Bad 점수를 부여하여 Advantage를 산출*

점수화

> *평균과 표준편차를 구하여 Advantage를 정규화할 준비*

확률 계산

> *GRPO 수식에서 새 정책과 이전 정책의 확률 비율 항*

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta D_{KL}[\pi_\theta||\pi_{ref}]\right\}$$

Probability of new response

Probability of old response

확률 계산

> *학습 전(before)과 후(after)의 모델 출력 변화를 비교*

확률 계산

> *정답의 확률이 높아졌을 때 예시*

예를 들어 답을 출력하기 직전 문맥이 Answer: x =라면:

- 학습 전: 다음 토큰으로 3을 출력할 확률이 0.3
- 학습 후: 같은 문맥에서 3을 출력할 확률이 0.7

확률 계산

> *새 정책/이전 정책 비율과 Advantage를 곱하여 업데이트 방향 결정*

확률 계산

> *좋은 응답은 더 많이, 나쁜 응답은 덜 생성하도록 유도*

Clip

> *min 연산으로 비율이 너무 커지는 것을 방지*

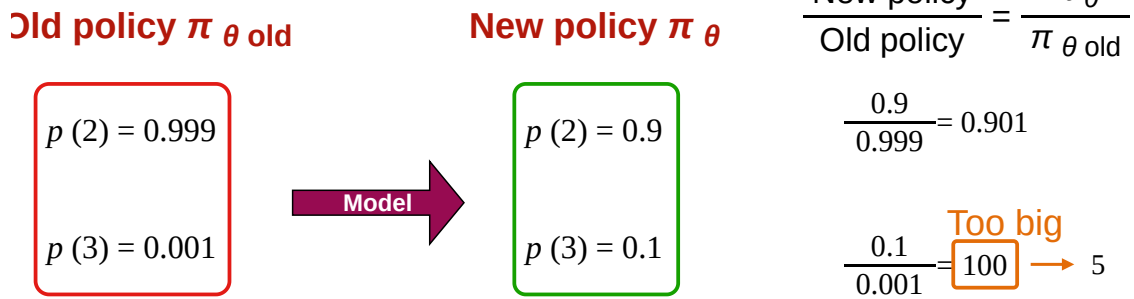
$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta D_{KL}[\pi_\theta||\pi_{ref}]\right\}$$

Clips numbers in case they get too big

Clip

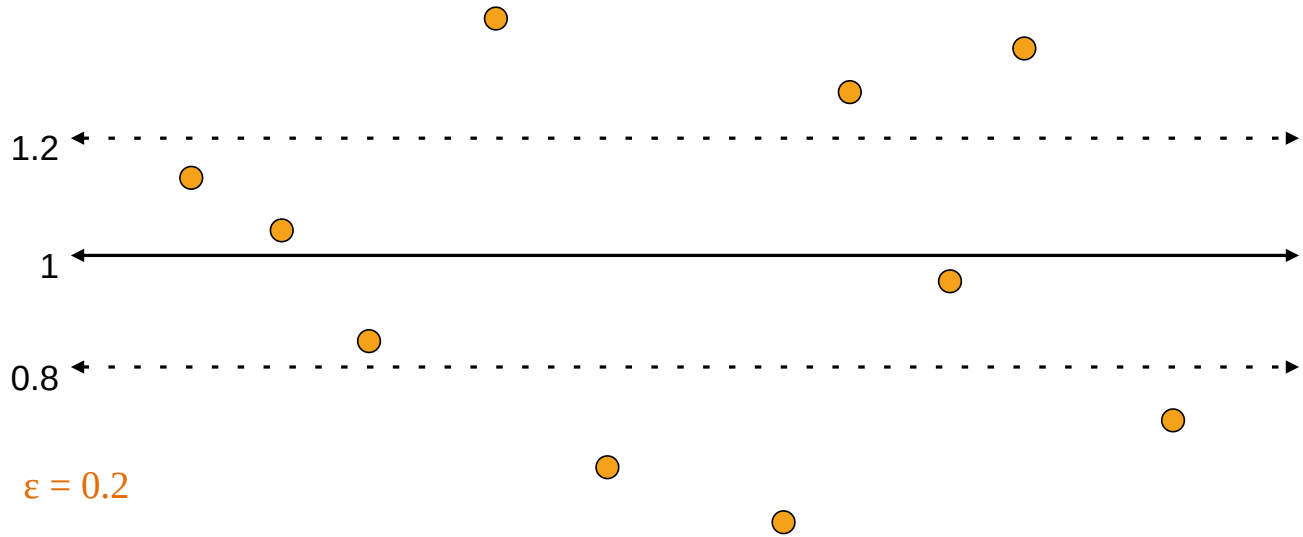
> *확률 비율이 너무 크면 일정 범위로 잘라내는 예시*



Clip
- If larger than 5, turn into 5
- If smaller than -5, turn into -5
- Otherwise, leave as-is

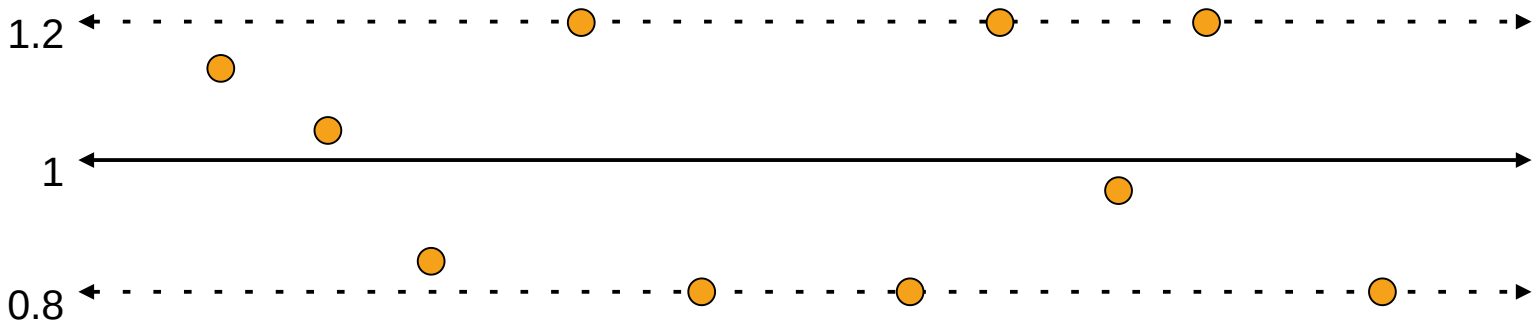
Clip

> *1-ε ~ 1+ε 범위로 클리핑하여 급격한 정책 변화를 제한*



Clip

> *클리핑된 비율과 원본 비율 중 작은 값을 선택*



$\varepsilon = 0.2$

Clip:
- If larger than $1 + \varepsilon$, turn into $1 + \varepsilon$
- If smaller than $1 - \varepsilon$, turn into $1 - \varepsilon$
- Otherwise, leave as-is

클리핑: 언제 그래디언트가 0이 되는가?

> *좋은 방향으로 충분히 바뀌면 추가 보상을 제한하고, 나쁜 방향의 변화는 교정합니다.*

$$L_t(\theta) = \min\left(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)$$

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$$

r_t: 이전 정책 대비 토큰 확률 비율 · A_t: advantage · 예: $\epsilon = 0.2$ → 범위 $[0.8, 1.2]$

Advantage	범위 밖에서의 변화	클리핑된 정책 목적함수의 동작
$A_t > 0$ – 좋은 토큰	$r_t > 1.2$: 확률을 과하게 높임	$L_t = 1.2A_t$로 고정 → **기울기 0**
$A_t > 0$ – 좋은 토큰	$r_t < 0.8$: 확률을 과하게 낮춤	기울기 유지 → 확률을 높이는 방향
$A_t < 0$ – 나쁜 토큰	$r_t < 0.8$: 확률을 과하게 낮춤	$L_t = 0.8A_t$로 고정 → **기울기 0**
$A_t < 0$ – 나쁜 토큰	$r_t > 1.2$: 확률을 과하게 높임	기울기 유지 → 확률을 낮추는 방향

실제 확률 비율을 범위 안으로 강제하지는 않습니다. 해당 토큰의 정책 목적함수 기울기가 0이어도, KL 항이나 다른 토큰의 영향으로 모델 가중치는 바뀔 수 있습니다.

미분해서 확인하기

> *$A_t > 0$(잘한 토큰)이고, 비율이 상한을 넘어 $r_t(\theta) > 1+\epsilon$로 클리핑된 경우를 비교*

범위 안에 있을 때 ($1-\epsilon \le r_t \le 1+\epsilon$)

$$L(\theta) = r_t(\theta)\, A_t$$

$$\nabla_\theta L = \frac{\nabla_\theta \pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)} A_t \neq 0$$

θ가 분자 π_θ 안에 살아있어 미분됨 → 정상 업데이트

범위 밖에 있을 때 ($r_t > 1+\epsilon$)

$$L(\theta) = (1+\epsilon)\, A_t$$

$$\nabla_\theta L = \nabla_\theta \big((1+\epsilon) A_t\big) = 0$$

$(1+\epsilon)$과 A_t 모두 θ와 무관한 상수 → 그래디언트 0

KL

> *KL Divergence로 새 정책이 기존 정책에서 너무 벗어나지 않게 제약*

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta D_{KL}[\pi_\theta||\pi_{ref}]\right\}$$

Model doesn't deviate too much from the previous one

KL: 값이 너무 달라지지 않게

> *β를 곱한 KL 패널티로 모델이 참조 정책 근처에 머무르게 함*